

IoT-Connected Smart Waste Measurement Prototype

Bacheloroppgave

B26-ITK26

Ole Sander Skjørberg

Omid Mahrofzade

Sander Smedstad

Institutt for informasjonsteknologi og kommunikasjon

Høgskolen i Østfold

Halden

Mai 2026



Forord

Denne rapporten er skrevet som avsluttende bacheloroppgave ved Høgskolen i Østfold, Institutt for informasjonsteknologi og kommunikasjon, våren 2026. Oppgaven er gjennomført av tre dataingeniørstudenter og har resultert i en fungerende IoT-prototype for automatisert matsvinnregistrering.

Vi ønsker å rette en stor takk til vår veileder Ida ved Høgskolen i Østfold for konstruktive tilbakemeldinger, faglige innspill og god oppfølging gjennom hele prosjektperioden. Møtene med deg har vært til stor hjelp, både for rapporten og for de tekniske valgene vi har tatt underveis.

En særlig takk rettes til TotalCtrl for et engasjerende og relevant prosjekt. Dere har bidratt med tilgang til test-API, teknisk dokumentasjon og tett dialog gjennom jevnlig møter. Deres åpenhet og tilgjengelighet har gjort det mulig å utvikle en løsning som er direkte knyttet til et reelt behov i markedet.

Vi vil også takke de to sykehjemmene som tok imot oss og stilte sin tid og erfaring til disposisjon. Innsikten vi fikk gjennom disse besøkene hadde direkte innvirkning på designvalgene i prosjektet, og gjorde løsningen langt mer tilpasset reelle brukerbehov enn den ellers ville vært. Til slutt ønsker vi å takke Høgskolen i Østfold for tilgang til verksteder, 3D-printere og teknisk utstyr som har vært avgjørende for den fysiske delen av prosjektet.

Sammendrag

Denne rapporten dokumenterer utviklingen av en IoT-basert prototype for automatisk registrering av matsvinn, gjennomført som bacheloroppgave ved Høgskolen i Østfold våren 2026, i samarbeid med TotalCtrl. Bakgrunnen for prosjektet er at manuell matsvinnregistrering i travle kjøkkenmiljøer er tidkrevende og fører til ufullstendige data. Målet var å utvikle en løsning som registrerer matsvinn automatisk, uten behov for manuell inntasting.

Prototypen er bygget rundt en Raspberry Pi 4 og består av vektceller, en infrarød nærhetssensor, kamera og mikrofon. Systemet er strukturert rundt en modulær arkitektur med fire kompleksitetsnivåer: vektmåling alene (L1), vektmåling kombinert med talegjenkjenning (L2), vektmåling kombinert med bildegjenkjenning (L3), og en planlagt hybridløsning (L4). For talegjenkjenning benyttes OpenAI Whisper, og for bildegjenkjenning benyttes en YOLOv8n-modell kjørt lokalt på enheten. Registrerte hendelser overføres til TotalCtrl sitt API via REST over HTTPS, med lokal mellomlagring som sikkerhetsnett ved nettverksbrudd.

Prosjektet følger et design science-orientert forskningsdesign, og inkluderer semistrukturerte brukerintervjuer ved to sykehjem. Disse intervjuene hadde direkte innvirkning på valget av modulær arkitektur, da behovene viste seg å variere betydelig mellom ulike kjøkkenmiljøer.

Testingen viser at vektmålingen gir stabile og gjentakbare resultater, med avvik på 5-30 gram avhengig av gjenstandens vekt. API-integrasjonen fungerte som forventet under alle gjennomførte testoverføringer. Bildegjenkjenning med deteksjonsmodellen oppnådde en gjennomsnittlig konfidens på 87 % under gode lysforhold, mens talegjenkjenningen fungerte godt under stille forhold, men var sensitiv for bakgrunnsstøy.

Prototypen er ikke produksjonsklar, men demonstrerer at automatisert matsvinnregistrering er teknisk gjennomførbart med rimelig maskinvare, og gir TotalCtrl et konkret grunnlag for videre utvikling.

Innholdsfortegnelse

Forord	i
Sammendrag	ii
Figurliste	vi
Tabelliste	vii
1 Introduksjon	1
1.1 Prosjektgruppen	1
1.2 Oppdragsgiver	2
1.3 Bakgrunn	2
1.4 Problemstilling	2
1.5 Formål og leveranse	3
1.6 Begrensninger	4
1.7 Rapportens struktur	4
2 Teoretisk grunnlag og analyse	6
2.1 IoT og innebygde systemer	6
2.2 Dataoverføring og pålitelighet i IoT-systemer	8
2.3 Talegjenkjenning	9
2.3.1 OpenAI Whisper	9
2.3.2 Google Cloud Speech to Text (Chirp)	10
2.3.3 Deepgram Nova-3	10
2.3.4 Meta Wav2Vec 2.0	11
2.3.5 Vosk	11
2.4 Objektgjenkjenning	12
2.4.1 YOLO (You Only Look Once)	12
2.4.2 MobileNetV2	13
2.4.3 EfficientNet	14
2.4.4 ViT (Vision Transformer)	14
2.4.5 CLIP (Contrastive Language–Image Pretraining)	14
2.5 Relatert arbeid	15

3	Metode	17
3.1	Forskningsmetodikk	17
3.2	Arbeidsmetodikk	18
3.3	Milepælsplan	20
3.4	Verktøy og teknologier	21
3.5	Kundeundersøkelser	22
4	Planlegging og design	25
4.1	Møter med oppdragsgiver og veileder	25
4.2	Idémyldring	26
4.3	Brukerscenarier og krav	27
4.4	Systemarkitektur	35
4.4.1	Modulær løsningsarkitektur	35
4.5	Hardware-design	37
4.6	Software-design	38
4.7	Protokollvalg	43
5	Utvikling og implementasjon	45
5.1	Hardware-implementasjon	45
5.2	Software-implementasjon	49
5.2.1	Grunnleggende vektmåling (L1)	50
5.2.2	Talegjenkjenning (L2)	51
5.2.3	Bildegjenkjenning (L3)	53
6	Testing og resultater	60
6.1	Testmetodikk	60
6.2	System-simulator	63
6.3	Fysiske tester	65
6.4	Resultater	68
7	Diskusjon	73
7.1	Oppnåelse av prosjektets mål	73
7.2	Arbeidsmetodikk og prosjektgjennomføring	74
7.3	Tekniske utfordringer	76
7.4	Brukerperspektiv	77
7.5	Posisjonering i markedet	78
7.6	Begrensninger	79
7.7	Samlet vurdering	79
8	Konklusjon	80
8.1	Videre arbeid	80
8.2	Oppsummering	83
	Referanseliste	84

A	Teknisk dokumentasjon	87
B	Brukermanual	88
C	Kildekode	89
	Vedlegg A - Teknisk dokumentasjon (engelsk versjon)	A1
	Vedlegg B- Brukermanual	B1

Figurliste

2.1	Prinsippskisse av Wheatstone bridge-konfigurasjon med fire motstander.	7
3.1	Utsnitt av prosjektets GitHub Projects Kanban-tavle, som viser oppgavefordeling mellom kolonnene <i>Todo</i> , <i>In Progress</i> og <i>Done</i> for en av prosjektets faser.	19
4.1	Brukerreise ved talebasert matsvinnregistrering (L2)	33
4.2	Brukerreise ved bildebasert matsvinnregistrering (L3)	34
4.3	Flytskjema som viser de fire kompleksitetsnivåene og dataflyten mellom komponentene.	36
4.4	Svar fra TotalCtrl sitt staging-API etter vellykket testmåling.	41
5.1	Kretsskjema for prototypen, med fire vektceller koblet til HX711, og IR-sensor koblet direkte til Raspberry Pi 4.	46
5.2	Vektcellefestene klargjort for 3D-printing i PrusaSlicer på skolens Original Prusa MK4S.	48
5.3	Hendelsesflyt fra IR-trigger til API-registrering for L1, L2 og L3.	49
6.1	Èn enkelt simulert hendelse hentet fra test-API	64
6.2	Testoppsett med Raspberry Pi 4, IR-sensor og HX711 koblet til et koblingsbrett, med et eksempel på YOLO-deteksjon av en kopp vist øverst til høyre.	67
6.3	En test av bildegjenkjenning med en classification basert modell	70
6.4	En test av bildegjenkjenning med en detection basert modell	71

Tabelliste

2.1	Sammenligning av talegjenkjenningsmodeller vurdert for prosjektet.	12
2.2	Sammenligning av objektgjenkjenningsmodeller vurdert for prosjektet.	15
3.1	Milepælsplan – overordnet faseinndeling	21
4.1	Ønsket funksjonalitet per virksomhetstype	27
4.2	Oversikt over systemets aktører	27
4.3	Sammendrag av brukstilfeller	28
4.4	UC-01: Logg matsvinn med tale	29
4.5	UC-02: Motta bekreftelse	29
4.6	UC-03: Håndter uklar tale	30
4.7	UC-04: Se matsvinndata	30
4.8	UC-05: Kalibrer vekt	31
4.9	UC-06: Håndter nettverksfeil	31
4.10	Sporingsmatrise: Krav til brukstilfeller	35
4.11	Sammenligning av kommunikasjonsprotokoller	44
5.1	Tilkoblinger til Raspberry Pi 4	45
5.2	Kategorier støttet av YOLO-modellen i L3-løsningen	55
6.1	Testresultater for reel vektmåling	69
6.2	Sammenligning av classification- og detection-modell	71

Kapittel 1

Introduksjon

Dette kapitlet introduserer prosjektgruppen, oppdragsgiver og beskriver bakgrunnen for oppgaven. Videre presenteres problemstillingen prosjektet skal undersøke, samt formål og ønsket leveranse. Kapitlet avsluttes med å gi en oversikt over rapportens struktur.

1.1 Prosjektgruppen

Prosjektgruppen består av tre dataingeniørstudenter ved Høgskolen i Østfold (HiØ). Medlemmene har tidligere samarbeidet i andre prosjekter ved HiØ, og bringer med seg god kompetanse fra studieløpet. Videre presenteres hvert medlem med relevant bakgrunn.

Omid Mahrofzade (f. 2002) oppvokst i Son, men opprinnelig fra Kurdistan. Omid har tidligere jobbet som lærling hos Norsk Sykepleierforbund (NSF) i Oslo, innen IT-support og tok fagbrev i 2023. Her lærte han mye om kundeservice, brukerstøtte og hvordan effektivisere prosessen ved å sette seg inn i nye systemer.

Ole Sander Skjørberg (f. 2000) er født og oppvokst i Sarpsborg. Han har fagbrev som dataelektroniker og jobber som Apple-sertifisert tekniker hos Care1, hvor han har vært ansatt i over syv år. Gjennom denne stillingen har han opparbeidet bred erfaring med feilsøking, hardware og kundebehandling. Han har en langvarig interesse for IoT og innebygde systemer. Erfaring fra førstegangstjenesten som nestlagfører, kombinert med deltakelse i flere bedriftsprosjekter i ledende roller, har gitt ham praktisk erfaring med prosjektledelse og koordinering av arbeidsoppgaver.

Sander Smedstad (f. 2002) er født og oppvokst i Lillestrøm. Han har fagbrev som telekommunikasjonsmontør. Han har jobbet og hatt læretiden sin i cyberforsvaret over tre år, før påbegynt studie. I Forsvaret har han opparbeidet seg erfaring rundt brukerstøtte, hendelseshåndtering, feilsøking og installasjon av nye nettverkstekniske lokasjoner.

1.2 Oppdragsgiver

TotalCtrl er et norsk teknologiselskap som leverer løsninger for automatisert kontroll av innkjøp, lager, forbruk og matsvinn. Selskapet tilbyr en SaaS-plattform (Software as a Service) som gir sanntidsinnsikt i drift og ressursbruk, med formål om å redusere kostnader, kutte matsvinn og forenkle bærekrafts- og ESG-rapportering (Environmental, Social, Governance). De opererer i bransjer som inkluderer hotell, kjøkken, sykehjem og andre bedrifter hvor oversikt og håndtering av lager er essensielt. Målgruppen er altså kjøkkensjefer, drift- og bærekraftsansvarlige, samt ledelse i hotell, restauranter, sykehjem eller kantine.

Betydning av prosjektet

Prosjektet er direkte koblet til TotalCtrl sin produktutvikling og deres fremtidige planer. En fungerende prototype vil kunne demonstrere hvordan IoT-teknologi kan eliminere behovet for manuell registrering av matsvinn, noe som i dag er en barriere for konsekvent datainnsamling hos deres kunder.

Engasjement og støtte

TotalCtrl bidrar med tilgang til et test-API uten dokumentasjon, innsikt i deres plattform, samt svar på tekniske spørsmål linket mot deres løsninger underveis i prosjektet. Det er også etablert en struktur med jevnlig oppfølgingsmøter og løpende dialog underveis.

1.3 Bakgrunn

Som gruppe ønsket vi en oppgave som involverte IoT og fysisk prototyping med sensorer og hardware. Da vi satte oss inn i TotalCtrl sitt prosjekt, var dette den oppgaven som umiddelbart fanget interessen vår. Vi så raskt for oss hvordan en slik løsning kunne bygges opp, og fikk et genuint engasjement for å utforske problemstillingen.

Gjennom emner som teknologiprojekt og software engineering har gruppen opparbeidet relevant erfaring med utvikling av sammensatte systemer. De anbefalte teknologiene i prosjektet var i stor grad kjente for oss fra tidligere, noe som ga et godt utgangspunkt for å komme raskt i gang.

TotalCtrl jobber med å automatisere kontroll av innkjøp, lager, forbruk og registrering av matsvinn. En helautomatisk løsning for registrering av matsvinn vil kunne redusere tidsbruk og friksjon for brukerne, samtidig som det sikrer mer pålitelige data. Vi ser for oss at en slik prototype kan ha praktisk verdi og bidra til å forenkle en prosess som i dag oppleves som tung og manuell.

1.4 Problemstilling

TotalCtrl sin plattform for registrering av matsvinn baserer seg i dag på manuell inntasting fra brukerne. I et travelt miljø innebærer dette at ansatte må bruke dyrebar tid til å veie opp avfallet, og registrere type og mengde i systemet. Denne prosessen er tidkrevende og skaper friksjon i arbeidsflyten, noe som kan føre til at registreringer blir unøyaktige, ufullstendige eller uteblir helt.

For at TotalCtrl sine kunder skal få en bedre verdi av plattformen, er de avhengige av konsistent og pålitelig data. Manglende eller unøyaktige registreringer gir over tid et skjevt bilde av det faktiske matsvinnet, noe som svekker grunnlaget for analyse og beslutninger.

Prosjektets problemstilling er derfor:

Hvordan kan IoT-teknologi brukes til å automatisere registrering av matsvinn, slik at data fanges uten behov for manuell inntasting?

For å besvare dette har gruppen definert følgende forskningsspørsmål:

- FS1** Hvilke IoT-teknologier egner seg best for automatisk klassifisering av matavfall i et ressursbegrenset miljø?
- FS2** I hvilken grad kan et IoT-basert system måle og overføre matsvinndata med tilstrekkelig nøyaktighet og pålitelighet for bruk i et reelt kjøkkenmiljø?
- FS3** Hvilke krav stiller et hektisk kjøkkenmiljø til brukergrensesnitt og interaksjonsdesign for at automatisk matsvinnregistrering skal fungere i praksis?
- FS4** Hvordan kan systemarkitekturen utformes slik at løsningen kan skaleres og tilpasses ulike miljøer og behov?

1.5 Formål og leveranse

TotalCtrl ønsker å automatisere registrering av avfall og matsvinn for brukere/kjøkken spesielt innenfor helsesektoren. De ønsker en helautomatisk løsning hvor brukere skal kunne kaste avfall og svinn slik at det automatisk blir registrert hvor mye og hva som er kastet, og deretter lagre dataen slik at de får en oversikt over hvilke og hvor mye av produktene de har igjen. Det er forventet av oss som gruppe å levere en fungerende prototype, som kan vise til funksjonaliteten å gi et godt grunnlag til videre utvikling. For at TotalCtrl skal kunne videreutvikle og levere sluttproduktet til deres kunder er det også forventet at løsningen er skalerbar og intuitivt satt opp.

Prosjektet er strukturert rundt en faseinndelt milepælsplan, utarbeidet i samarbeid med TotalCtrl ved oppstart. Planen definerer forventede leveranser per fase og vil fungere som et overordnet styringsverktøy gjennom prosjektperioden. Konkrete leveranser inkluderer en fungerende prototype som demonstrerer automatisk vektmåling og API-integrasjon mot TotalCtrl sitt system, samt dokumentasjon som gir grunnlag for videre utvikling. Prototypen er bygget rundt en modulær arkitektur med fire kompleksitetsnivåer (L1–L4), som beskrevet i kapittel 4.4.1. En detaljert gjennomgang av milepælsplanen og arbeidsmetodikken er presentert i kapittel 3.3.

1.6 Begrensninger

I ethvert utviklingsprosjekt vil det finnes faktorer som kan påvirke gjennomføringen og det endelige resultatet. For å kunne planlegge realistisk samt ta gode beslutninger underveis, har gruppen identifisert følgende utfordringer som kan oppstå i løpet av prosjektet:

Tidsramme: Prosjektet gjennomføres innenfor en begrenset tidsperiode, noe som kan begrense hvor mange av kompleksitetsnivåene som blir fullstendig implementert og testet.

Hardware: Prototypen bygges med rimelig og tilgjengelig utstyr, ikke industriell hardware. Dette kan medføre begrensninger i ytelse, nøyaktighet og holdbarhet sammenlignet med en produksjonsklar løsning.

Ressurser på mikrokontroller: En mikrokontroller har begrenset prosessorkraft og minne, noe som kan påvirke hvilke operasjoner som kan kjøres lokalt kontra i skyen.

Nettverksforbindelse: Ustabil eller treg nettverksforbindelse på mikrokontrolleren kan påvirke påliteligheten til dataoverføring og responstid fra skybaserte tjenester.

Klassifisering: Matavfall kan ha svært varierende utseende avhengig av tilberedning og tilstand. En most potet ser vesentlig annerledes ut enn en rå potet, noe som gjør automatisk gjenkjenning utfordrende.

Miljøforhold: Et kjøkkenmiljø innebærer støy som kan påvirke talegjenkjenning, samt varierende belysning som kan påvirke bildegjenkjenning.

Treningsdata: Tilgang til representative datasett for trening av AI-modeller kan være begrenset, noe som kan påvirke nøyaktigheten til klassifiseringen.

1.7 Rapportens struktur

Rapporten er organisert i åtte kapitler som følger prosjektets naturlige progresjon fra teoretisk grunnlag til ferdig prototype og evaluering.

Kapittel 2 presenterer det teoretiske grunnlaget for prosjektet. Her gjennomgås sentrale konsepter innen IoT og innebygde systemer, dataoverføring og pålitelighet, samt en sammenligning av aktuelle modeller for talegjenkjenning og objektgjenkjenning. Kapittelet avsluttes med en gjennomgang av eksisterende løsninger innen automatisk matsvinnregistrering.

Kapittel 3 beskriver prosjektets metodikk, både akademisk og praktisk. Her redegjøres det for forskningsdesign, datainnsamlingsmetoder, arbeidsmetodikk, milepælsplan og verktøy som er benyttet. Kapittelet inkluderer også funnene fra kundeundersøkelsene som ble gjennomført ved to sykehjem.

Kapittel 4 presenterer planleggings- og designprosessen. Dette inkluderer idémyldring, brukerscenarier og krav, systemarkitektur med den modulære løsningsmodellen (L1–L4), hardware- og softwaredesign, samt valg av kommunikasjonsprotokoll.

Kapittel 5 beskriver den konkrete utviklingen og implementasjonen av prototypen. Her gjennomgås fysisk montering av hardware, elektriske tilkoblinger, kalibrering av sensorer og den fullstendige software-implementasjonen for alle funksjonsnivåer.

Kapittel 6 presenterer testmetodikk og resultater. Kapitlet beskriver komponenttesting, integrasjonstesting og funksjonell testing, og presenterer målte resultater for vektmåling og bildegjenkjenning.

Kapittel 7 diskuterer prosjektets resultater og erfaringer opp mot de definerte forskningsspørsmålene og suksesskriteriene. Her vurderes også arbeidsmetodikk, tekniske utfordringer, brukerperspektiv og sammenligning med eksisterende løsninger.

Kapittel 8 beskriver anbefalte retninger for videre arbeid og utvikling av løsningen. Avslutningsvis oppsummeres prosjektet og presenterer konklusjoner.

Kapittel 2

Teoretisk grunnlag og analyse

Dette kapitlet presenterer det teoretiske grunnlaget som ligger til grunn for systemet som er utviklet i prosjektet. Først gis en innføring i IoT og innebygde systemer, med fokus på de komponentene og prinsippene som er direkte relevante for prototypen. Deretter beskrives sentrale aspekter ved dataoverføring og pålitelighet i IoT-kontekst. Videre gjennomgås og sammenlignes aktuelle modeller for henholdsvis talegjenkjenning og objektgjenkjenning – begge vurdert opp mot krav som offline-støtte, ressursbruk og egnethet for Raspberry Pi 4. De endelige modellvalgene begrunnes i kapittel 4 på bakgrunn av vurderingene gjort her. Avslutningsvis presenteres relatert arbeid som setter prosjektet i kontekst av eksisterende løsninger.

2.1 IoT og innebygde systemer

Begrepet *Internet of Things* (IoT) refererer til fysiske enheter utstyrt med sensorer, prosesseringskapasitet og nettverkstilkobling, som muliggjør innsamling og utveksling av data uten direkte menneskelig interaksjon [4]. I industri- og næringssammenheng benyttes IoT-systemer blant annet til å automatisere datainnsamling i miljøer der manuell registrering er upraktisk eller upålitelig [18].

Et sentralt konsept i IoT-arkitektur er *edge computing*, der databehandling utføres lokalt på enheten fremfor å sende rå sensordata til skyen [37]. Dette reduserer nettverksbelastning og latenstid, og gjør systemet mer robust ved ustabil nettverksforbindelse. I dette prosjektet utføres all sensoravlesning, klassifisering og hendelseslogikk lokalt på Raspberry Pi, mens kun strukturerte hendelsesdata overføres til TotalCtrl sitt API.

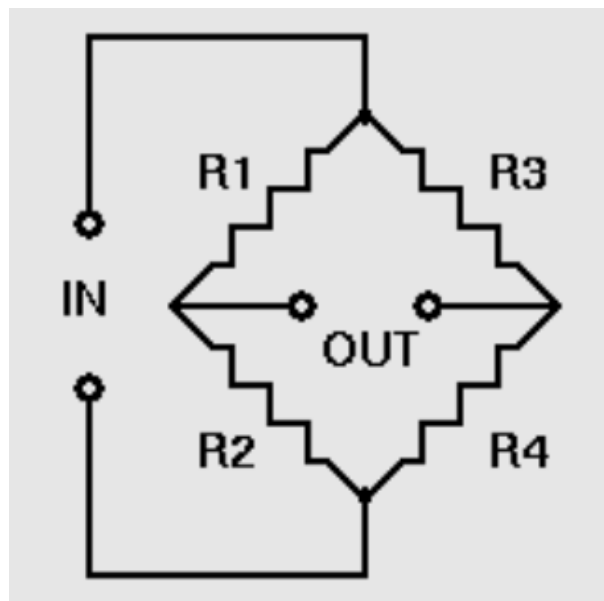
Raspberry Pi som edge-enhet

Raspberry Pi 4 Model B er en enkortsdatamaskin basert på en ARM Cortex-A72-prosessor med 1,8 GHz klokkehastighet og 4 GB RAM [32]. Enheten kjører et fullverdig Linux-basert operativsystem (Raspberry Pi OS) og tilbyr blant annet USB 3.0, kameraport (CSI), og et 40-pins GPIO-grensesnitt for tilkobling av ekstern hardware.

GPIO-grensesnittet (*General Purpose Input/Output*) lar utviklere kommunisere direkte med sensorer og aktuatorer via digitale og analoge signaler. I dette prosjektet benyttes GPIO til kommunikasjon med HX711-forsterkeren, vektceller og IR-sensoren. Raspberry Pi er en mye brukt plattform for IoT-prototyping nettopp fordi den kombinerer tilstrekkelig prosessorkraft for lokale beregninger med et fleksibelt grensesnitt mot ekstern hardware [32]. Hvordan edge computing benyttes i denne prototypen og hvorfor det er nødvendig beskrives videre i 4.6 og 5.2

Vektceller og vektmåling

En vektcelle er en elektro-mekanisk sensor som konverterer mekanisk kraft til et elektrisk signal [13]. Den vanligste typen benytter *strain gauges* – tynne motstandstråder limt på et metalllegeme – som endrer elektrisk motstand når legemet deformeres under belastning. Denne motstandsendringen er svært liten og måles ved hjelp av en *Wheatstone bridge*-kobling, som gir et differensielt spenningssignal proporsjonalt med den pålagte vekten. Figur 2.1 viser prinsippet bak denne koblingen[38].



Figur 2.1: Prinsippskisse av Wheatstone bridge-konfigurasjon med fire motstander.

Råsignalet fra en vektcelle er i størrelsesorden noen få millivolt og må forsterkes og digitaliseres før det kan leses av en mikrokontroller. I dette prosjektet benyttes **HX711**, en dedikert 24-bits analog-til-digital-omformer (ADC) med integrert instrumenteringsforsterker, spesielt utviklet for vektapplikasjoner [5]. HX711 kommuniserer med Raspberry Pi via et enkelt to-leder serielt grensesnitt (klokke og data), og støtter to inngangskanaler med valgbar forsterkning på 32, 64 eller 128 ganger.

For å oppnå pålitelige målinger er korrekt kalibrering avgjørende. Kalibreringsprosessen innebærer to steg: først registreres *offset* (nullpunkt) uten belastning, deretter plasseres en kjent referansevekt og systemet beregner en skaleringskoeffisient. Målinger gjøres typisk som et gjennomsnitt over flere avlesninger for å redusere støy fra vibrasjoner og elektrisk interferens i et kjøkkenmiljø.

IR-sensor som trigger

En infrarød nærhetssensor fungerer ved at en IR-LED sender ut modulert infrarødt lys, som reflekteres av objekter i nærheten og fanges opp av en tilhørende IR-mottaker [13]. Mottakeren er typisk utstyrt med et båndpassfilter som kun reagerer på lys modulert ved en bestemt frekvens – ofte 38 kHz – noe som gjør sensoren robust mot forstyrrelser fra omgivelseslys som sollys og kunstig belysning.

Sensoren returnerer et digitalt TTL-signal (*Transistor-Transistor Logic*) til mikrokontrolleren: lavt signal ved detektert objekt, høyt signal ellers. Dette gjør den enkel å integrere med GPIO-grensesnittet på en Raspberry Pi uten behov for analog-til-digital-konvertering.

I dette prosjektet benyttes IR-sensoren som hendelsestrigger. Når sensoren detekterer at avfall plasseres i beholderen, initieres vektmåling og eventuelt bildeopptak. Dette er mer energi- og prosessoreffektivt enn kontinuerlig polling av vektsensoren, og sikrer at registreringer kun skjer ved faktisk aktivitet.

Kameramodul

For bildebasert klassifisering (L3 og L4) benyttes et USB-kamera tilkoblet Raspberry Pi. Kameraet styres via OpenCV-biblioteket i Python, som tilbyr et standardisert grensesnitt for bildeopptak og -prosessering [7]. En praktisk utfordring ved bildeopptak i kjøkkenmiljøer er bevegelsesuskarphet (også kalt *motion blur*), som kan oppstå dersom lukkertiden er for lang relativt til hastigheten på objekter som kastes. Lukkertiden må derfor balanseres mot behovet for tilstrekkelig lysinnslipp, særlig under varierende belysningsforhold.

2.2 Dataoverføring og pålitelighet i IoT-systemer

En sentral utfordring i IoT-systemer er å sikre at data samlet inn på kanten av nettverket faktisk når frem til den skybaserte plattformen [18]. I motsetning til tradisjonelle programvaresystemer opererer IoT-enheter ofte i miljøer med ustabil nettverksforbindelse, begrenset prosessorkraft og uten mulighet for direkte menneskelig overvåkning. Dette stiller særskilte krav til hvordan dataoverføring designes og feilhåndteres.

Hendelsesbasert dataoverføring

IoT-enheter kan overføre data enten via kontinuerlig sanntidsstrømming eller hendelsesbasert [4]. Ved kontinuerlig overføring sendes målinger med faste tidsintervaller uavhengig av om en hendelse har inntruffet, hvilket kan medføre unødvendig nettverksbelastning og høyt energiforbruk. Hendelsesbasert overføring sender derimot data kun når en definert hendelse inntreffer, for eksempel at en sensor detekterer aktivitet. Dette er mer ressurseffektivt og gir et naturlig samsvar mellom fysiske hendelser og dataposter, noe som er særlig fordelaktig i systemer for matsvinnregistrering der hvert kast representerer én avgrenset hendelse.

Feilhåndtering og lokal mellomlagring

Et robust IoT-system må håndtere situasjoner der nettverksforbindelsen er midlertidig utilgjengelig [37]. Uten mekanismer for lokal mellomlagring vil data som genereres under et nettverksbrudd gå tapt permanent. Et vanlig mønster er å benytte en lokal kø på enheten der hendelser bufres dersom API-kallet feiler. Når forbindelsen gjenopprettes, sendes de bufrede hendelsene i riktig rekkefølge med opprinnelige tidsstempeler, slik at dataintegritet bevares.

Retry-logikk – det vil si automatiske nye forsøk ved feil – bør implementeres med stigende ventetid mellom forsøkene (*exponential backoff*) for å unngå at enheten overbelaster API-et ved gjentatte raske kall [2]. HTTP-statuskoder benyttes til å skille mellom feil som kan løses ved nytt forsøk (nettverksfeil, 5xx-feil fra server) og permanente feil som krever menneskelig intervensjon (4xx-feil som indikerer feil i dataformatet).

Dataintegritet og tidsstempling

For at registrerte hendelser skal ha analytisk verdi må de inneholde et pålitelig tidsstempel som reflekterer når hendelsen faktisk inntraff, ikke når den ble overført [18]. I systemer med lokal mellomlagring er dette særlig viktig: dersom tidsstempelet settes ved overføringstidspunktet vil data fra perioder med nettverksbrudd fremstå som feilregistrert i etterkant. Tidsstempling bør derfor skje lokalt på enheten idet hendelsen registreres.

2.3 Talegjenkjenning

Talegjenkjenning, ofte referert til som Automatic Speech Recognition (ASR), er et område innen maskinlæring og signalbehandling som blir brukt til å konvertere menneskelig tale til maskinlesbar tekst. Moderne ASR-systemer er i hovedsak basert på dype nevrale nettverk og har erstattet tidligere regelbaserte og statistiske metoder.

Et typisk ASR-system tar inn rå lyd fra en mikrofon, som deretter prosesseres til akustiske representasjoner, ofte i form av mel-spectrogrammer eller tilsvarende frekvensbaserte trekk [27]. Disse representasjonene analyseres videre av nevrale nettverk ofte basert på transformer arkitektur [43], for å lære sammenhengen mellom lydsignaler og språklige enheter som fonemer, ord og setninger.

Tradisjonelt besto ASR-systemer av separate komponenter for akustisk modellering og språklig modellering. Moderne ende-til-ende modeller kombinerer disse stegene i én samlet arkitektur, der hele prosessen fra lyd til tekst læres direkte fra data. Dette gir bedre robusthet mot støy og uformell tale.

2.3.1 OpenAI Whisper

Whisper er en ende-til-ende tale til tekst modell utviklet av OpenAI, basert på deep learning [31]. Modellen er designet for robust talegjenkjenning på tvers av språk, aksenter og støynivåer, og er tilgjengelig som åpen kildekode i flere størrelser, fra «tiny» til «large» [29].

Whisper benytter en transformer basert encoder, decoder arkitektur [31]. Rå lyd konverteres først til et mel spectrogram, som er en visuell representasjon av lyd der ulike frekvenser vises over tid på en skala

tilpasset menneskelig persepsjon, som deretter behandles av encoderen for å lære akustiske representasjoner. Decoderen genererer tekst sekvensielt, token for token. Modellen er trent på store mengder variert tale, noe som gir høy robusthet mot støy og uformell tale [31]. I motsetning til tradisjonelle ASR systemer er Whisper en sekvens til sekvens modell og benytter ikke separate akustiske og språklige modeller.

En av Whisper sine største styrker er høy nøyaktighet på naturlig tale, også uten spesifikk tilpasning eller tuning [31]. Modellen støtter både norsk og engelsk og presterer godt i miljøer med bakgrunnsstøy. Samtidig er modellen relativt beregningskrevende, spesielt i medium og large variantene. Whisper har heller ikke innebygd støtte for speaker diarization, som er prosessen av å identifisere hvem som snakker når i et lydopptak hvor det kan være flere talere [3], og er ikke optimalisert for svak hardware.

Ved bruk av tiny eller base modellene kan Whisper kjøres på en Raspberry Pi 4, men dette kan føre til høy latenstid på grunn av begrenset CPU ytelse og manglende GPU akselerasjon, samt kan nøyaktigheten bli redusert ettersom de mindre modellvariantene er trent med færre parameter og lavere gjenkjenningsevne. Dette gjør modellen mindre egnet for kontinuerlig bruk i en ressursbegrenset IoT-kontekst.

2.3.2 Google Cloud Speech to Text (Chirp)

Chirp er Googles nyeste generasjon talegjenkjenningsmodell og benyttes i Google Cloud Speech-to-Text [15]. Modellen er utviklet for kommersielle løsninger med høye krav til nøyaktighet, skalerbarhet og lav responstid.

Chirp er en transformer basert ASR modell som er trent på Googles omfattende og varierte tale datasett [17]. Modellen kjøres utelukkende i skyen og er optimalisert for sanntidsstrømming, lav latenstid og høy tilgjengelighet [15]. Dette gjør den godt egnet for applikasjoner som krever rask respons og høy stabilitet.

Modellen har svært høy presisjon, særlig for korte kommandoer, og støtter norsk språk godt [15]. Samtidig er løsningen avhengig av konstant internettilkobling og tredjepartsinfrastruktur. Det er også begrenset kontroll over modellvalg og dataflyt, noe som kan være en ulempe i løsninger med strenge krav til personvern eller offline drift.

Chirp benyttes i dag blant annet i Google Assistant, kundesenter-løsninger og ulike smarthus systemer [15].

2.3.3 Deepgram Nova-3

Deepgram Nova-3 er en kommersiell, skybasert ASR-modell utviklet av Deepgram, optimalisert for sanntids talegjenkjenning [8]. Modellen er spesifikt trent på tale og har fokus på sanntidsstrømming, korte kommandoer og rask transkribering.

Nova-3 har lav latenstid og høy presisjon for korte setninger, noe som gjør den godt egnet for kommando-baserte systemer og keyword spotting [9]. Modellen tilbyr ferdige API-er som gjør integrasjon relativt enkel.

Ulempen med Nova-3 er at den krever skytilgang og kommersiell lisens for bruk. Dette kan medføre både kostnader og avhengighet av ekstern infrastruktur.

I dag brukes Deepgram Nova-3 blant annet i talebaserte roboter, samtaleanalyser og ulike IoT-løsninger [9].

2.3.4 Meta Wav2Vec 2.0

Wav2Vec 2.0 er en open-source tale-representasjonsmodell utviklet av Meta. Modellen benytter selv-supervisert læring, der rå lyd først prosesseres av en feature encoder før en transformer lærer å predikere maskerte deler av signalet. Etter forhåndstrening kan modellen fin-tunes til tale-til-tekst-oppgaver [6].

En styrke ved Wav2Vec 2.0 er muligheten for videre trening på egne datasett, noe som gir stor fleksibilitet og potensial for tilpasning til spesifikke domener [6]. Samtidig krever dette relativt høy kompetanse innen maskinlæring, og modellen er ikke en ferdig “plug-and-play”-løsning. Det finnes heller ingen komplett ferdig pipeline tilsvarende kommersielle ASR-tjenester.

Wav2Vec 2.0 benyttes i dag hovedsakelig innen forskning, akademia og i tilpassede ASR-systemer utviklet av større aktører.

2.3.5 Vosk

Vosk er et open-source talegjenkjenningsbibliotek som kjører fullt offline og er spesielt designet for bruk på ressursbegrensede enheter som Raspberry Pi og Android-systemer [1]. Modellen er basert på Kaldi-arkitekturen og tilbyr både kontinuerlig transkribering og støtte for tilpasset ordliste og vokabular.

Vosk benytter kompakte, forhåndstrengte ASR-modeller der lyd mates inn som et kontinuerlig signal, og en decoder analyserer akustiske trekk og returnerer transkribert tekst i sanntid [1]. Dette gir lav latenstid og relativt lavt ressursforbruk.

Fordelen med Vosk er at modellen fungerer helt offline, har lav CPU-bruk og er enkel å integrere i prototyper [1]. Ulempen er at nøyaktigheten generelt er lavere enn for større transformer-baserte modeller [43], spesielt i støyete omgivelser eller ved fri tale.

Vosk brukes i dag blant annet i offline stemmestyrte systemer, hjemmeroboter, IoT-enheter og automatisk tale-til-tekst-logging [1].

Sammenligning av talegjenkjenningsmodeller

Tabell 2.1 gir en oversikt over de vurderte talegjenkjenningsmodellene sett opp mot kriterier som er særlig relevante for dette prosjektet. Ettersom prototypen kjører på en Raspberry Pi 4 i et kjøkkenmiljø med potensielt ustabil nettverksforbindelse, er egenskaper som offline-støtte, ressursforbruk og norsk språkstøtte vektlagt i vurderingen.

Valget av talegjenkjenningsmodell for prototypen begrunnes i 4.6, basert på vurderingene i tabell 2.1

Tabell 2.1: Sammenligning av talegjenkjenningsmodeller vurdert for prosjektet.

Modell	Type	Offline	Norsk støtte	Pi 4-egnet	Lisens
OpenAI Whisper	Open source	Ja	God	Delvis	MIT
Google Chirp	Kommersiell	Nei	God	Ja (sky)	Proprietær
Deepgram Nova-3	Kommersiell	Nei	Delvis	Ja (sky)	Proprietær
Meta Wav2Vec 2.0	Open source	Ja	Delvis	Nei	MIT
Vosk	Open source	Ja	Delvis	Ja	Apache 2.0

2.4 Objektgjenkjenning

Image recognition, eller bildeanalyse, omfatter metoder innen maskinlæring og dyp læring som har som formål å analysere og tolke visuelt innhold i digitale bilder [24]. Moderne image recognition-systemer er i stor grad basert på nevrale nettverk, spesielt konvolusjonelle nevrale nettverk (CNN) [22], som er i stand til å lære hierarkiske trekk fra bilder, slik som kanter, former og mer komplekse strukturer. Konvolusjoner er en matematisk operasjon som bruker filtre på bilder for å trekke frem lokale mønstre og egenskaper og utgjør grunnlaget for hvordan CNN prosesserer visuell informasjon.

Innen image recognition finnes det flere ulike tilnærminger, avhengig av hvilken type informasjon som ønskes ut fra bildet. Image classification er den enkleste formen, der hele bildet tilordnes én eller flere klasser, for eksempel ved å avgjøre hvilket type objekt bildet inneholder [22]. Denne tilnærmingen gir ingen informasjon om hvor i bildet objektet befinner seg, kun hva bildet representerer som helhet.

Objektgjenkjenning utvider dette ved å kombinere klassifisering med lokalisering [14]. I tillegg til å identifisere hvilke objekter som er til stede i bildet, bestemmer objektgjenkjenningsmodeller også posisjonen til hvert objekt ved hjelp av avgrensningsbokser (bounding boxes). Dette gjør object detection spesielt egnet for applikasjoner der flere objekter kan forekomme samtidig, og der romlig informasjon er viktig.

En tredje kategori er representation- og multimodale modeller, som lærer semantiske representasjoner av bilder, ofte i kombinasjon med andre datatyper som tekst [30]. Disse modellene projiserer visuelle og tekstlige data til et felles representasjonsrom, noe som muliggjør fleksible oppgaver som tekstbasert bildeklassifisering og zero-shot-gjenkjenning uten behov for omfattende treningsdata.

Valg av image recognition-modell avhenger av både applikasjonens krav og tilgjengelige maskinvareresurser. I ressursbegrensede systemer, som innebygde systemer og edge-enheter, er det spesielt viktig å vurdere balansen mellom nøyaktighet, beregningskostnad og sanntidsytelse [39].

2.4.1 YOLO (You Only Look Once)

YOLO (You Only Look Once) er en modellfamilie for objektgjenkjenning som benyttes til å oppdage og klassifisere objekter i sanntid [33]. Modellen produserer som output både klasser og tilhørende avgrensningsbokser (bounding boxes) for hvert identifiserte objekt i et bilde.

YOLO-modeller mottar et bilde som input og behandler hele bildet i én enkelt fremoverpassing (forward pass) gjennom et nettverk (CNN). Denne arkitekturen skiller seg fra tradisjonelle to-steps detek-

sjonsmetoder [14] ved at både lokalisering og klassifisering utføres samtidig, noe som bidrar til høy prosesseringshastighet. Bildet deles konseptuelt inn i et rutenett (grid), der hver celle predikerer et sett med avgrensningsbokser samt sannsynligheter for tilhørende klasser [33].

Det finnes flere versjoner av YOLO, blant annet YOLOv3, YOLOv5 og YOLOv8 [21], hvor nyere varianter generelt tilbyr bedre nøyaktighet og mer effektiv ressursbruk enn eldre versjoner. YOLO er særlig kjent for sin gode balanse mellom deteksjonsnøyaktighet og beregningshastighet, og benyttes derfor ofte i sanntidsapplikasjoner. Modellen er optimalisert for bruk på både kraftige systemer og ressursbegrensede edge-enheter.

En ulempe med YOLO er at fullstørrelsesmodeller kan være krevende å kjøre på maskinvare med begrensede ressurser, slik som en Raspberry Pi 4, på grunn av begrenset CPU-ytelse og tilgjengelig minne. I tillegg kan modellen ha utfordringer med å gjenkjenne små objekter dersom bilde kvaliteten eller oppløsningen er lav. For spesialiserte bruksområder kan det derfor være nødvendig med ytterligere trening (fine-tuning) på et domene-spesifikt datasett.

I dag benyttes YOLO i en rekke praktiske anvendelser, blant annet innen automatisering av kassefrie butikker, videoovervåkning, robotikk og bildebasert kvalitetskontroll i industriell produksjon. Disse bruksområdene demonstrerer modellens egnethet for sanntids objektgjenkjenning i operative systemer.

2.4.2 MobileNetV2

MobileNetV2 er en lettvekts konvolusjonell nevral nettverksarkitektur (CNN) utviklet spesielt for ressursbegrensede enheter som mobiltelefoner og edge-enheter [35]. Modellen er designet med fokus på lav beregningskostnad og effektiv ressursbruk, samtidig som den opprettholder akseptabel nøyaktighet for bildeklassifisering.

Arkitekturen i MobileNetV2 bygger på såkalte «inverted residual blocks» og dybde-separable konvolusjoner (depthwise separable convolutions), hvor inverted residual blocks er byggeblokker hvor informasjon blir ekspandert til høyere dimensjoner, prosesseres og deretter komprimert tilbake, mens dybde-separable konvolusjoner splitter standard konvolusjoner i to enklere operasjoner for å redusere beregningskompleksiteten [20, 35]. Denne kombinasjonen reduserer antall parametere og beregninger betydelig sammenlignet med tradisjonelle CNN-arkitekturer. Som følge av dette oppnår modellen høy inferenshastighet og lavt minneforbruk.

MobileNetV2 gir et godt kompromiss mellom nøyaktighet og ressursbruk, og benyttes ofte som en rygggrad i objektgjenkjenningsmodeller, for eksempel i kombinasjon med SSD (Single Shot Detector) [25]. Modellen er derfor godt egnet for bruk på plattformer som Raspberry Pi 4. Samtidig er nøyaktigheten generelt lavere enn for større og mer komplekse modeller, og MobileNetV2 egner seg best til bildeklassifisering fremfor objektgjenkjenning alene.

I praksis benyttes MobileNetV2 i en rekke applikasjoner, blant annet i mobilkamera-API-er, bildeklassifisering på edge-enheter og integrasjon med TensorFlow Lite [40]. Modellen brukes også ofte som grunnarkitektur i lette objektgjenkjenningssystemer.

2.4.3 EfficientNet

EfficientNet er en familie av nevrale nettverksmodeller utviklet for bildeklassifisering, og er basert på prinsippet om compound scaling [39]. Denne tilnærmingen forbedrer modellens ytelse ved å balansere nettverkets dimensjoner: dybde, bredde og bildeoppløsning på en systematisk måte, i stedet for å øke én av disse dimensjonene isolert.

EfficientNet-modellene, som strekker seg fra EfficientNet-B0 til EfficientNet-B7, tilbyr høy nøyaktighet i forhold til modellstørrelse og antall parametere [39]. Dette gjør dem mer effektive enn mange tradisjonelle CNN-arkitekturer med tilsvarende ressursbruk. De mindre variantene, som B0 og B1, er spesielt utviklet for å kunne benyttes på systemer med begrensede ressurser.

En ulempe med EfficientNet er at modellene fortsatt kan være relativt krevende å kjøre på enheter som Raspberry Pi 4 uten ytterligere optimalisering, for eksempel gjennom kvantisering eller bruk av TensorFlow Lite [40]. Sammenlignet med MobileNetV2 krever EfficientNet generelt mer beregningskraft, men gir til gjengjeld høyere klassifiseringsnøyaktighet[35].

EfficientNet benyttes i dag i industrielle applikasjoner, skybaserte klassifiseringstjenester og edge-installasjoner via TensorFlow Lite [40]. Modellfamilien er også mye brukt i forskningssammenheng der høy nøyaktighet er et sentralt krav.

2.4.4 ViT (Vision Transformer)

Vision Transformer (ViT) er en modellarkitektur for bildeklassifisering som benytter transformer-teknologi [10], opprinnelig utviklet for naturlig språkprosessering [43]. I stedet for konvolusjoner deles bildet opp i mindre bildeflater (patches), der hver patch behandles som et token på tilsvarende måte som ord i en tekstsekvens.

Disse tokenene behandles videre gjennom transformerblokker som benytter selvoppmerksomhetsmekanismer (self-attention) for å modellere globale sammenhenger i bildet [10]. Dette gjør ViT spesielt godt egnet til å fange helhetlig kontekst i komplekse bilder, noe som kan gi høy nøyaktighet ved bildeklassifisering.

En viktig fordel med ViT er modellens sterke generaliseringsevne, særlig når den forhåndstrenes på svært store datasett [10]. Samtidig er transformer-baserte modeller generelt mer beregningskrevende enn tradisjonelle CNN-er [22], både når det gjelder minne og prosessorkapasitet. Dette gjør ViT lite egnet for direkte bruk på ressursbegrensede enheter som Raspberry Pi 4 uten maskinvareakseleratorer.

I dag benyttes ViT hovedsakelig i skybaserte API-er, forskningsprosjekter og avanserte maskinsynssystemer. For edge-enheter anvendes modellen normalt kun i kombinasjon med dedikerte akseleratorer.

2.4.5 CLIP (Contrastive Language–Image Pretraining)

CLIP (Contrastive Language–Image Pretraining) er en multimodal modell utviklet for å lære felles representasjoner av bilder og tekst [30]. Modellen trenes ved å projisere både bilder og tekstbeskrivelser til et felles embedding-rom, der semantisk relaterte bilde- og tekstoppar får lignende representasjoner.

Denne tilnærmingen muliggjør såkalt zero-shot-klassifisering, der modellen kan klassifisere bilder basert på tekstlige beskrivelser uten å være eksplisitt trent på det aktuelle datasettet [30]. Dette gir stor fleksibilitet og reduserer behovet for omfattende merket treningsdata.

Til tross for disse fordelene er CLIP en relativt stor og beregningskrevende modell, noe som gjør den lite egnet for bruk på en Raspberry Pi 4 uten betydelig maskinvareakselerasjon. Inferens krever både høy prosessorkapasitet og tilstrekkelig minne, spesielt ved bruk av større modellvarianter.

CLIP benyttes i dag innen multimodalt søk, innholdsmatching og som komponent i ulike AI-assistenter [30]. Modellen anvendes hovedsakelig i skybaserte løsninger og forskningsmiljøer.

Sammenligning av objektgjenkjenningsmodeller

Tilsvarende tabell 2.1 gir tabell 2.2 en oversikt over de vurderte objektgjenkjenningsmodellene. Her er sanntidsytelse og egnethet for edge-enheter sentrale kriterier, ettersom klassifisering av matavfall må skje lokalt og raskt nok til ikke å hindre arbeidsflyten i kjøkkenet.

Tabell 2.2: Sammenligning av objektgjenkjenningsmodeller vurdert for prosjektet.

Modell	Oppgave	Pi 4-egnet	Sanntid	Lisens
YOLO (v8)	Objektgjenkjenning	Ja (nano/small)	Ja	AGPL-3.0
MobileNetV2	Bildeklassifisering	Ja	Ja	Apache 2.0
EfficientNet	Bildeklassifisering	Delvis (B0/B1)	Delvis	Apache 2.0
ViT	Bildeklassifisering	Nei	Nei	Apache 2.0
CLIP	Multimodal/zero-shot	Nei	Nei	MIT

Valget av objektgjenkjenningsmodell for prototypen begrunnes i 4.6, basert på vurderingene i tabell 2.2

2.5 Relatert arbeid

De siste årene har det vært økende fokus på digitalisering av matsvinnregistrering i profesjonelle kjøkken. Løsninger basert på sensorer, bildeanalyse og maskinlæring har gjort det mulig å automatisere deler av denne prosessen. Dette delkapittelet presenterer relevante eksisterende løsninger og setter dem i sammenheng med prototypen som er utviklet i dette prosjektet.

Winnow Vision

Winnow er et av de mest kjente systemene for automatisk matsvinnregistrering i profesjonelle kjøkken [44]. Systemet kombinerer vektorer, kameraer og KI-basert bildeanalyse for å identifisere og registrere matavfall uten manuell brukerinteraksjon. Når avfall kastes, tas det automatisk bilde, og en trent modell klassifiserer matvaren og registrerer mengden. Dataene gir kjøkkenet innsikt i når og hva som kastes.

Ifølge leverandøren kan systemet bidra til opp til 50 % reduksjon i matavfall og 3–8 % reduksjon i innkjøpskostnader [44]. På grunn av den avanserte teknologien og kostnadsnivået er Winnow Vision primært rettet mot større kommersielle kjøkken som restauranter, hoteller og kantiner, og er mindre tilgjengelig for organisasjoner med begrensede ressurser.

Leanpath

Leanpath er en annen sentral aktør innen digital matsvinnregistrering i profesjonelle kjøkken [23]. Systemet kombinerer digitale vekter, kameraer og skybasert analyseprogramvare for å registrere og kategorisere matavfall. Dataene brukes til å generere rapporter og analyser som hjelper organisasjoner med å identifisere årsaker til matsvinn og iverksette forebyggende tiltak.

En sentral styrke ved Leanpath er at systemet ikke bare samler inn data, men også gir konkrete anbefalinger om hvordan kjøkkenet kan redusere svinn gjennom bedre planlegging og produksjon [23]. Dette gjør løsningen særlig nyttig for større organisasjoner der systematisk rapportering er viktig for å nå bærekraftsmål. Leanpath benyttes primært innen hotell-, restaurant- og cateringbransjen.

Andre IoT-baserte systemer

Utover kommersielle løsninger som Winnow og Leanpath finnes det flere forskningsbaserte tilnærminger til automatisk matgjenkjenning. Rokhva mfl. demonstrerer at lettvektsmodeller som MobileNetV2 kan benyttes til nøyaktig matgjenkjenning i sanntid, selv med begrenset tilgang til kraftig hardware [34]. Studien benytter Food11-datasettet med over 16 000 bilder og viser at forhåndstreinte modeller kombinert med teknikker som transfer learning og dataaugmentering gir god ytelse uten behov for store beregningsressurser. Dette underbygger at bildebasert klassifisering av matavfall er teknisk gjennomførbart på ressursbegrensede enheter som Raspberry Pi 4.

Mange av disse forskningsprosjektene er imidlertid fortsatt på prototypestadiet og ikke fullt utviklet for praktisk bruk i reelle kjøkkenmiljøer.

Sammenlikning med vår løsning

Til tross for ulike tilnærminger deler Winnow, Leanpath og vår løsning det samme overordnede målet: å redusere matsvinn gjennom automatisert datainnsamling og analyse. Alle tre systemene benytter sensorer og maskinlæring for å registrere og kategorisere avfall, og alle er rettet mot profesjonelle kjøkkenmiljøer.

Den viktigste forskjellen ligger i kompleksitet, målgruppe og tilgjengelighetsnivå. Winnow og Leanpath er utviklet for større kommersielle aktører og krever betydelige investeringer i både maskinvare og infrastruktur. Vår løsning skiller seg fra dette ved å være bygget rundt en modulær arkitektur med fire kompleksitetsnivåer beskrevet i avsnitt 4.4.1, der hvert nivå kan tas i bruk uavhengig av de andre. Dette betyr at en organisasjon kan starte med kun automatisk vektmåling og API-integrasjon, og gradvis utvide med tale- eller bildegjenkjenning etter behov og budsjett.

Dette gjør løsningen mer tilgjengelig for mindre organisasjoner, som sykehjem og kantiner, der behovet er reelt men ressursene begrensede – noe som ble bekreftet gjennom kundeundersøkelsene i avsnitt 3.5. Samtidig gir den modulære strukturen rom for videre utvikling, slik at løsningen kan vokse i takt med brukerens behov uten at hele systemet må redesignes, som beskrevet i avsnitt 8.1. Prototypen er dermed ikke ment som en direkte konkurrent til Winnow eller Leanpath, men som et alternativ rettet mot et segment disse løsningene i liten grad dekker.

Kapittel 3

Metode

Dette kapitlet beskriver hvordan prosjektet er gjennomført, både metodisk og praktisk. Først redegjøres det for det akademiske forskningsdesignet og hvilke metoder som er benyttet for datainnsamling og evaluering. Deretter beskrives arbeidsmetodikken gruppen har fulgt gjennom prosjektperioden, inkludert prosjektstyring, kommunikasjon og dokumentasjon. Avslutningsvis presenteres milepælsplanen, verktøyene som er benyttet, samt funnene fra kundeundersøkelsene som har hatt direkte innvirkning på designvalgene i prosjektet.

3.1 Forskningsmetodikk

Prosjektet er gjennomført som anvendt forskning, der målet er å utvikle en fungerende prototype som løser et konkret problem for oppdragsgiver. Tilnærmingen er derfor praktisk og løsningsorientert, men støtter seg på etablerte forskningsmetoder for datainnsamling og evaluering.

Forskningsdesign

Prosjektet følger et *design science*-orientert forskningsdesign, der ny kunnskap bygges opp gjennom å utvikle og teste en konkret løsning i praksis, i dette tilfellet en IoT-prototype. Dette innebærer at forskningsspørsmålene besvares gjennom en kombinasjon av teoretisk grunnlag, praktisk utvikling og systematisk testing, snarere enn gjennom tradisjonell hypotesetesting.

Kvalitative metoder

For å kartlegge brukerbehov og kontekst ble det gjennomført semistrukturerte intervjuer med ansatte ved to sykehjem. Denne metoden ble valgt fordi den gir rom for åpen dialog og rike beskrivelser av praksis, samtidig som sentrale temaer til intervjuet dekkes [42]. Intervjuene ga innsikt i nåværende arbeidspraksis, opplevde utfordringer og holdninger til automatisering og ny teknologi. Dataene fra intervjuene er hovedsakelig anvendt for å besvare **FS3** (brukervennlighet) og har direkte påvirket designvalgene beskrevet i kapittel 4.4.1.

I tillegg er det gjennomført en systematisk litteraturstudie for å kartlegge relevante teknologier innen talegjenkjenning og objektgjenkjenning. Litteraturen er hentet fra anerkjente kilder som IEEE, arXiv og offisiell API-dokumentasjon, og danner det teoretiske grunnlaget for teknologivalgene som besvarer **FS1** (IoT-teknologier).

Kvantitative metoder

Evalueringen av prototypen baserer seg på kvantitative målinger av tre sentrale ytelsesparametere:

- **Vektmålingsnøyaktighet:** Avvik mellom målt og faktisk vekt, uttrykt i gram ($\pm$ gram).
- **API-pålitelighet:** Andel vellykkede dataoverføringer til TotalCtrl sitt API, uttrykt som prosentandel av totalt antall forsøk.
- **Kategoriseringsnøyaktighet:** Andel korrekt kategoriserte hendelser, uttrykt som prosentandel av totalt antall registreringer.

Disse målingene er utformet for å besvare **FS2** (pålitelighet og nøyaktighet), og gjennomføres som beskrevet i kapittel 6.

Suksesskriterier

For at prototypen skal anses som vellykket innenfor prosjektets rammer, er følgende kriterier lagt til grunn:

- Vektmåling med avvik på maksimalt ± 5 gram under kontrollerte forhold.
- Vellykket API-overføring i minst 95 % av forsøkene under stabil nettverksforbindelse.
- Kategoriseringsnøyaktighet på minst 80 % for de vanligste avfallstypene.

Den modulære arkitekturen (L1–L4) adresserer **FS4** (Skalering) og evalueres kvalitativt gjennom en vurdering av hvor godt systemet lar seg tilpasse og utvide, basert på erfaringer fra utviklings- og testfasen.

3.2 Arbeidsmetodikk

Prosjektgruppen benyttet en hybrid arbeidsmodell som kombinerer elementer fra Scrum og Kanban. Valget ble tatt fordi gruppen var i jevnlig dialog med en ekstern oppdragsgiver, noe som innebar løpende endringer i krav og prioriteringer. Kombinasjonen ga strukturert planlegging i korte perioder samtidig som den sikret en kontinuerlig flyt av oppgaver.

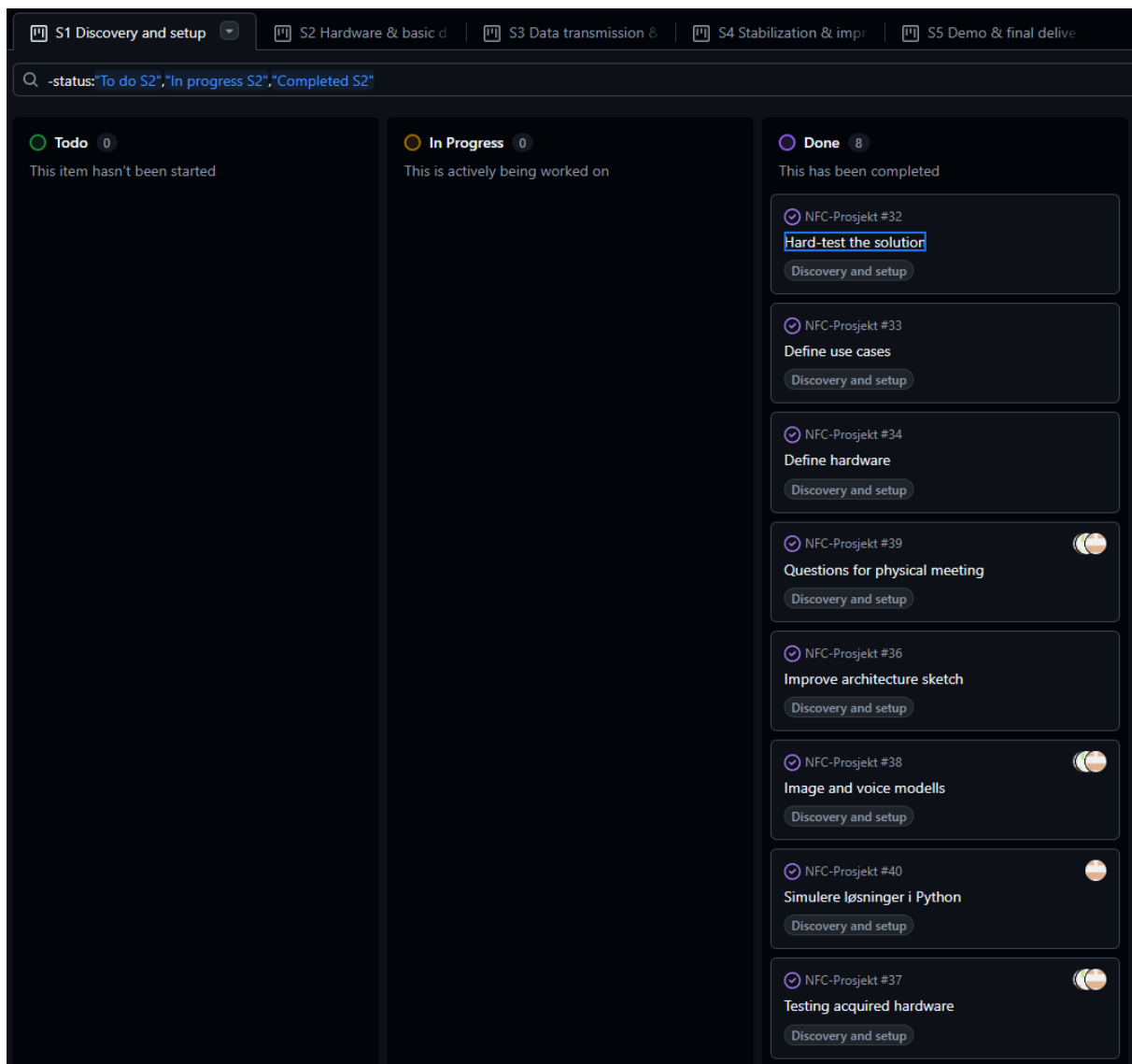
Scrum-inspirert tilnærming

Av Scrum-elementer ble følgende praksis fulgt gjennom prosjektet: ukentlige møter med oppdragsgiver der gruppeleder presenterte status og avklarte prioriteringer for neste periode, korte arbeidsiterasjoner tilsvarende sprinter der gruppen planla hvilke oppgaver som skulle løses den påfølgende uken, og interne samtaler i gruppen der erfaringer fra forrige periode ble diskutert med tanke på forbedringer fremover.

Gruppen fulgte ikke Scrum fullt ut etter metodens definisjon, men prinsipper som hyppige tilbakemeldinger, iterativ utvikling og løpende forbedringer var sentrale gjennom hele prosjektperioden.

Kanban-håndtering

Som oppgavesystem ble GitHub Issues og GitHub Projects benyttet. Dette ga en god visuell oversikt over oppgaver fordelt på kolonnene *Todo*, *In Progress* og *Done*. Systemet gjorde det enkelt å fordele arbeidsoppgaver mellom gruppemedlemmene og ga en fleksibel arbeidsflyt uten behov for faste sprinter. For et prosjekt med varierende oppgavetyper bidro dette til mer effektiv koordinering. Figur 3.1 viser et utsnitt fra en av prosjektets faser.



Figur 3.1: Utsnitt av prosjektets GitHub Projects Kanban-tavle, som viser oppgavefordeling mellom kolonnene *Todo*, *In Progress* og *Done* for en av prosjektets faser.

Dokumentasjon

Prosjektet krevde ukentlige og månedlige statusrapporter til oppdragsgiver. Rapportene fulgte en fast struktur og inkluderte status på utviklingen, eventuelle utfordringer gruppen hadde møtt, endringer i krav eller prioriteringer, spørsmål til oppdragsgiver og plan for kommende periode.

Den faste strukturen bidro til at viktig informasjon ikke gikk tapt, og at både gruppen og oppdragsgiver alltid hadde et oppdatert bilde av prosjektets tilstand. Regelmessig rapportering fungerte også som en intern disiplin, der kravet om å dokumentere fremdrift bidro til jevnere arbeidsinnsats gjennom prosjektperioden.

En ulempe var at rapporteringen tidvis opplevdes som tidkrevende, særlig i perioder med høy arbeidsbelastning. I faser der det skjedde lite nytt kunne det også være utfordrende å fylle rapportene med meningsfullt innhold.

Kommunikasjon

Prosjektgruppen benyttet flere kommunikasjonsplattformer gjennom prosjektperioden, tilpasset de ulike behovene og partene involvert.

For intern kommunikasjon var Discord hovedkanalen, brukt til planlegging av fysiske møter, avklaringer og løpende dialog. I tillegg møttes gruppen fysisk én til to ganger i uken for å vise frem fremgang, diskutere prioriteringer og jobbe praktisk sammen.

For ekstern kommunikasjon ble Outlook benyttet til e-post og kalenderplanlegging mot oppdragsgiver og veileder. Møter med veileder ble gjennomført via Microsoft Teams, mens møter med TotalCtrl ble holdt via Google Meet etter deres ønske.

TotalCtrl satte i tillegg opp en delt Google Drive-mappe for asynkron kommunikasjon. Her ble ukentlige og månedlige statusrapporter lagt inn, og oppdragsgiver opprettet et eget regneark der gruppen kunne stille spørsmål som ikke krevde umiddelbart svar. Dette fungerte som et nyttig supplement til e-post og møter.

3.3 Milepælsplan

Ved oppstart av prosjektet ble det i samarbeid med TotalCtrl utarbeidet en overordnet milepælsplan fordelt over fem faser. Planen fungerte som et felles utgangspunkt for forventninger til fremdrift og leveranser, og ble behandlet som en retningslinje snarere enn en rigid tidsplan. Underveis i prosjektet ble enkelte prioriteringer justert, særlig som følge av innsikten fra brukerintervjuene, som førte til at gruppen valgte en mer modular løsningsarkitektur enn opprinnelig skissert. Dette gjenspeiles også i arbeidsmetodikken beskrevet i kapittel 3.2, der kontinuerlig tilpasning og tett dialog med oppdragsgiver har vært sentrale prinsipper.

Tabell 3.1 gir en oversikt over de fem fasene med tilhørende tidsramme og forventede leveranser.

Tabell 3.1: Milepælsplan – overordnet faseinndeling

Fase	Uker	Leveranse
Fase 1: Kartlegging og oppsett	Uke 1–2	Arkitekturskisse og teknisk plan. Avklaring av API-krav, hardware-valg og brukstilfeller.
Fase 2: Hardware og datainnsamling	Uke 3–5	Fungerende hardware-prototype med lokal datainnsamling fra vektsensor.
Fase 3: Dataoverføring og integrasjon	Uke 6–8	Live dataflyt til TotalCtrl sitt testmiljø via REST. Feilhåndtering og validering av datakvalitet.
Fase 4: Stabilisering og forbedring	Uke 9–10	Stabil prototype med ryddig kodestruktur, kanttilfelle-håndtering og oppdatert dokumentasjon.
Fase 5: Demo og slutt-leveranse	Uke 11–12	Demovideo, presentasjon og fullstendig teknisk dokumentasjon.

3.4 Verktøy og teknologier

Dette delkapittelet gir en oversikt over verktøyene og teknologiene gruppen har benyttet gjennom prosjektperioden. Hardware-komponenter og programvarebiblioteker knyttet direkte til prototypen beskrives i kapittel 4.

Prosjektstyring og samarbeid

GitHub er brukt som hovedverktøy for prosjektstyring. Via GitHub Issues og GitHub Projects har gruppen holdt oversikt over oppgaver og fremdrift, som beskrevet i avsnitt 3.2. Overleaf er benyttet som skriveplattform for rapporten, med delt tilgang for alle gruppelemmene.

Utvikling

Python er benyttet som programmeringsspråk for all programvare i prosjektet. Valget ble gjort på bakgrunn av god bibliotekstøtte for maskinlæring, GPIO-kommunikasjon og HTTP-integrasjon, samt gruppens samlede kompetanse med språket.

Utviklingen foregikk i virtuelle Python-miljøer (venv) for å holde avhengigheter isolert og reproducerbart på tvers av maskiner. For L3-løsningen ble det opprettet et separat virtuelt miljø for bildegjenkjenning, da bibliotekene som kreves her ikke lot seg kombinere med resten av systemet uten konflikter. Dette er nærmere beskrevet i avsnitt 5.2.3.

Sentrale biblioteker benyttet i prosjektet inkluderer `hx711` for kommunikasjon med vektcelle-forsterkeren, `RPi.GPIO` for GPIO-styring, `requests` for HTTP-kommunikasjon mot TotalCtrl sitt API, `OpenCV` for bildeopptak og bildehåndtering, `onnxruntime` for inferens av YOLO-modellen, samt `PyAudio` og `OpenAI Whisper` for lydopptak og talegjenkjenning.

AI-modeller

For talegjenkjenning benyttes OpenAI Whisper med base-modellen, som kjøres lokalt på Raspberry Pi. For bildegjenkjenning benyttes en YOLOv8n-modell eksportert til ONNX-format (Open Neural Network

Exchange), trent på et deteksjonsdatasett fra Roboflow. Begrunnelsen for disse valgene er beskrevet i avsnitt [4.6](#).

Design og produksjon

For mekanisk design av monteringsfester til vektcellene er Autodesk Fusion 360 benyttet som modelleringsverktøy. Modellene er klargjort for 3D-printing i PrusaSlicer og skrevet ut på Høgskolen i Østfolds Prusa-printere. Dette er nærmere beskrevet i avsnitt [5.1](#).

3.5 Kundeundersøkelser

For å sikre at prototypen møter reelle brukerbehov, besøkte prosjektgruppen to sykehjem som skal benytte TotalCtrl sine løsninger for matsvinnregistrering.

Formålet med undersøkelsene var tredelt:

1. Kartlegge nåværende praksis for avfallshåndtering
2. Identifisere utfordringer med eksisterende løsninger
3. Samle innsikt som kan være med å forme designvalg for prototypen

Metode for datainnsamling

Undersøkelsene ble gjennomført som semistrukturerte intervjuer kombinert med observasjon. Denne metoden ble valgt da den gir rom for åpen dialog samtidig som den sikrer at sentrale temaer blir dekket. Det var viktig for gruppen å kunne følge opp interessante svar og la deltakerne utdype sine erfaringer og behov. Prosjektgruppen besøkte to sykehjem, heretter omtalt som sykehjem A og sykehjem B. Begge institusjonene er eksisterende kunder av oppdragsgiver og representerer typiske brukere av matsvinnregistreringssystemer i helsesektoren.

Ved sykehjem A ble det gjennomført en guidet omvisning gjennom kjøkken og avdelinger, etterfulgt av intervjuer med nøkkelpersonell. Hos sykehjem B ble det gjennomført et lengre møte hvor også TotalCtrl ville vise frem systemet for deres representanter. Intervjuene inneholdt spørsmål innenfor følgende kategorier:

- Nåværende rutiner og hva som fungerer godt
- Opplevde utfordringer med dagens system
- Eksisterende tiltak for å redusere matsvinn
- Mønstre i type avfall og tidspunkt for kasting
- Holdninger til automatisering og nye teknologier
- Prioriteringer mellom hastighet og nøyaktighet

- Ønsker for en ideell løsning

Utvalget er begrenset til helsesektoren og representerer derfor ikke andre bransjer som hotell eller restauranter. Dette var et bevisst valg da TotalCtrl spesifikt ønsket innsikt fra sykehjem, som utgjør en sentral del av deres kundebase. Funnene fra undersøkelsene og deres innvirkning på systemdesignet er presentert i avsnitt [4.4.1](#).

Nåværende praksis

Begge sykehjemmene følger en etablert prosedyre for matsvinnregistrering. Når mat kastes, enten på avdelingskjøkken eller hovedkjøkken, må personalet manuelt veie avfallet, registrere vekten i TotalCtrl applikasjon, knytte registreringen til riktig måltid, og eventuelt legge til en beskrivelse av hva som ble kastet og årsaken. For produksjonssvinn på hovedkjøkkenet registreres i tillegg nøyaktig hvilket produkt som kastes.

Sykehjemmene opererer med et matbudsjett på omtrent 135 kroner per beboer per døgn, noe som understreker den økonomiske betydningen av effektiv ressursstyring. Begge institusjonene er pålagt av kommunen å gjennomføre systematisk matsvinnregistrering to til fire ganger årlig.

Intervjuene avdekket flere gjennomgående utfordringer med dagens manuelle registreringsprosess. Disse utfordringene kan du lese mer om under.

Tidspress og arbeidsbelastning

Periodene før hovedmåltidene, særlig rundt klokken 9 og 12, ble beskrevet som spesielt hektiske. I disse periodene har personalet begrenset kapasitet til oppgaver utover kjernevirksomheten. Veiing og registrering blir ofte nedprioritert eller glemt når andre oppgaver krever oppmerksomhet.

Praktiske hindringer

Informantene rapporterte at vektutstyr kan forsvinne dersom de blir feil plassert og er ikke tilgjengelig når det trengs. Den manuelle prosessen oppleves som tungvint og tidkrevende, noe som bidrar til at registreringer uteblir. På avdelingene er det stor variasjon i bemanning, med faste ansatte, deltidsansatte og vikarer, noe som gjør det vanskelig å opprettholde konsistente rutiner.

Årsaker til matsvinn

De vanligste årsakene til matsvinn som ble identifisert var varierende og endret appetitt hos beboerne, samt at ansatte åpner nye forpakninger fremfor å bruke opp allerede åpnete produkter. Typisk avfall innen frokost inkluderer brødskorper, kjøttpålegg, ost og syltetøy fra forpakninger som ikke blir brukt opp.

Behov og ønsker

Representantene uttrykte et tydelig ønske om økt automatisering for å redusere den manuelle arbeidsbyrden. Sentrale ønsker inkluderte automatisk veiing uten behov for manuell håndtering, en løsning der

vekten nullstilles automatisk når avfallsposen fjernes, samt overflyllingsvern som varsler når beholderen er full.

Når det gjelder informasjonsbehov, fremhevet representantene viktigheten av å vite både hva som kastes og hvor mye, ikke bare totalvekt. Denne informasjonen er verdifull for å justere innkjøp og redusere fremtidig svinn. Økonomisk oversikt over verdien av kastet mat ble også etterspurt.

Vurdering av taleteknologi

Et sentralt spørsmål i undersøkelsen var hvordan representantene stilte seg til talebasert interaksjon med avfallssystemet. Her forekom det betydelige forskjeller mellom de ulike miljøene.

På hovedkjøkkenet, som er adskilt fra pasientområdene, ble talefunksjonalitet vurdert som fullstendig gjennomførbart. Miljøet er mer kontrollert, og personalet kan snakke med systemet uten forstyrrelser.

På avdelingskjøkkenene var vurderingen mer nyansert. Disse kjøkkenene er ofte plassert i tilknytning til fellesområder der beboerne oppholder seg. Bakgrunnsstøy fra TV og samtaler, kombinert med ønsket om et rolig miljø, gjør taleteknologi mindre egnet. Informantene fra sykehjem B påpekte særlig at lydbaserte tilbakemeldinger, inkludert talesvar og lydsignaler, kan være problematiske på psykiatriske og skjermede avdelinger. Uventede lyder kan skape forvirring eller uro hos sårbare pasienter.

En mulig mellomløsning som ble foreslått var bruk av lavt volum eller begrenset talefunksjonalitet kun på egnede lokasjoner.

Implikasjoner for prototypeutvikling

Kundeundersøkelsene ga verdifull innsikt som direkte påvirker designvalgene for prototypen. Tre hovedprinsipper ble identifisert.

1. For det første bør løsningen ha modulær kompleksitet. Systemet bør kunne tilpasses ulike miljøer gjennom konfigurerbare funksjonsnivåer. Et grunnnivå kan tilby kun automatisk veiing, et mellomnivå kan legge til talefunksjonalitet, og et fullverdig nivå kan kombinere veiing med både tale- og kamerabasert gjenkjenning.
2. For det andre må løsningen ha kompakt formfaktor. Observasjoner fra kjøkkenfasilitetene viste begrenset tilgjengelig plass, noe som stiller krav til en fysisk kompakt løsning.
3. For det tredje er det behov for fleksibel identifiseringsmetode. Erkjennelsen av at taleteknologi ikke er universelt egnet styrker beslutningen om å utvikle støtte for flere gjenkjenningsmetoder, der kamerabasert identifisering kan fungere som et alternativ i støyfølsomme miljøer.

Undersøkelsene bekreftet også den grunnleggende verdien av prosjektet: *en automatisert løsning som reduserer manuelt arbeid vil møte et reelt behov i sektoren og kan bidra til bedre datakvalitet i matsvinnregistreringen.*

Kapittel 4

Planlegging og design

Med utgangspunkt i det teoretiske grunnlaget og metodiske rammeverket beskrevet i kapittel 2 og 3, presenterer dette kapittelet planleggings- og designprosessen som ligger til grunn for prototypen. Først redegjøres det for møtestrukturen med oppdragsgiver og veileder, etterfulgt av en gjennomgang av idémyldringsprosessen som ledet frem til de sentrale teknologi- og arkitekturvalgene. Deretter presenteres brukerscenarier og krav, før systemarkitekturen og den modulære løsningsmodellen beskrives i detalj. Kapittelet avsluttes med en gjennomgang av hardware- og software-design, samt begrunnelse for valg av kommunikasjonsprotokoll.

4.1 Møter med oppdragsgiver og veileder

I løpet av prosjektperioden har gruppen hatt jevnlige møter med både oppdragsgiver og veileder. Møtene med oppdragsgiver har i hovedsak vært ukentlige, mens møtene med veileder Ida fra Høgskolen i Østfold har vært annenhver uke.

I møtene med oppdragsgiver har målet vært å presentere fremdrift, stille kritiske spørsmål og vise til tanker om videre arbeid. Gruppen har vist frem ukentlig arbeid samt månedlige rapporter og gjennomganger etter deres ønske. Møtene har vært nyttige for motivasjonen i gruppen, ettersom det å regelmessig vise fremdrift bidrar til jevnere arbeidsinnsats over tid.

Siden oppdragsgiver i hovedsak prioriterer utvikling av prototype, har dette naturlig vært hovedtemaet i deres møter. I begynnelsen, da mye av arbeidet handlet om planlegging og rapportskriving, var veiledningsmøtene særlig viktige. Etter hvert som gruppen fikk satt en tydelig plan og begynte å se løsningen for seg, ble møtene med oppdragsgiver mer produktive. De hadde egne tanker og innspill som gruppen kunne diskutere, og begge parter kom frem til løsninger de var komfortable med å utvikle videre.

Gruppen startet med ukentlige møter med oppdragsgiver, noe som fungerte godt i starten. Mot slutten av perioden ble det imidlertid mindre nytt å diskutere fra uke til uke, og møtehyppigheten ble justert til annenhver uke. Dette ga mer innhold i hvert enkelt møte og opplevdes som mer effektivt.

Møtene med veileder har hatt et noe annerledes fokus. Her har rapport og det akademiske arbeidet stått sentralt. Disse møtene gir gruppen muligheten til å diskutere det faglige aspektet av oppgaven, få innspill på valg som er tatt underveis og avklare om arbeidet er på rett spor. Gruppen har presentert fremdrift i rapportskrivningen, diskutert innholdet av det som er produsert og fått tilbakemeldinger på ting som bør endres eller utdypes. Møtene har gitt god verdi gjennom faglige samtaler og idéutveksling, og har hjulpet gruppen med konkrete valg knyttet til både rapporten og prototypen. I tillegg har veileder gitt skriftlige kommentarer til rapporten, slik at feil og mangler har kunnet rettes underveis.

Generelt har møtene med begge parter vært viktige for gruppens motivasjon og bidratt til jevnere progresjon gjennom hele prosjektperioden.

4.2 Idémyldring

Med utgangspunkt i et design science-orientert forskningsdesign tok idémyldringen i dette prosjektet form ved bruk av en systematisk kartlegging av løsningsrommet, snarere enn fri brainstorming. Målet var å identifisere hvilke teknologier og arkitekturvalg som best kunne løse problemstillingen innenfor de gitte rammene.

Tidlig i prosjektet ble det gjennomført en felles gjennomgang av TotalCtrl sine krav og ønsker, kombinert med gruppens egen tekniske bakgrunn. Det ble raskt klart at løsningen måtte balansere flere til dels motstridende hensyn: automatisering uten å kreve for mye av brukeren, tilstrekkelig nøyaktighet med rimelig hardware, og integrasjon mot et eksisterende API uten å bygge unødvendig kompleks infrastruktur.

Gruppen identifiserte tidlig tre overordnede tekniske spørsmål som måtte besvares før design kunne begynne:

- Hvilken metode egner seg best for automatisk klassifisering av matavfall – tale, bilde eller en kombinasjon?
- Hvilken kommunikasjonsprotokoll skal benyttes for dataoverføring til TotalCtrl?
- Hvilken prosesseringsplattform er egnet for lokal kjøring av modeller og sensoravlesning?

For å besvare disse spørsmålene ble det gjennomført en strukturert litteraturgjennomgang av relevante teknologier, presentert i kapittel 2, samt innhentet innsikt fra brukerintervjuer som beskrevet i avsnitt 4.3. Intervjuene viste seg å være særlig avgjørende for retningsvalget: der gruppen opprinnelig så for seg én fast løsning, avdekket brukerdialogen at behovene varierer betydelig mellom ulike kjøkkenmiljøer. Dette la grunnlaget for den modulære arkitekturen med fire kompleksitetsnivåer, beskrevet i avsnitt 4.4.1.

Tidlig i prosessen ble flere tilnærminger vurdert og lagt til side. Arduino-baserte løsninger ble raskt utelukket da de mangler prosessorkraft til å kjøre lokale maskinlæringsmodeller. For lydopptak ble en dedikert ekstern mikrofon vurdert, men forkastet da den krevde ekstra konverteringshardware som ikke var tilgjengelig. Pi Camera Module ble også vurdert for bildeopptak, men et USB-webkamera med innebygget mikrofon løste begge behovene mer praktisk.

For talegjenkjenning ble skybaserte tjenester vurdert, men forkastet da systemet er designet for å fungere også ved nettverksbrudd. Ettersom at hendelser mellomlagres lokalt og synkroniseres når forbindelsen gjenopprettes, må klassifiseringen kunne skje lokalt for at denne funksjonaliteten skal ha reell verdi. I tillegg kom personvern hensyn og løpende kostnader per forespørsel inn som forsterkende argumenter. Innen bildegjenkjenning, ble tyngre modeller som ViT og CLIP fort uaktuelt da ressurskravene er for høye for en Raspberry Pi 4. Valget sto til slutt mellom YOLO og MobileNetV2, der YOLO ble foretrukket på grunn av bedre balanse mellom nøyaktighet og hastighet på ressursbegrensede enheter.

Valget om å benytte REST over HTTPS fremfor MQTT, Raspberry Pi 4 som edge-enhet og Python som programmeringsspråk ble alle tatt på et tidlig stadie, begrunnet i henholdsvis integrasjonskrav, tilgjengelighet og gruppens kompetanse. Disse valgene er nærmere redegjort for i kapittel 4.5 og 4.7.

4.3 Brukerscenarier og krav

Dette kapitlet beskriver systemets brukstilfeller fra et brukerperspektiv. Brukstilfellene definerer hvordan ulike aktører interagerer med systemet for å oppnå sine mål, og danner grunnlaget for de tekniske kravene til løsningen.

Ettersom løsningen er rettet mot ulike virksomhetstyper med forskjellige behov og rammebetingelser, gir tabell 4.1 en oversikt over hvilken funksjonalitet som anses som relevant per virksomhetstype. Dette danner grunnlaget for brukstilfellene beskrevet videre i kapitlet.

Tabell 4.1: Ønsket funksjonalitet per virksomhetstype

Virksomhet	Vektmåling	Talegjenkjenning	Bildegjenkjenning	API-integrasjon
Sykehjem (avdeling)	Ja	Begrenset	–	Ja
Sykehjem (hovedkjøkken)	Ja	Ja	Valgfritt	Ja
Hotell/restaurant	Ja	Ja	Ja	Ja
Kantine	Ja	Valgfritt	–	Ja

Aktører og interessenter

Tabell 4.2 identifiserer de primære aktørene som interagerer med systemet.

Tabell 4.2: Oversikt over systemets aktører

Aktør	Beskrivelse
Kjøkkenpersonale / Kokk	Primær bruker som kaster matavfall og logger dette via talekommando. Har ofte travle hender og begrenset tid.
Kjøkkensjef / Leder	Gjennomgår matsvinndata i TotalCtrl for å identifisere mønstre og forbedringsmuligheter.
Systemadministrator	Ansvarlig for oppsett, kalibrering og vedlikehold av enheten.
TotalCtrl-plattformen	Eksternt system som mottar, lagrer og visualiserer matsvinndata.

Oversikt over brukstilfeller

Tabell 4.3 gir en samlet oversikt over alle identifiserte brukstilfeller, med prioritering basert på prosjektets omfang som MVP (Minimum Viable Product). Prioriteringen følger MoSCoW-metodikken, der *Må ha* angir absolutte krav for en fungerende prototype, *Bør ha* angir viktige men ikke kritiske krav, og *Kjekt å ha* angir ønskelig funksjonalitet utenfor kjerneomfanget. Brukstilfeller merket *Fremtidig* er identifisert som relevante, men ligger utenfor prosjektets tidsramme og er inkludert for å illustrere systemets utviklingspotensial.

Tabell 4.3: Sammendrag av brukstilfeller

ID	Brukstilfelle	Aktør	Mål	Prioritet
UC-01	Logg matsvinn med tale	Kjøkkenpersonale	Raskt logge matsvinn uten å bruke hendene	Må ha
UC-02	Motta bekreftelse	Kjøkkenpersonale	Vite at matsvinnet ble registrert korrekt	Må ha
UC-03	Håndter uklar tale	Kjøkkenpersonale	Prøve på nytt ved misforståelse	Må ha
UC-04	Se matsvinndata	Kjøkkensjef	Se loggført matsvinn i TotalCtrl	Må ha
UC-05	Kalibrer vekt	Systemadministrator	Sikre nøyaktige vektmålinger	Må ha
UC-06	Håndter nettverksfeil	System	Lagre data lokalt til tilkobling gjenopprettes	Bør ha
UC-07	Manuell overstyring	Kjøkkenpersonale	Korrigere feilkategorisert avfall	Kjekt å ha
UC-08	Logg med kamera	Kjøkkenpersonale	Automatisk identifikasjon uten tale	Bør ha
UC-09	Hybrid klassifisering	Kjøkkenpersonale	Kombinerer bilde- og tale-gjenkjenning for høyere pålitelighet	Fremtidig

Detaljerte brukstilfeller

Denne seksjonen beskriver seks ulike brukstilfeller i detalj, inkludert forutsetninger, hovedflyt og alternative flyter, presentert i tabell 4.4 til og med tabell 4.9.

UC-01: Logg matsvinn med tale

Tabell 4.4: UC-01: Logg matsvinn med tale

Felt	Beskrivelse
ID	UC-01
Navn	Logg matsvinn med tale
Aktør	Kjøkkenpersonale
Forutsetninger	• Systemet er påslått og tilkoblet nettverk • Vekten er kalibrert • Mikrofon og høyttaler fungerer
Trigger	Bruker plasserer matavfall på vekten
Hovedflyt	1. Bruker plasserer matavfall på vekten 2. Systemet registrerer vektendring 3. Systemet spør: «<i>Hva kaster du?</i>» 4. Bruker sier: «<i>Poteter</i>» 5. Systemet gjenkjenner talen og mapper til kategori 6. Systemet bekrefter: «<i>Poteter, 340 gram registrert</i>» 7. Systemet sender data til TotalCtrl API 8. Bruker fjerner avfallet og kaster det i søppel
Postbetingelser	Matsvinn er registrert i TotalCtrl med kategori, vekt og tidsstempel
Alternative flyter	Se UC-03 for håndtering av talegjenkjenningsfeil

UC-02: Motta bekreftelse

Tabell 4.5: UC-02: Motta bekreftelse

Felt	Beskrivelse
ID	UC-02
Navn	Motta bekreftelse
Aktør	Kjøkkenpersonale
Forutsetninger	Bruker har sagt en matsvinnskategori
Trigger	Systemet har prosessert talekommando vellykket
Hovedflyt	1. Systemet konverterer tale til tekst 2. Systemet matcher tekst mot matkategori 3. Systemet leser gjeldende vekt 4. Systemet gir talebekreftelse: «<i>[Kategori], [vekt] gram registrert</i>» 5. Valgfritt: LED-indikator blinker grønt
Postbetingelser	Bruker har fått hørbar og/eller visuell bekreftelse
Merknader	Bekreftelse bør skje innen 2 sekunder for å ikke forsinke arbeidsflyten

UC-03: Håndter uklar tale

Tabell 4.6: UC-03: Håndter uklar tale

Felt	Beskrivelse
ID	UC-03
Navn	Håndter uklar tale
Aktør	Kjøkkenpersonale
Forutsetninger	Bruker har snakket, men systemet forstod ikke
Trigger	Talegjenkjenning returnerer lav konfidens eller ingen treff
Hovedflyt	1. Systemet klarer ikke å gjenkjenne talen 2. Systemet sier: «<i>Beklager, jeg forstod ikke. Vennligst gjenta.</i>» 3. Bruker snakker på nytt 4. Systemet gjenkjenner og fortsetter til bekreftelse
Alternativ flyt A	Etter 3 mislykkede forsøk sier systemet: « <i>Vennligst bruk manuell inndata</i> » (knapp eller annen reserve)
Alternativ flyt B	Bruker kan si « <i>avbryt</i> » for å avbryte registreringen
Postbetingelser	Enten er matsvinn registrert vellykket, eller bruker har avbrutt

UC-04: Se matsvinndata

Tabell 4.7: UC-04: Se matsvinndata

Felt	Beskrivelse
ID	UC-04
Navn	Se matsvinndata
Aktør	Kjøkkensjef / Leder
Forutsetninger	Matsvinn har blitt logget via IoT-enheten
Trigger	Leder åpner TotalCtrl-dashbordet
Hovedflyt	1. Leder logger inn i TotalCtrl 2. Navigerer til matsvinnoversikt 3. Ser oppføringer med kategori, vekt, tidsstempel og kildeenhet 4. Kan filtrere på dato, kategori eller enhet
Postbetingelser	Leder har innsikt i matsvinnmønstre
Merknader	Dette brukstilfellet skjer i TotalCtrl, ikke på IoT-enheten, men viser ende-til-ende-verdien av løsningen

UC-05: Kalibrer vekt

Tabell 4.8: UC-05: Kalibrer vekt

Felt	Beskrivelse
ID	UC-05
Navn	Kalibrer vekt
Aktør	Systemadministrator
Forutsetninger	Enheten er satt opp, administrator har en kjent referansevekt
Trigger	Førstegangsoppsett eller periodisk rekalibrering
Hovedflyt	1. Administrator går inn i kalibreringsmodus 2. Systemet sier: «Fjern alle gjenstander fra vekten» 3. Systemet registrerer nullpunkt (tara) 4. Systemet sier: «Plasser 1 kg referansevekt» 5. Administrator plasserer kjent vekt 6. Systemet beregner og lagrer kalibreringsfaktor 7. Systemet bekrefter: «Kalibrering fullført»
Postbetingelser	Vektmålinger er nøyaktige
Merknader	Bør kun gjøres én gang ved oppsett, med mindre vekten flyttes

UC-06: Håndter nettverksfeil

Tabell 4.9: UC-06: Håndter nettverksfeil

Felt	Beskrivelse
ID	UC-06
Navn	Håndter nettverksfeil
Aktør	System (automatisk)
Forutsetninger	Enheten har logget matsvinn, men kan ikke nå TotalCtrl API
Trigger	API-kall feiler på grunn av nettverksfeil
Hovedflyt	1. Systemet forsøker å sende data til TotalCtrl 2. Tilkobling feiler 3. Systemet lagrer oppføring lokalt med tidsstempel 4. Systemet varsler bruker: «Lagret offline, synkroniseres senere» 5. Systemet prøver på nytt periodisk 6. Når tilkobling er gjenopprettet, sendes køede oppføringer 7. Bekrefter at synkronisering er fullført
Postbetingelser	Ingen data går tapt til tross for nettverksproblemer
Merknader	Viktig for pålitelighet i ekte kjøkkenmiljøer

Brukerscenarioer

Denne seksjonen presenterer konkrete scenarioer som illustrerer hvordan systemet brukes i praksis. Scenarioene gir et narrativt bilde av brukeropplevelsen.

Scenario 1: Travelt under lunsjservering

Scenario 1

Emma er kokk på en hotellrestaurant. Under lunsjserveringen tilbereder hun flere retter samtidig. Hun skjærer fettete av noe biff og må logge det raskt. Hun plasserer avskjæret på vekten, sier «*biffavskjær*» mens hun allerede snur seg tilbake til skjærebrettet, hører «*Biffavskjær, 180 gram registrert*» fra høyttaleren, og fortsetter arbeidet.

Total interaksjonstid: ca. 4 sekunder.

Scenario 2: Opprydding ved stengetid

Scenario 2

Ved stengetid rydder kjøkkenassistent Lars bort dagens avfall. Han har flere ting å logge. Han plasserer ris på vekten, sier «*ris*», får bekreftelse. Plasserer grønnsaksskreller, sier «*grønnsaker*», får bekreftelse. Plasserer litt brød, sier «*brød*», får bekreftelse.

Systemet håndterer raske, sekvensielle oppføringer uten forvirring.

Scenario 3: Støyende kjøkken

Scenario 3

I rushtiden er kjøkkenet støyende med vifter, klirrende kjeler og samtaler. Kokk Marius plasserer pastaavfall på vekten og snakker. Systemet fanger det ikke opp tydelig og sier «*Beklager, jeg forstod ikke. Vennligst gjenta.*»

Marius snakker litt høyere og nærmere mikrofonen: «*Pasta*». Systemet bekrefter: «*Pasta, 420 gram registrert*».

Scenario 4: Nettverksbrudd

Scenario 4

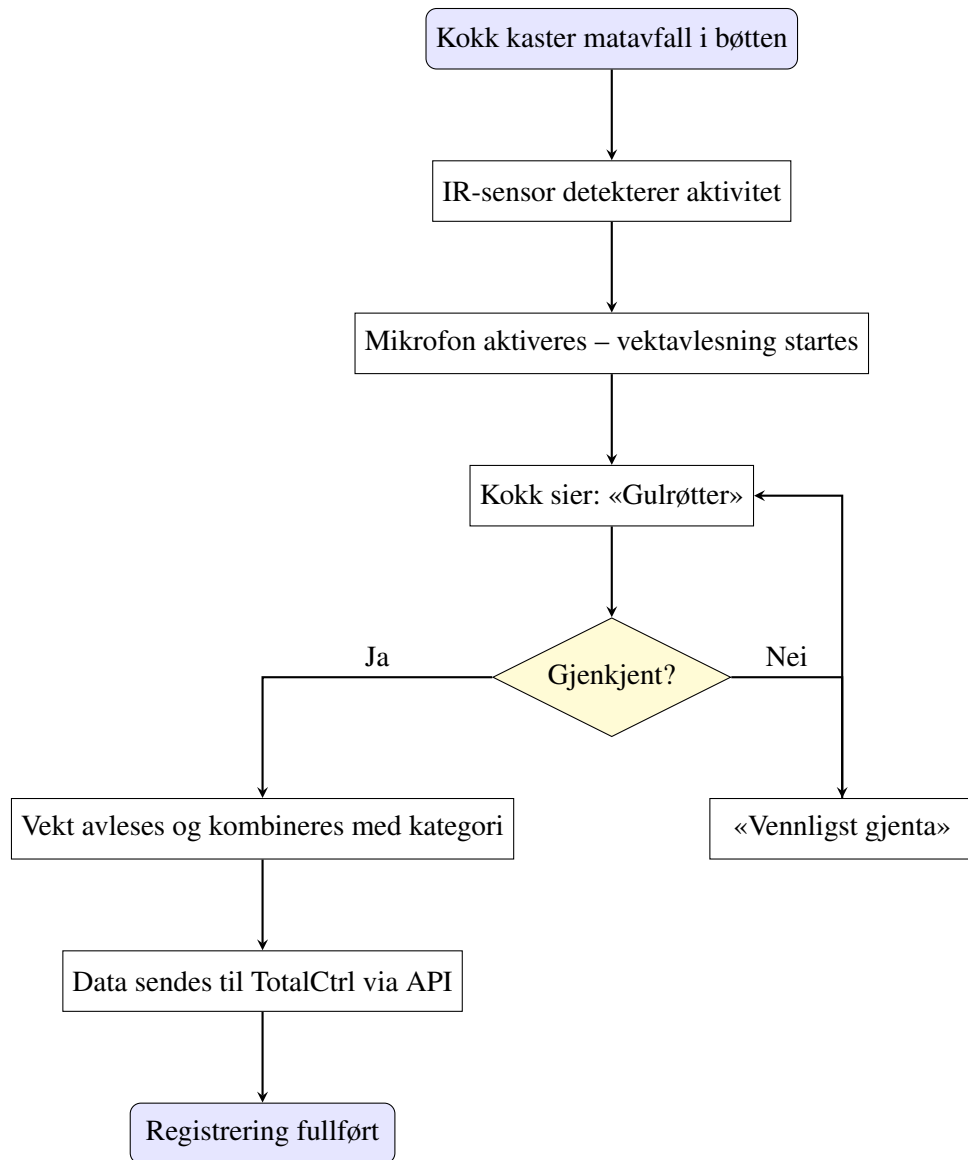
Restaurantens internett går ned under middagsserveringen. Kokk Maria logger fiskeavfall som vanlig. Systemet bekrefter «*Fisk, 310 gram registrert — lagret offline*». Når hun logger en annen vare senere, samme melding.

Neste morgen, når IT har fikset internettet, synkroniserer systemet automatisk alle køede oppføringer til TotalCtrl. Kjøkkensjefen ser komplette data med korrekte tidsstempler.

Brukerreise

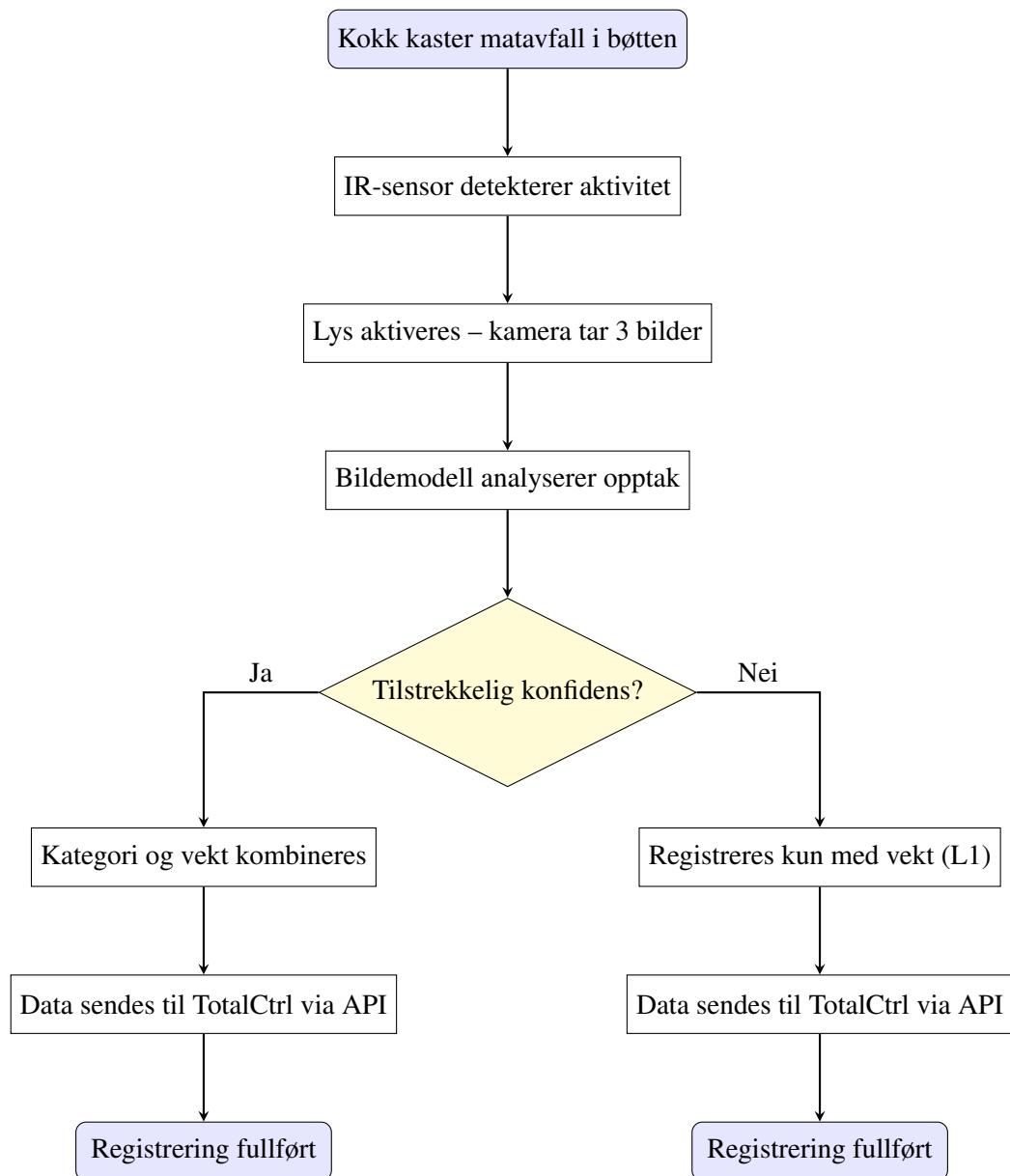
Figurene 4.1 og 4.2 viser brukerreisen ved logging av matsvinn for henholdsvis tale- og bildegjenkjenning. Felles for begge er at selve registreringen skjer automatisk – brukeren trenger ikke å gjøre annet enn å kaste avfallet.

Ved talebasert registrering (L2) aktiveres mikrofonen automatisk når IR-sensoren detekterer at noe er på vei ned i beholderen. Brukeren har et kort tidsvindu til å si hva som kastes, før systemet gjenkjenner kommandoen, leser av vekten og sender data til TotalCtrl.



Figur 4.1: Brukerreise ved talebasert matsvinnregistrering (L2)

Ved bildebasert registrering (L3) skjer hele klassifiseringen automatisk. Når IR-sensoren trigges, aktiveres lyset og kameraet tar tre bilder i rask rekkefølge. Bildemodellen analyserer opptakene og identifiserer avfallstypen. Dersom modellen ikke er sikker nok, registreres hendelsen kun med vekt – tilsvarende L1 – for å unngå feilklassifisering.



Figur 4.2: Brukerreise ved bildebasert matsvinnregistrering (L3)

Kravsporing

Tabell 4.10 viser hvordan brukstilfellene kobles til tekniske krav for systemet. Dette sikrer sporbarhet mellom brukerens behov og implementasjonen.

Tabell 4.10: Sporingsmatrise: Krav til brukstilfeller

Krav-ID	Beskrivelse	Relaterte UC
REQ-01	Systemet skal måle vekt med nøyaktighet på ± 5 gram	UC-01, UC-05
REQ-02	Systemet skal gjenkjenne norsk tale	UC-01, UC-03
REQ-03	Systemet skal gi talebekreftelse innen 2 sekunder	UC-02
REQ-04	Systemet skal forsøke talegjenkjenning opptil 3 ganger	UC-03
REQ-05	Systemet skal overføre data via REST API	UC-01, UC-04
REQ-06	Systemet skal lagre data lokalt ved nettverksfeil	UC-06
REQ-07	Systemet skal støtte kalibreringsprosedyre	UC-05
REQ-08	Systemet skal kartlegge talte ord til TotalCtrl-kategorier	UC-01
REQ-09	Systemet skal identifisere avfallstype via bildegjenkjenning	UC-08

4.4 Systemarkitektur

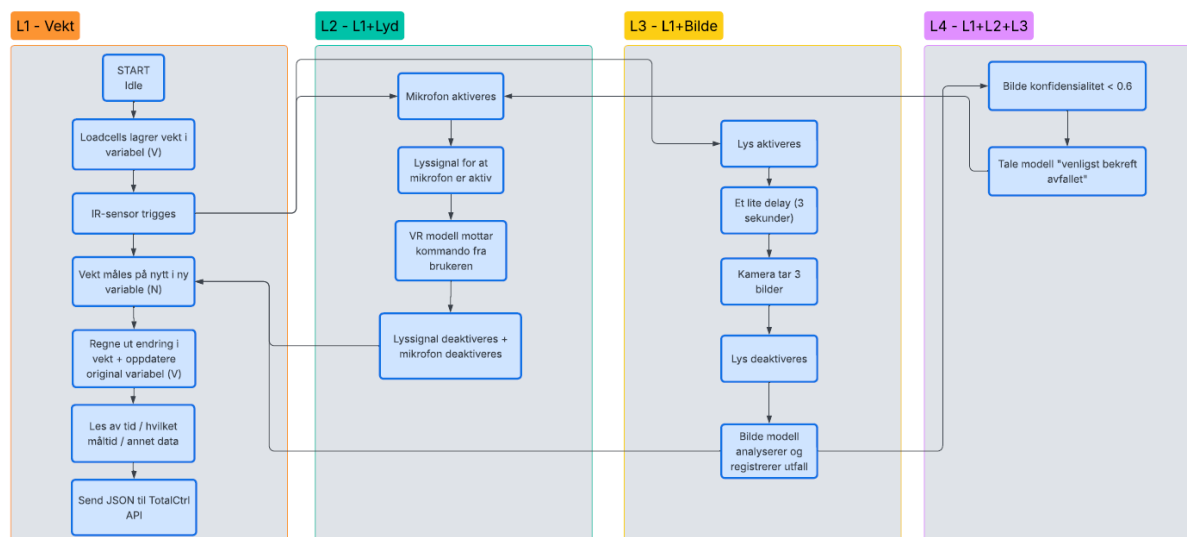
Systemet er bygget rundt en Raspberry Pi 4 som sentral edge-enhet, og består av tre overordnede lag: sensorlaget, prosesseringslaget og integrasjonslaget. Sensorlaget omfatter vektceller, IR-sensor, kamera og mikrofon som samler inn data fra omgivelsene. Prosesseringslaget håndterer all lokal databehandling på Raspberry Pi, inkludert vektmåling, talegjenkjenning og bildegjenkjenning. Integrasjonslaget står for kommunikasjonen mot TotalCtrl sitt API via REST over HTTPS. Hver registrert hendelse sendes som en strukturert JSON pakke til et stagingmiljø, med lokal mellomlagring som sikkerhetsnett ved nettverksbrudd. Dette er nærmere beskrevet i henholdsvis avsnitt 4.5, 4.6 og 4.7.

4.4.1 Modulær løsningsarkitektur

Med utgangspunkt i innsikten fra brukerintervjuene har gruppen valgt en modulær tilnærming til løsningen. Intervjuene avdekket at behovene varierer mellom ulike kjøkken og brukssituasjoner, hvor noen ønsker en enkel løsning som bare registrerer vekt, mens andre har behov for mer avansert kategorisering. En modulær arkitektur gjør det mulig å tilpasse løsningen til ulike krav uten å måtte redesigne hele systemet.

Denne tilnærmingen har også praktiske fordeler for utvikling og vedlikehold. Nye funksjoner kan legges til uten å påvirke eksisterende funksjonalitet, og eventuelle feil kan isoleres til enkeltmoduler. For TotalCtrl betyr dette at løsningen også kan rulles ut gradvis og skaleres eller tilby riktig modul etter kundens behov og prisklasse.

Gruppen har definert fire kompleksitetsnivåer som bygger på hverandre. Figur 4.3 viser dataflyten og sammenhengen mellom nivåene:



Figur 4.3: Flytskjema som viser de fire kompleksitetsnivåene og dataflyten mellom komponentene.

Den modulære arkitekturen sikrer at hver komponent kan utvikles, testes og forbedres uavhengig av de andre, samtidig som løsningen forblir skalerbar for fremtidige behov.

L1 – Vekt

Grunnivået håndterer automatisk vektmåling. Vektceller registrerer startvekt, og når IR-sensoren detekterer aktivitet, måles vekten på nytt. Differansen beregnes, tidspunkt registreres, og dataen sendes til TotalCtrl sitt API. Dette nivået krever ingen input fra brukeren og fungerer helautomatisk.

L2 – Vekt og tale

Dette nivået utvider L1 med talebasert kategorisering. Etter at vektendring er registrert, aktiveres mikrofonen med et lyssignal. Brukeren sier hva som kastes, en talemodell tolker kommandoen, og kategorien legges til før data sendes. Mikrofon og lyssignal deaktiveres når registreringen er fullført.

L3 – Vekt og bilde

Dette nivået erstatter tale med bildegjenkjenning. Etter vektmåling tar kameraet flere bilder over en kort tidsperiode. En bildemodell analyserer opptakene og forsøker å identifisere avfallstypen automatisk. Dette nivået krever ingen aktiv handling fra brukeren.

L4 – Hybrid

Det mest avanserte nivået kombinerer både bilde og tale. Systemet forsøker først å kategorisere via bildegjenkjenning. Dersom modellen returnerer en konfidens under en definert terskel, bes brukeren om å bekrefte eller korrigere via tale. Dette gir høyere pålitelighet uten å alltid kreve brukerinput.

L4 er ikke implementert i den nåværende prototypen, men arkitekturen er utformet slik at dette kan realiseres uten større endringer i kjernesystemet. Videre utvikling av L4 er beskrevet i avsnitt 8.1.

4.5 Hardware-design

Dette delkapittelet beskriver sentrale designvalg for maskinvaren som er tatt under utviklingen av prototypen, samt begrunnelser for valgene

Raspberry Pi 4

Prototypen er bygget rundt en Raspberry Pi 4 som hovedprosesseringsenhet. Dette ble valgt på bakgrunn av flere faktorer. Først og fremst er Raspberry Pi 4 bedre egnet enn Arduino baserte løsninger for dette formålet, hovedsakelig fordi Arduino plattformer har begrenset prosessorkraft og mangler støtte for å kjøre komplekse maskinlæringsmodeller lokalt. Raspberry Pi 4 kjører et Linux-basert operativsystem og har tilstrekkelig RAM (minne) til å håndtere både bilde og talegjenkjenningsmodeller, samt WiFi kommunikasjon mot API-et.

Ideelt sett ville gruppen foretrukket å benytte en Raspberry Pi 5, som tilbyr høyere prosessorhastighet og bedre ytelse for kjøring av AI-modeller. Raspberry Pi 4 var imidlertid lett tilgjengelig, tilfredsstillende kravene til prototypen og er en rimeligere løsning. Dette gjorde den til et naturlig valg for dette prosjektet.

Raspberry Pi 4 tilbyr også et bredt utvalg av tilkoblingsmuligheter som har vært sentrale i hardware-designet. De fire USB-portene muliggjør direkte tilkobling av eksterne enheter som kamera og mikrofon uten behov for ekstra grensesnitt eller adaptere. HDMI-utgangen gir mulighet for tilkobling av ekstern skjerm, noe som forenkler feilsøking og utvikling direkte på enheten. Ethernet-porten sikrer stabil nettverksforbindelse under testing og integrasjon mot TotalCtrl sitt API, uavhengig av trådløs tilkobling. Disse egenskapene gjør Raspberry Pi 4 til en fleksibel plattform for utvikling og testing av IoT-prototyper.

Vektceller og HX711

For vektmåling benyttes fire *50 kg half bridge vektceller med 1000 ohm måler*. Disse vektcellene var tilgjengelige hos HiØ, og gruppen valgte derfor å bruke disse fremfor å lete etter alternative sensorer. Vektcellene er koblet til Raspberry Pi via en HX711 analog til digital omformer, som forsterker og digitaliserer signalet fra vektcellene til et format Raspberry Pi kan lese via GPIO grensesnittet.

HX711 er spesielt utviklet for vektapplikasjoner og tilbyr 24-bits oppløsning, som gir tilstrekkelig presisjon for å måle avfallets vekt i gram. Kommunikasjonen mellom HX711 og Raspberry Pi skjer gjennom et to leder grensesnitt (klokke og data), som gjør integrasjonen enkel og pålitelig.[5]

IR-sensor som trigger

Som triggersensor benyttes en *IR-08H V02.45.DC infrarød nærhetssensor*. Sensoren detekterer når en bruker beveger hånden sin over søppelbøtten, og initierer dermed vektmåling og eventuelt bilde eller lydopptak.

Gruppen har også vurdert å bruke en ultralyd basert avstandssensor som alternativ, men dette ble lagt til siden da IR-sensoren viste seg å fungere tilfredsstillende under oppsett og testing. IR-sensoren har den fordelen at den er enkel å integrere via GPIO, returnerer digitalt signal, og er robust mot vanlig omgivelseslys takket være modulert IR deteksjon.

Valget av IR-sensor fremfor kontinuerlig polling av vekten reduserer også prosessor belastningen og energi forbruket, ettersom systemet kun aktiveres ved faktiske hendelser.

Kamera og mikrofon

For bilde og lydopptak benyttes et Logitech HD Pro webkamera med innebygget mikrofon. Dette kameraet ble valgt etter en anbefaling fra en ansatt hos HiØ, som påpekte at USB kameraet generelt gir bedre kvalitet på bildene enn ved bruk av Raspberry Pi camera module v2 (PiCamera).

Gruppen vurderte opprinnelig å bruke Pi Camera kombinert med en separat mikrofon spesifikt en Adafruit MAX9814, men denne mikrofonen krever en analog til digital konverter som ikke var tilgjengelig for Raspberry Pi 4. Da det viste seg at Logitech sitt kamera hadde en innebygget mikrofon av akseptabel kvalitet, ble dette valgt som en enhet å bygge prototypen videre med.

USB tilkoblingen gjør det også enklere å bytte ut kameraet ved behov, uten å måtte endre på hardware oppsettet eller noe i programvaren.

4.6 Software-design

Dette delkapittelet beskriver de sentrale programvarevalgene og beslutningene rundt arkitekturen som ligger som et grunnlag for prototypens software løsning.

Alternative tilnærminger

Flere alternative tilnærminger ble vurdert før den endelige arkitekturen ble valgt. Skybasert prosessering av tale og bilde ville gitt høyere nøyaktighet, men ville gjort systemet avhengig av konstant nettverksforbindelse og introdusert løpende kostnader per forespørsel, samt personvernmessige utfordringer knyttet til å sende lyd- og billedata ut av systemet. MQTT ble vurdert som kommunikasjonsprotokoll, men krevde en dedikert mellomtjener og passet dårlig med TotalCtrl sitt eksisterende API, som beskrevet i avsnitt 4.7. Arduino-baserte løsninger ble raskt utelukket da de mangler prosessorkraft til å kjøre lokale maskinlæringsmodeller. Valget om lokal prosessering på Raspberry Pi ble tatt for å sikre robusthet ved nettverksbrudd og for å redusere latenstid for brukeren.

Lokal prosessering

Et av de viktigste valgene i prosjektet er at all databehandling, vektmåling, bilde og talegjenkjenning, utføres lokalt på Raspberry Pi, fremfor å sende rå sensor data til skybaserte løsninger for prosessering.

Dette valget ble tatt på grunnlag av flere årsaker. For det første gjør lokal prosessering systemet robust mot nettverksbrudd. Selv om produktet skulle miste internett forbindelse, så kan prototypen fortsatt registrere hendelser basert på vekt, og i L2 og L3 løsningene også klassifisere avfallet lokalt på Raspberry Pi. Data lagres midlertidig lokalt og skal synkronisere med API-et når forbindelsen gjenopprettes, dette sikrer at ingen hendelser går tapt.

For det andre reduserer lokal prosessering latenstid og avhengighet av ekstern infrastruktur. Brukeren får umiddelbar respons fra systemet uten å måtte vente på at en sky basert løsning skal prosessere alle verdiene før det blir loggført.

Alternativet ville vært å benytte skybaserte tjenester for tale og bildegjenkjenning, sånn som Google Cloud Speech-to-Text eller lignende API-er, som beskrevet i kapittel 2.3.2 . Dette ville gitt høyere nøyaktighet, men også introdusert avhengighet av konstant nettverksforbindelse og potensielt høyere kostnader per forespørsel, samt personvernmessige utfordringer knyttet til å sende lyd- og bilde-data ut av systemet.

Gruppen ser at lokal prosessering på en Raspberry Pi 4 kan medføre mindre nøyaktighet enn skybaserte alternativer, spesielt for bilde og talegjenkjenning. Dette er en akseptabel avveining i denne fasen av prosjektet. Ved oppgradering slik som bruk av dedikerte akseleratorer, vil ytelsen kunne forbedres betydelig uten å endre den grunnleggende arkitekturen.

Modulær kodestruktur

Programvaren er strukturert som en modulær arkitektur der ulike funksjonsnivåer (L1–L4) kan velges uavhengig av hverandre, som beskrevet i avsnitt 4.4.1. Dette designet gjenspeiles også i kodestrukturen, der hver funksjonalitet er implementert som separate Python-moduler.

L1-løsningen fungerer som kjernen i systemet og håndterer kommunikasjon med HX711, vektmåling og IR-sensor-triggering. L2-løsningen er bygget som en utvidelse av L1, der et eget Python-script kalles når talegjenkjenning er nødvendig. L3-løsningen følger samme prinsipp som L2, men bildegjenkjenning kjøres i et eget virtuelt miljø og aktiveres via et subprocess-kall.

Denne modulære tilnærmingen gir flere fordeler. Hver modul kan utvikles, testes og vedlikeholdes uavhengig av de andre. Systemet kan også enkelt tilpasses eller oppgraderes til ulike brukerbehov ved å velge det nivået som passer best. I tillegg gjør det kodebasen mer oversiktlig og lettere å feilsøke.

Ulempen er at subprocess-kommunikasjon og separate virtuelle miljøer medfører en viss ytelseskostnad, men dette er en akseptabel avveining for den fleksibiliteten det gir.

Hendelsesbasert dataoverføring

Systemet benytter hendelsesbasert dataoverføring fremfor kontinuerlig strømming av sensordata. Data sendes kun til TotalCtrl sitt API når en faktisk hendelse har inntruffet, det vil si når avfall blir kastet.

Hendelsesbasert overføring reduserer nettverksbelastning og unngår unødvendige dataoverføringer. Triggeren for en hendelse er IR-sensoren, som beskrevet i avsnitt 2.1. Når sensoren detekterer aktivitet, starter systemet målingene avhengig av hvilket funksjonsnivå som er i bruk. For L1 måles kun vekt og tid. For L2 og L3 legges tale- og bildegjenkjenning til i sekvensen for å avgjøre hva som ble kastet, før data sendes.

Ekstern integrasjon og API-design

Kommunikasjonen mellom prototypen og TotalCtrl sin plattform skjer via et dedikert REST-endepunkt satt opp i deres stagingmiljø. Endepunktet mottar strukturerte JSON-meldinger over HTTPS og autentiseres med en API-nøkkel sendt i forespørselens header (X-API-Key).

Datastrukturen som sendes ble utformet av gruppen og presentert for TotalCtrl som et designforslag tidlig i prosjektet. Forslaget tok utgangspunkt i hvilke felt som ville gi mest analytisk verdi for deres plattform, og ble deretter bekreftet og tilpasset av TotalCtrl. Den endelige strukturen er organisert i fire objekter:

- **measurement** – inneholder kjernedata for hendelsen: tidsstempel i ISO 8601-format med tidssone, vekt i gram, hovedkategori og eventuell underkategori for avfallet.
- **classification** – beskriver hvordan klassifiseringen ble gjort. Feltet method angir hvilken metode som ble brukt og aksepterer verdiene image, voice eller weight. I tillegg inkluderes konfidensscore og eventuelle alternative klassifiseringer fra modellen.
- **context** – angir hvilket måltid hendelsen tilhører (breakfast, lunch, dinner eller other), samt årsak til matsvinnet.
- **device** – identifiserer enheten med en unik enhets-ID, fysisk plassering og programvareversjon.

Feltene er prioritert i tre nivåer. Tidsstempel, vekt, avfallskategori og enhets-ID er obligatoriske for at en registrering skal ha analytisk verdi. Konfidensverdi, klassifiseringsmetode og måltidsperiode er anbefalt, da disse gir TotalCtrl mulighet til å filtrere usikre målinger og sammenligne like måltider på tvers av installasjoner. Underkategori, plassering og firmware-versjon er valgfrie felt som legges til dersom systemet har tilgang til denne informasjonen.

Integrasjonen er verifisert ved å sende en testmåling til stagingmiljøet, som returnerte HTTP 201 Created med meldingen «*Waste measurement recorded successfully*». Dette bekrefter at dataformatet er korrekt og at kommunikasjonen fungerer som forventet. Skjermbildet av API-svaret er vist i figur [4.4](#).

```

[Running] python -u "c:\Users\Ole Sander\Desktop\TotalCtrl prosjektmappe\test_totalCtrl_api.py"
Sender testmåling til TotalCtrl staging API...
Timestamp: 2026-03-03T10:59:04+01:00
Payload:
{
  "measurement": {
    "timestamp": "2026-03-03T10:59:04+01:00",
    "weight_grams": 250,
    "waste_category": "vegetables",
    "sub_category": "salad"
  },
  "classification": {
    "method": "weight",
    "confidence": 1.0,
    "alternatives": []
  },
  "context": {
    "meal_period": "lunch",
    "waste_reason": "plate_waste"
  },
  "device": {
    "device_id": "proto-001",
    "location": "test-bench",
    "firmware_version": "0.1.0"
  }
}

Status code: 201
Svar fra server:
{
  "StatusCode": 201,
  "Status": true,
  "Message": "Waste measurement recorded successfully.",
  "id": 2
}

SUKSESS! API-et aksepterer dataene.
[Done] exited with code=0 in 0.907 seconds

```

Figur 4.4: Svar fra TotalCtrl sitt staging-API etter vellykket testmåling.

Visning av data i et brukergrensesnitt er ikke fullført som del av denne prototypen. Gjennom prosjektperioden har TotalCtrl arbeidet med å etablere en kobling mot deres eksisterende plattform for å vise registrerte hendelser direkte i deres grensesnitt, men dette lot seg ikke ferdigstille innen innleveringsfristen. Dette arbeidet blir gjort av TotalCtrl med implementering av vår data inn i deres plattform. Det er planlagt et oppfølgingsmøte med TotalCtrl etter prosjektleveransen for å ferdigstille denne integrasjonen før prototypen skal presenteres.

Dataflyt

Rekkefølgen på operasjonene er designet for å minimere ventetid og sikre konsistent data på tvers av alle funksjonsnivåer.

For L1 er flyten enkel. IR-sensoren detekterer aktivitet, vekt måles og data sendes til API-et.

For L2 kjøres vektmåling og lydopptak parallelt for å spare tid. Når IR-sensoren trigges, startes begge operasjonene samtidig. Etter at lydopptaket er ferdig, kjøres talegjenkjenning, og resultatet kombineres med vektdata før det sendes til API-et.

For L3 må bilder tas før vektmålingen, fordi brukeren holder avfallet foran kameraet før det kastes. Etter bildeopptak kjøres bildegjenkjenning, deretter måles vekt, og den kombinerte dataen sendes til API-et.

Denne rekkefølgen er valgt for å balansere brukeropplevelse, systemytelse og datakvalitet på best mulig måte.

For L4 kjøres bildegjenkjenning først, på samme måte som i L3. Dersom modellen returnerer en konfidens under en definert terskel, aktiveres mikrofonen og brukeren blir bedt om å bekrefte eller korrigere kategorien via tale. Vekt måles parallelt med bildeanalysen, og den kombinerte dataen sendes til API-et når begge operasjonene er fullført.

Lokal mellomlagring av data

For å sikre at ingen hendelser går tapt ved nettverksbrudd, er systemet designet med lokal mellomlagring som en integrert del av dataflyten. Når en hendelse registreres lagres dataen først lokalt på Raspberry Pi, før systemet forsøker å overføre den til TotalCtrl sitt API. Dette betyr at mellomlagringen ikke er et unntak som kun aktiveres ved feil, men en fast del av hvordan systemet håndterer alle hendelser.

Dersom API-kallet lykkes, fjernes den lokalt lagrede hendelsen etter fem hendelser. Dersom kallet feiler, beholdes hendelsen i en lokal kø og systemet forsøker å sende den på nytt når forbindelsen er tilgjengelig igjen. Hendelser i køen sendes i den rekkefølgen de ble registrert, og med de opprinnelige tidsstemplene fra registreringstidspunktet. Dette sikrer at dataene fremstår korrekt i TotalCtrl sin plattform selv om overføringen er forsinket.

Valget om å lagre lokalt først, fremfor å forsøke API-kallet direkte, er tatt for å gjøre systemet robust i et kjøkkenmiljø der nettverksforbindelsen kan være ustabil. En IoT-enhet uten mellomlagring vil miste data permanent ved selv korte nettverksavbrudd, noe som over tid vil gi et ufullstendig bilde av matsvinnet. Den tekniske implementasjonen av mellomlagrings- og gjenforsøksmekanismen er beskrevet nærmere i avsnitt [5.2.3](#).

I tillegg holder systemet oversikt over akkumulert avfallsvekt mellom tømminger, som lagres separat ved deteksjon av tømmehendelse og gir grunnlag for analyse av svinnevolum per tømmeperiode.

Av hensyn til begrenset lagringsplass på Raspberry Pi er mellomlagringen implementert som et sirkulært buffer med plass til fem hendelser. Når kapasiteten er nådd, overskrives den eldste hendelsen. Dette er en akseptabel avveining i en prototype-kontekst der nettverksbrudd antas å være kortvarige.

Valg av Whisper AI

For talegjenkjenning benyttes OpenAI Whisper med base-modellen, som kan kjøres lokalt på Raspberry Pi 4.

Whisper ble valgt fordi modellen har god støtte for norsk, men også gode muligheter for andre språk, som engelsk. Whisper kan også kjøres offline uten avhengighet av eksterne API-er. Sammenlignet med skybaserte alternativer som Google Chirp eller Deepgram Nova-3 gir Whisper full kontroll over dataen, samt med Whisper unngår vi også kostnader per forespørsel.

Ulempen med Whisper er at den er relativt langsom når den blir kjørt på en Raspberry Pi 4, og kan være mindre nøyaktig enn de større modellene. Dette er en akseptabel avveining i prototypefasen, og ytelsen

kan forbedres ved oppgraderinger av prosessorkraft, nøyaktigheten kan forbedres ved å oppgradere til en større versjon dersom maskinvaren tillater det.

Valg av YOLO

For bildegjenkjenning benyttes YOLOv8n. Denne modellen er bedre optimalisert for ressursbegrensede enheter. YOLO ble valgt fordi modellen kombinerer god deteksjonsnøyaktighet med høy hastighet, og er spesielt tilpasset edge enheter som Raspberry Pi. Modellen støtter også videre trening (fine tuning) på egne datasett.

For å tilpasse YOLO modellen til egnet bruksområde, ble det gjennomført trening på datasett fra åpne kilder. Opprinnelig ble et *vegetables classification datasett* fra Kaggle benyttet, som inneholder bilder av ulike grønnsaker kategorisert etter type. Ved praktisk testing ble det vist at classification modellen slet med gjenstander når de ble holdt i en hånd. Dette skyldes at modellen klassifiserer hele bildet som en kategori, og hånden blir da tolket som en del av gjenstanden.

Basert på denne innsikten ble det besluttet å bytte til et *detect basert datasett* fra Roboflow, der hvert bilde inneholder annoteringer (bounding boxes) som markerer nøyaktig hvor i hvert bilde et objekt befinner seg. Testene med detection modellen viste forbedringer når gjenstanden ble holdt i hånden. Dette passer godt med systemets tiltenkte brukerreise som beskrevet i [4.3](#)

Valget av *detection* fremfor *classification* åpner for enklere videreutvikling av produktet. Detection modeller gir ikke bare informasjon om hva som er i bildet men også hvor et objekt befinner seg i bildet og hvor mange det er av objektene. Dette kan benyttes til å telle antall gjenstander som kastes, noe som gir mer detaljert innsikt. For eksempel kan systemet registrere at «3 tomater, totalt 240 gram» ble kastet, fremfor kun «tomater, 240 gram» - dette kan bidra til å gjøre løsningen mer skalerbar og legger til rette for fremtidig funksjonalitet uten å måtte redesigne hele bildegjenkjenningssystemet.

4.7 Protokollvalg

Systemet benytter REST over HTTPS for kommunikasjon med TotalCtrl.

Valget ble tatt på bakgrunn av at TotalCtrl allerede tilbyr et veldokumentert REST-API [12], noe som gjorde integrasjonen betydelig enklere enn å bygge opp en egen meldingsbasert infrastruktur. MQTT ble vurdert som et alternativ på grunn av lavt overhead og egnethet for IoT-enheter med begrenset båndbredde [28], men ble forkastet av tre grunner: det krever en dedikert broker-server, det ville medføre betydelig mer kompleksitet i oppsettet, og TotalCtrl sin plattform er ikke bygget for å konsumere MQTT-meldinger direkte. REST over HTTPS gir tilstrekkelig ytelse for dette bruksområdet, der dataoverføringer skjer hendelsesbasert og ikke kontinuerlig.

Tabell [4.11](#) oppsummerer de viktigste forskjellene mellom de to vurderte protokollene.

Tabell 4.11: Sammenligning av kommunikasjonsprotokoller

Protokoll	Fordeler	Ulemper	Valg
REST/HTTPS	• Enkel integrasjon med eksisterende API • Bred støtte og dokumentasjon • Synkron kommunikasjon • Ingen ekstra infrastruktur	• Høyere overhead per melding • Krever aktiv tilkobling	Valgt
MQTT	• Lavt overhead • God for mange enheter • Asynkron kommunikasjon	• Krever broker server • Mer kompleks oppsett • Må integreres med TotalCtrl	Ikke valgt

Kapittel 5

Utvikling og implementasjon

Dette kapitlet beskriver hvordan systemet er utviklet og implementert, fra fysisk montering av hardware til konkrete programvareløsninger. Først beskrives den mekaniske konstruksjonen og de elektriske tilkoblingene som utgjør grunnlaget for prototypen. Deretter gjennomgås software-implementasjonen, inkludert den modulære arkitekturen, hendelsesbasert dataflyt, vektmåling, talegjenkjenning, bildegjenkjenning og integrasjon mot TotalCtrl sitt API.

5.1 Hardware-implementasjon

Dette delkapitlet beskriver den fysiske monteringen av prototypen, inkludert montering av sensorer, mekanisk konstruksjon og tilkoblinger til Raspberry Pi.

Elektriske tilkoblinger

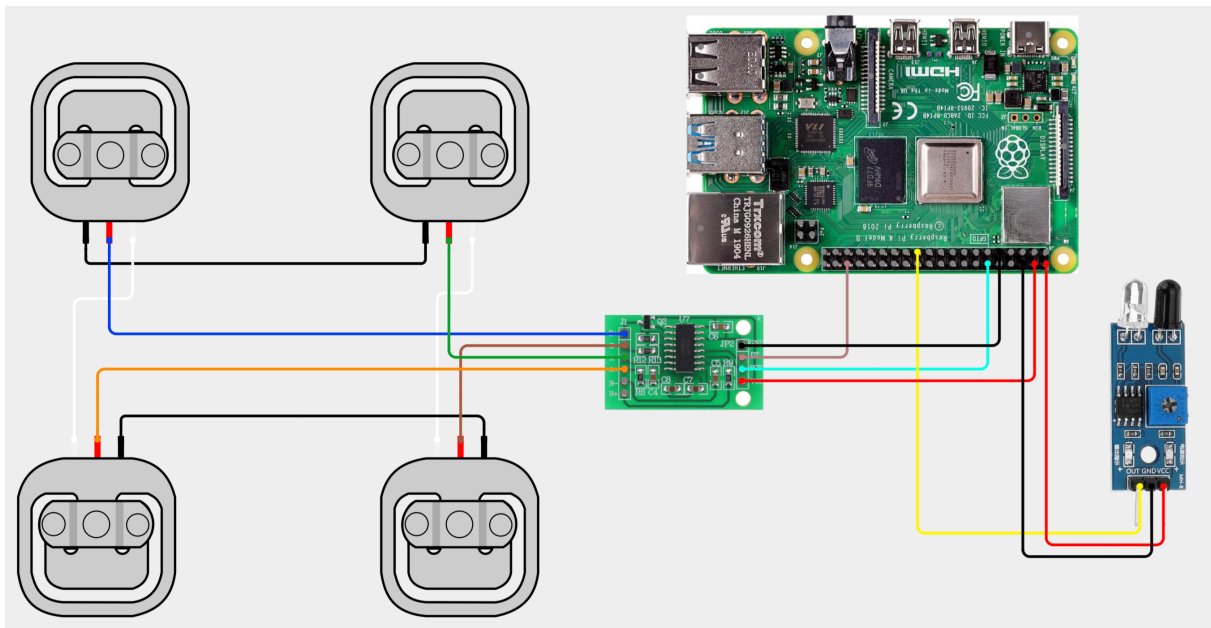
Tabell 5.1 gir en oversikt over hvordan de ulike komponentene er koblet til Raspberry Pi 4.

Tabell 5.1: Tilkoblinger til Raspberry Pi 4

Komponent	Pin (GPIO)	Funksjon
HX711 (DT)	GPIO 16	Data fra vektceller
HX711 (SCK)	GPIO 18	Klokke til HX711
IR-sensor (OUT)	GPIO 11	Triggersignal
Kamera	USB Port	Avbildning
Mikrofon	USB Port	Lydopptak

USB-kameraet med innebygget mikrofon er koblet til én av Raspberry Pi sine USB-porter, og fungerer dermed som én enkelt enhet for både bildeopptak og lydopptak.

Figur 5.1 viser en skjematisk oversikt over hvordan komponentene er koblet sammen.



Figur 5.1: Kretsskjema for prototypen, med fire vektceller koblet til HX711, og IR-sensor koblet direkte til Raspberry Pi 4.

IR-sensor og triggermekanisme

IR-08H infrarød nærhetssensoren er montert ved åpningen på avfallsbeholderen slik at man alltid passer sensoren når man kaster noe. Sensorens rekkevidde kan justeres slik at man kan unngå så mange feillesninger som mulig avhengig av omgivelsene.

Når IR-sensoren detekterer bevegelse, sendes et digitalt LOW signal til Raspberry Pi gjennom GPIO. Dette signalet fungerer som en trigger for vektmålingen og bilde eller lydopptak basert på funksjonsnivået til systemet (L1-L4).

En tidlig utfordring vi møtte på var falske og konstante trigger forårsaket av bevegelser som var nærme sensoren men som ikke var ment for avfallsbeholderen. Dette ble løst ved å justere sensorens følsomhet til potensiometrene på modulen, noe som også kan justeres senere dersom andre problemer skulle oppstå.

Kalibrering av sensorer

For å sikre nøyaktige vektmålinger må vektcellene kalibreres ved førstegangsoppstart. Denne kalibreringsprosessen består av to trinn:

1. **Nullpunktskalibrering (offset):** Alle gjenstander fjernes fra vekten, og systemet registrerer nullpunktet som en basisverdi.
2. **Skaleringskoeffisient (scale):** En kjent referansevekt (1 kg) plasseres på vekten, og systemet beregner forholdet mellom råverdien fra HX711 og faktisk kjent vekt.

Skaleringskoeffisienten som ble beregnet under kalibrering er negativ (-20.62), noe som skyldes orienteringen av vektcellene og polariteten på signalene som HX711 leverer. Dette er normalt for denne typen

oppsett og håndteres automatisk i beregningen. Koeffisienten ble funnet og lagret som en variabel for alle påfølgende målinger, mens nullpunktskalibreringen kjøres på nytt hver gang systemet starter opp. Dette sikrer at målingene er korrekte etter en eventuell omstart, og fungerer samtidig som en automatisk nullstilling av vektgrunnlaget.

IR-sensoren kalibreres ved å justere følsomheten via potentiometeret som er innebygget på modulen. For å verifisere kalibreringen av IR-sensoren beveges en hånd foran sensoren ved ulike avstander og justerer deretter til ønsket deteksjonsrekkevidde.

Montering av vektceller

De fire 50 kg vektcellene er montert på en monteringsplate i en firkantet formasjon, der hver celle utgjør et hjørne. En fast bunnplate ligger over cellene for å sikre jevn vektfordeling, noe som er avgjørende for pålitelige målinger.

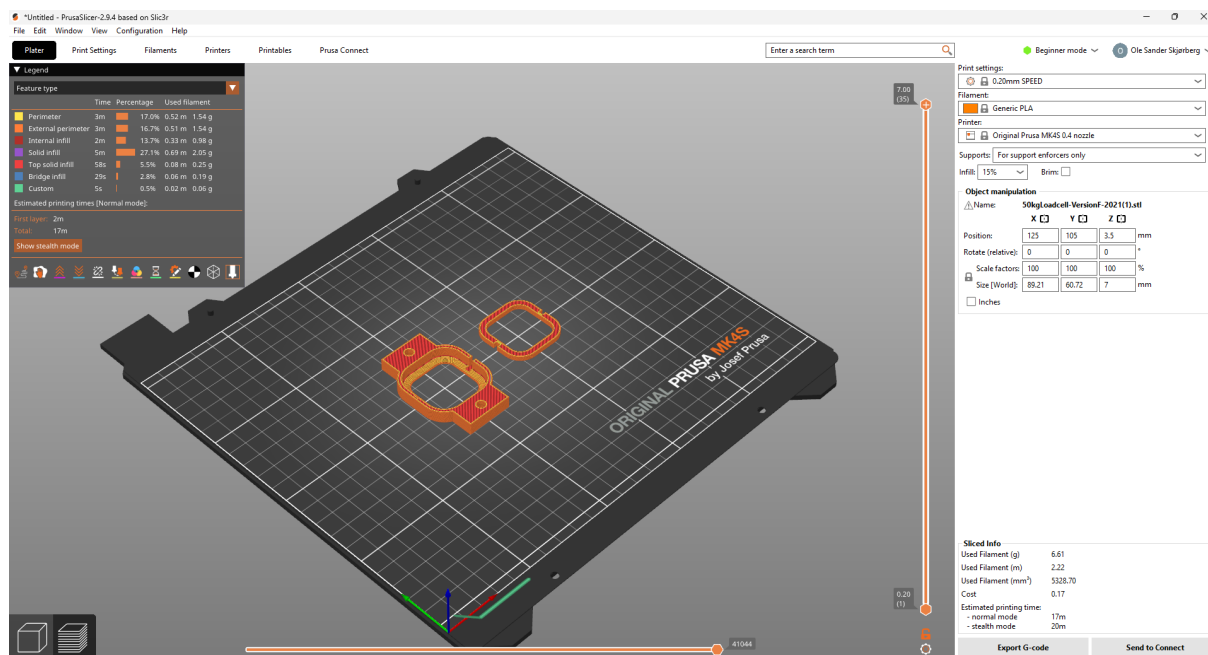
Vektcellene er festet til monteringsplaten med 3D-printede fester. Festene er utformet slik at de holder cellene på plass uten å begrense den mekaniske fjæreffekten som gjør at cellene kan deformeres under belastning. Det er nettopp denne deformasjonen som endrer den elektriske motstanden i cellen og danner grunnlaget for vektberegningen.

Tidlig i utviklingen oppsto ustabile målinger som følge av ujevn montering og feil orientering av cellene. Dette ble løst ved å justere orienteringen og ta i bruk de 3D-printede festene.

3D-printede fester

For å sikre stabil og presis montering av vektcellene ble det designet og 3D-printet egne festebraketter. Som utgangspunkt for designet ble en eksisterende modell fra Thingiverse benyttet som inspirasjon [41], men denne måtte tilpasses prosjektets behov. Ved gjennomgang av originaldesignet ble det identifisert to sentrale utfordringer: kabler risikerte å komme i klem ved montering, og festene var vurdert som lite holdbare ved kontinuerlig bruk over tid. Det ble derfor gjort justeringer i geometrien for å gi kablene bedre klaring og for å styrke festepunktene.

Modellene ble klargjort for printing i PrusaSlicer med følgende innstillinger: 15% infill, generisk PLA-filament (Polyactic Acid) og 0,20 mm lagtykkelse i en SPEED-profil. Printing ble gjennomført på skolens Original Prusa MK4S-printer. Estimert utskriftstid per sett med to fester var 17 minutter i normalmodus. Figur 5.2 viser komponentene slik de fremstår i PrusaSlicer klargjort for printing.



Figur 5.2: Vektcellefestene klargjort for 3D-printing i PrusaSlicer på skolens Original Prusa MK4S.

En praktisk utfordring som oppsto etter utskrift var at kablene fra vektcellene ble klemt ved montering dersom festene ikke ble orientert riktig. Dette ble løst gjennom en kombinasjon av designjusteringer og nøye kabling under fysisk montering. De ferdige komponentene er montert i hjørnene av vektplaten og holder vektcellene stabilt på plass, noe som er avgjørende for pålitelige vektmålinger over tid. I tillegg ble veggtykkelsen på festepunktene økt for å gi bedre robusthet ved kontinuerlig bruk, ettersom originaldesignet ble vurdert som for sårbart for mekanisk slitasje over tid.

Fysisk innkapsling

Prototypen er bygget inn i et mindre skap designet for å illustrere hvordan løsningen kan se ut i et typisk brukermiljø. Skapet etterligner en integrert søppelbøtte slik den kan tenkes implementert i et kjøkken, for eksempel under en benk eller i et skap. Ettersom prototypen er et MVP (Minimum Viable Product) og ikke en ferdig løsning, er det fysiske designet ment som en demonstrasjon av konseptet – den endelige formfaktoren vil videreutvikles av TotalCtrl.

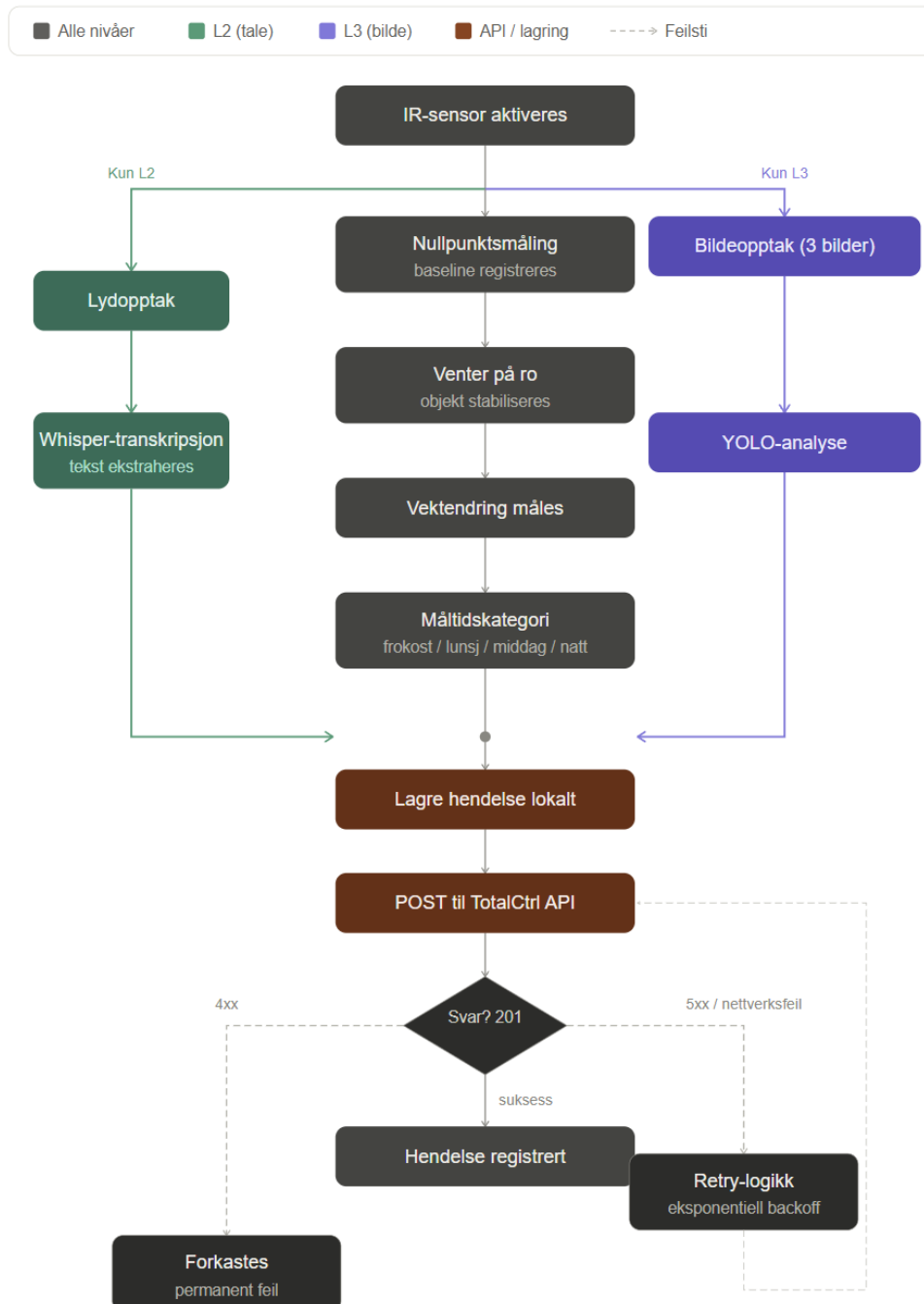
Skapet har en bakplate hvor all elektronikk er montert og kablet. Platen kan løftes opp slik at man enkelt får tilgang til Raspberry Pi, HX711 og øvrig elektronikk for vedlikehold og feilsøking. Vektplaten er montert under søppelbøtten for å minimere støy og vibrasjoner i målingene.

IR-sensoren er plassert ved åpningen der brukeren kaster avfall, slik at den kun trigges ved faktiske hendelser. En mindre hette er montert over åpningen for å huse kameramodulen. Hetten dekker ikke hele bredden av skapet, slik at brukeren enkelt kan holde avfall foran kameraet. Kameraet er plassert maksimalt 35 cm over kastepunktet for å sikre tilstrekkelig bildekvalitet uten å hindre normal bruk.

5.2 Software-implementasjon

Dette delkapittelet beskriver implementasjonen av prototypens software, fra grunnleggende vektmåling til avansert bilde- og talegjenkjenning.

Figur 5.3 gir en overordnet oversikt over hendelsesforløpet fra IR-trigger til API-registrering, og viser hvordan de ulike funksjonsnivåene skiller seg fra hverandre i kjøretid.



Figur 5.3: Hendelsesflyt fra IR-trigger til API-registrering for L1, L2 og L3.

5.2.1 Grunnleggende vektmåling (L1)

L1-løsningen representerer det mest grunnleggende funksjonsnivået i systemet og håndterer automatisk vektmåling uten behov for brukerinteraksjon, ved bruk av en HX711 og fire vektceller.

For kommunikasjon med HX711 analog-til-digital-omformer benyttes Python-biblioteket *hx711*. Biblioteket tilbyr et enkelt grensesnitt for å lese råverdier fra vektcellene via GPIO på Raspberry Pi. Kommunikasjonen skjer via to GPIO-pinner, én for data og én for klokke, som vist i tabell 5.1. HX711 sender 24-bits digitale verdier som representerer den totale belastningen på alle fire vektceller. Disse råverdiene skaleres og nullpunktsjusteres for å gi nyttig vektinformasjon i gram.

Ved oppstart gjennomfører systemet en automatisk kalibreringsprosess som bestemmer nullpunktet og beregner offsetet hver gang systemet starter på nytt. Vekten må være tom under prosessen for å fastsette verdier basert på absolutt null. Systemet tar ti avlesninger over noen sekunder og beregner gjennomsnittet som offsetverdi. I tillegg til offset benyttes en skaleringskoeffisient som ble bestemt gjennom manuell kalibrering med en kjent vekt. Denne verdien konverterer råverdiene fra HX711 til gram.

Funksjonen *measure_weight* er ansvarlig for selve vektmålingen. Den tar en rekke avlesninger over noen sekunder og returnerer gjennomsnittet. Å basere målingen på flere avlesninger over tid er nødvendig fordi én enkelt måling kan inneholde støy som påvirker resultatet. Dersom ingen gyldige avlesninger registreres i løpet av måleperioden, returnerer funksjonen 0,0 gram. Dette kan skje dersom HX711-modulen er frakoblet eller ikke responderer.

Systemet holder oversikt over en dynamisk baseline-vekt som representerer avfallsbeholderens totale vekt til enhver tid. Ved oppstart måles en innledende baseline ved hjelp av *measure_weight*. Hver gang noe kastes, oppdateres baseline til den nye totalvekten. Dette gjør at systemet håndterer flere påfølgende hendelser uten at vekten må nullstilles manuelt.

Dersom systemet registrerer en vektendring under -100 gram, tolkes dette som at avfallsbeholderen er fjernet eller tømt. Hendelsen håndteres i to trinn: først lagres total akkumulert avfallsvekt siden forrige tømning til en egen fil (*total_waste.json*), deretter nullstilles telleren og baseline rekalkuleres ved å ta en ny vektmåling. Dette sikrer at systemet fortsetter å fungere korrekt etter tømning, uten at historiske data går tapt.

Parallelt med hendelsesregistreringen holder systemet løpende oversikt over akkumulert avfallsvekt siden siste tømning. Hver gang en hendelse logges, legges vektendringen til en samlet teller. Denne telleren gir et bilde av total mengde avfall som er kastet siden beholderen sist ble tømt, og lagres ved tømmehendelsen med tidsstempel og måltidskategori. Dette bidrar til mer fullstendig datadekning og er relevant for TotalCtrl sin analyse av svinnmønstre over tid.

Når en gyldig hendelse registreres, lagres dataen lokalt i en JSON-fil gjennom funksjonen *save_event*. Dersom filen allerede eksisterer, leses eksisterende data inn, den nye hendelsen legges til listen, og data-settet skrives tilbake til filen. Umiddelbart etter lagring forsøker systemet å sende hendelsen til TotalCtrl sitt API via *send_waste_event()*. Systemet holder alltid de siste fem hendelsen lagret lokalt. Lokal lag-

ring fungerer som en buffer før dataen overføres til TotalCtrl sitt API, som blir beskrevet senere i dette kapittelet.

Kodesnutt 5.1 viser hvordan det sirkulære bufferet er implementert. Funksjonen `get_next_event_filename()` sykler gjennom filnavnene `waste_event1.json` til `waste_event5.json` og overskriver den eldste filen når kapasiteten er nådd. `save_event()` benytter denne funksjonen til å lagre hendelsen lokalt før den samme dataen umiddelbart forsøkes sendt til API-et via `send_waste_event()`. Dette betyr at lokal backup alltid eksisterer uavhengig av om API-kallet lykkes.

```
1 def get_next_event_filename():
2     global event_counter
3     event_counter = (event_counter % MAX_EVENT_FILES) + 1
4     filename = f"{EVENT_FILE_PREFIX}{event_counter}.json"
5     return filename, event_counter
6
7 def save_event(weight_change, meal, timestamp):
8     global total_waste_weight
9
10    filename, event_num = get_next_event_filename()
11    data = {
12        "event_number": event_num,
13        "timestamp": timestamp,
14        "weight_change_grams": round(weight_change, 2),
15        "meal_category": meal
16    }
17    with open(filename, "w") as f:
18        json.dump(data, f, indent=4)
19
20    total_waste_weight += weight_change
21
22    api_success = send_waste_event(
23        weight_change=weight_change,
24        meal_category=meal,
25        timestamp_str=timestamp,
26        waste_category=None,
27        confidence=None,
28        classification_method="weight"
29    )
30    if not api_success:
31        print("Warning: Event saved locally but not synced to cloud")
```

Listing 5.1: Sirkulært buffer og parallell lokal lagring og API-sending i L1

5.2.2 Talegjenkjenning (L2)

Talegjenkjenningen er implementert med OpenAI Whisper, som ble valgt som modell for dette funksjonsnivået på bakgrunn av vurderingen gjort i avsnitt 4.6. Kort oppsummert ble Whisper foretrukket fremfor skybaserte alternativer som Google Cloud Speech-to-Text fordi modellen kan kjøres helt offline,

ikke har løpende kostnader per forespørsel og gir full kontroll over lyd- og tekstdata. Avveiningen er noe høyere ventetid og lavere nøyaktighet sammenlignet med skybaserte tjenester, noe som er akseptabelt i en prototype-kontekst.

L2-løsningen utvider grunnlaget fra L1 med talebasert kategorisering av matavfallet. Etter at vektendringen er registrert, aktiveres mikrofonen og brukeren sier hva som kastes. Talegjenkjenningsmodellen tolker kommandoen, trekker ut objektnavnet og legger kategorien til hendelsesdataen før den lagres. Feltet `classification_method` settes automatisk til "voice" dersom talegjenkjenningen returnerte et gyldig objektnavn, og til "weight_only" dersom ingen kategori ble identifisert. Dette gjør at `TotalCtrl` i etterkant kan skille mellom hendelser som er fullstendig klassifisert og de som kun er registrert med vekt.

Talegjenkjenningen er implementert som en egen Python-klasse som håndterer hele flyten fra lydopptak til transkripsjon. Klassen initialiseres ved oppstart av L2-systemet og laster inn Whisper base-modellen i minnet. Dette gjør at modellen kun lastes én gang, noe som reduserer ventetiden ved påfølgende gjenkjenninger.

Lydopptak konfigureres med `PyAudio` og følgende parametere: 1024 bytes per buffer, 16 bits lydformat, monokanal, 16 kHz samplingsfrekvens og en opptakstid på noen sekunder som kan justeres ved behov. Når `listen`-funksjonen kalles, åpnes en lydstrøm som leser fra mikrofonen i noen sekunder. Lyddataen samles i en liste og skrives til en WAV-fil, som deretter sendes til Whisper for transkripsjon til tekst. Whisper kalles med språket eksplisitt satt til norsk (`language=no`), noe som både øker gjenkjenningsnøyaktigheten og reduserer latenstiden sammenlignet med automatisk språkdeteksjon.

Den transkriberte teksten sendes videre til `extract_object_name`-funksjonen, som først sjekker om teksten inneholder nøkkelord fra listen `trigger_words`. Denne listen inkluderer varianter som «kaster», «kastet», «kasta», «koster» og «kostet» for å håndtere dialekter og vanlige transkripsjonsfeil fra modellen. Dersom ingen trigger word finnes, returneres `None` og hendelsen logges som en L1-registrering.

Dersom et trigger word er til stede, fjernes forhåndsdefinerte fraser fra teksten ved hjelp av listen `REMOVE_PHRASES`. Denne listen inneholder setninger som «jeg kaster en», «kastet et» og «vi kaster», sortert etter lengde for å sikre at lengre fraser fjernes først. Etter fjerning renses teksten for ekstra mellomrom ved hjelp av `re.sub()`, og resultatet trimmes. Dersom ingen ord gjenstår etter filtreringen, returneres `None` og hendelsen logges igjen som en L1-registrering. Ellers returneres det filtrerte objektnavnet for videre logging.

I L2 kjøres talegjenkjenning og vektmåling parallelt i separate tråder for å redusere total behandlingstid. Når IR-sensoren trigger en hendelse, startes to tråder samtidig:

- **Voice thread:** Kaller `voice.get_object_name`, som tar opp lyd, transkriberer og trekker ut objektnavnet fra det som sies.
- **Weight thread:** Venter noen sekunder for at gjenstanden skal roe seg i avfallsbeholderen, måler vekt over tid og beregner vektendringen.

Begge trådene deler en felles dictionary (`results`) der de lagrer resultatene sine. Hovedtråden venter på at begge er ferdige før dataen kombineres og lagres.

Uten threading ville total behandlingstid vært summen av begge operasjonene. Med parallell kjøring reduseres dette til varigheten av den prosessen som tar lengst tid.

5.2.3 Bildegjenkjenning (L3)

L3-løsningen erstatter talegjenkjenning med bildebasert kategorisering av avfallet. Når IR-sensoren detekterer aktivitet, aktiveres kameraet og tar flere bilder av avfallet. En YOLO-modell analyserer bildene og forsøker å identifisere avfallstypen automatisk.

I motsetning til L1 og L2, der all kode kjøres i samme Python-prosess, benytter L3 en subprocess-basert arkitektur for bildeopptak og bildegjenkjenning. Denne arkitekturen er nærmere beskrevet senere i dette kapitlet.

Funksjonen `capture_waste_images` starter en subprocess som kaller `capture_images` med mappen for bilder som argument. Scriptet styres via Python-biblioteket OpenCV, som brukes til bildeopptak og bildebehandling. Ved oppstart gjennomgår kameraet en oppvarmingsfase der 30 frames leses over tre sekunder uten at bildene lagres. Dette gir kameraet tid til å automatisk justere eksponering, hvitbalanse og fokus før selve opptaket starter, noe som er særlig viktig i et kjøkkenmiljø med varierende lysforhold.

Etter oppvarmingsfasen tas tre bilder med 0,5 sekunders mellomrom og lagres som `waste1.jpg`, `waste2.jpg` og `waste3.jpg`. Flere bilder øker sannsynligheten for at minst ett opptak har tilstrekkelig kvalitet for vellykket objektgjenkjenning, særlig dersom avfallet er i bevegelse når det kastes.

Etter at bildene er tatt, kalles funksjonen `analyze_waste`, som starter en ny subprocess for å kjøre bildegjenkjenningskoden med flagget `-json`. Dette laster inn den trente YOLO-modellen, som er eksportert til ONNX-format (Open Neural Network Exchange) og kjøres via `onnxruntime`-biblioteket. ONNX-formatet gir raskere inferens på CPU enn å kjøre modellen direkte via Ultralytics-biblioteket, noe som er en klar fordel på Raspberry Pi 4 der det ikke finnes GPU-akselerasjon. Deretter analyseres de tre bildene. For hvert bilde gjennomføres følgende: bildet leses inn med OpenCV, skaleres til en størrelse YOLO kan ta imot, normaliseres til verdier mellom 0 og 1, og transponeres fra OpenCV sitt format (høyde, bredde, kanaler) til YOLO sitt format (kanaler, høyde, bredde). Bildet sendes deretter til YOLO-modellen, som returnerer deteksjoner med tilhørende avgrensningsbokser, klasseprediksjoner og konfidensscore.

Modellen analyserer alle tre bildene og aggregerer resultatene på tvers av opptakene. For hvert unikt objekt som detekteres beregnes gjennomsnittlig konfidens over alle bilder der objektet er funnet. Objektet med høyest gjennomsnittlig konfidens returneres som det endelige resultatet. Denne tilnærmingen gir mer stabile resultater enn å basere seg på én enkelt deteksjon, og reduserer påvirkningen av bevegelsesuskarphet og varierende bildekvalitet i enkeltopptak. Scriptet returnerer et JSON-objekt via `stdout`. Hovedkoden leser denne JSON-strengen og henter ut kategori og konfidens. Dersom subprocess-kallet feiler, returneres kategorien som `unknown` med konfidens 0. Dersom timeout inntreffer, returneres `error` med konfidens 0.

Etter at modellen har kjørt, filtreres overlappende deteksjoner av samme objekt ved hjelp av Non-Maximum Suppression (NMS). Dette sikrer at hvert objekt kun telles én gang selv om modellen skulle returnere flere overlappende avgrensingsbokser for samme gjenstand.

I motsetning til L2, der tale og vekt kjøres parallelt, kjøres bildeopptak og vektmåling i sekvens. Dette er fordi brukeren må holde avfallet foran kameraet før det kastes, og bildene må tas før gjenstanden treffer vektcellene. Hendelsen følger derfor denne sekvensen:

- IR-sensoren detekterer bevegelse
- *capture_waste_images* kalles og tar tre bilder
- Systemet venter på at gjenstanden skal treffe vektcellene og roe seg
- *measure_weight* måler vektendringen
- *analyze_waste* analyserer bildene med YOLO
- Resultater kombineres og lagres via *save_event*

Med denne rekkefølgen sikres det at bildene tas mens avfallet fortsatt er synlig foran kameraet, og at vektmålingen skjer etter at gjenstanden har roet seg.

L3 bygger på grunnlaget fra L1 og L2 og gjenbruker funksjoner som *measure_weight*, *determine_meal* og grunnstrukturen i *save_event*. Forskjellen er at JSON-objektene nå inkluderer *waste_category* og *confidence*. Dersom bildegjenkjenningen feiler eller returnerer lav konfidens, settes *waste_category* til *unknown* og *confidence* til 0, og hendelsen lagres på samme måte som i L1-løsningen. På samme måte som i L2 settes *classification_method* til "image" ved vellykket gjenkjenning, og til "weight_only" ved feil eller lav konfidens.

Modellen som benyttes i L3 er trent på et deteksjonsdatasett fra Roboflow og støtter gjenkjenning av 12 kategorier, som vist i tabell 5.2. Kategoriene er valgt på bakgrunn av hva som er vanlig matavfall i et norsk kjøkkenmiljø, og dekker grønnsaker og råvarer som typisk forekommer i produksjonssvinn.

Tabell 5.2: Kategorier støttet av YOLO-modellen i L3-løsningen

Nr.	Kategori
1	Rødbete
2	Paprika
3	Kål
4	Gulrot
5	Agurk
6	Egg
7	Aubergine
8	Hvitløk
9	Løk
10	Potet
11	Tomat
12	Zucchini

API-integrasjon

API-integrasjonen er implementert i tråd med designet beskrevet i avsnitt 4.6, ved hjelp av Python-biblioteket `requests` som håndterer HTTP-kommunikasjon mot TotalCtrl sitt stagingendepunkt. Ved hver registrert hendelse bygges en JSON-payload opp dynamisk basert på hvilket funksjonsnivå som er aktivt. For L1-løsningen inkluderes kun vekt og tidsstempel, mens L2 og L3 i tillegg legger til klassifiseringsmetode, kategori og konfidensscore.

Autentisering skjer ved at API-nøkkelen sendes i HTTP-headeren `X-API-Key` for hver forespørsel. Nøkkelen er lagret lokalt i en konfigurasjonsfil på enheten og lastes inn ved oppstart.

Ved et vellykket kall returnerer API-et HTTP 201 Created, og hendelsen anses som registrert. Dersom kallet feiler skilles det mellom to typer feil: nettverksfeil og serverfeil (5xx) behandles som midlertidige, og hendelsen lagres da lokalt mens systemet forsøker å sende den på nytt med gradvis økende venteintervall mellom hvert forsøk, for å unngå unødvendig belastning på API-et. Klientfeil (4xx) behandles derimot som permanente og utløser ikke ny sending, da disse typisk indikerer feil i dataformatet som ikke vil løse seg av seg selv. Tidsstempelet settes lokalt på enheten idet hendelsen registreres, slik at korrekt tidspunkt bevares også ved forsinkede overføringer. I tillegg til vanlige avfallshendelser håndterer systemet også tømmehendelser for hele avfallsbeholderen via `send_emptying_event()`. Når systemet detekterer en stor negativ vektendring (under -100 gram), tolkes dette som at avfallsbeholderen er fjernet eller tømt. Systemet sender da en egen hendelse til API-et med total akkumulert avfallsvekt siden forrige tømning. Dette sikrer at TotalCtrl får fullstendig oversikt over løpende avfallshåndtering og totalt volum per tømmeperiode.

Hendelser som ikke kan sendes overføres til en lokal JSON-fil på enheten og synkroniseres med API-et når forbindelsen gjenopprettes.

Subprocess arkitektur

For L3 og L4 benyttes en subprocess-basert arkitektur der bildeopptak og bildegjenkjenning kjøres som separate prosesser fremfor å importeres som moduler i hovedkoden. `capture_images.py` og `image_recognition.py` startes via Pythons `subprocess.run()`, noe som gir hver prosess sin egen minneallokering, prosess-ID og kjøretidsmiljø. Hovedkoden fungerer dermed som en koordinator fremfor å kjøre alt i én og samme applikasjon.

Denne tilnærmingen gjør det mulig å isolere feil: dersom ett av subprocess-skriptene krasjer, kan hovedkoden fortsatt opprettholde driften. Den løser også avhengighetskonflikter, ettersom bibliotekene som kreves for bildegjenkjenning — særlig `onnxruntime` og `ultralytics` — ikke lot seg kombinere med resten av systemets avhengigheter uten versjonskonflikter.

Kommunikasjon mellom hovedprosessen og underprosessene skjer via `stdout` og `stderr`. Ved vellykket kjøring returnerer `image_recognition.py` et JSON-objekt via `stdout` med kategori og konfidens. Feilmeldinger fanges opp via `stderr` og skrives til loggfilen, slik at hovedkoden kan fortsette driften selv om bildegjenkjenningen skulle feile. For å støtte både integrasjon og manuell testing er `image_recognition.py` utviklet med to kjøremodi. Når scriptet kalles med flagget `-json`, som er tilfellet fra `L3.py` via subprocess, returneres kun et JSON-objekt via `stdout`. Uten flagget kjøres en utvidet analysemodus med detaljert output for hvert bilde, beregnet på manuell testing og feilsøking under utvikling.

For å hindre at langvarige operasjoner blokkerer systemet er alle subprocess-kall satt opp med en timeout på 30 sekunder. Dersom bildeopptak eller bildegjenkjenning ikke fullføres innen denne grensen, avbrytes prosessen automatisk og hendelsen lagres kun med vekt og tidsstempel, tilsvarende en L1-registrering.

Kodesnutt 5.2 viser hvordan subprocess-kallene er implementert i `L3.py`. `capture_waste_images()` starter bildeopptak i et eget virtuelt miljø via en dedikert Python-tolker, mens `analyze_waste()` kaller bildegjenkjenningsskriptet med flagget `-json` slik at kun et strukturert JSON-objekt returneres via `stdout`. Begge kallene har en timeout på 30 sekunder. Ved timeout eller annen feil returneres henholdsvis kategorien `error` med konfidens 0.0, og hendelsen degraderes til en L1-registrering uten klassifisering.

```
1 def capture_waste_images():
2     try:
3         result = subprocess.run(
4             ['python3', 'Capture_images.py', IMAGE_FOLDER],
5             capture_output=True, text=True, timeout=30
6         )
7         if result.returncode == 0:
8             print(result.stdout)
9         else:
10            print(f"Feil ved bildetaking: {result.stderr}")
11    except Exception as e:
12        print(f"Feil: {e}")
13
14 def analyze_waste():
15     try:
```

```

16     result = subprocess.run(
17         ['python3', 'image_recognition.py', '--json'],
18         capture_output=True, text=True, timeout=30
19     )
20     if result.returncode == 0:
21         data = json.loads(result.stdout)
22         return data
23     else:
24         return {"waste_category": "unknown", "confidence": 0.0}
25
26 except subprocess.TimeoutExpired:
27     return {"waste_category": "error", "confidence": 0.0}
28 except Exception as e:
29     return {"waste_category": "error", "confidence": 0.0}

```

Listing 5.2: Subprocess-kall for bildeopptak og bildegjenkjenning i L3

Måltidskategorisering

Måltidsdeteksjon er en sentral del av systemet, som er implementert i `determine meal()` funksjonen. Denne funksjonen sitt formål er å bestemme hvilket måltid søppelen som blir kastet hører til basert på hvilket tidspunkt avfallsregistreringen skjer. Disse kategoriene kan være frokost, lunsj, middag eller natt.

- **Frokost** tidsrommet mellom kl 05.00 og 09.00
- **Lunsj** tidsrommet mellom kl 09.00 og 13.00
- **Middag** tidsrommet mellom kl 13.00 og 18.00
- **Kveldsmat** tidsrommet mellom kl 18.00 og 00.00
- **Natt** tidsrommet mellom kl 00.00 og 05.00

Kodestruktur

Koden for prosjektet er delt opp i flere separate filer, som bidrar til å gjøre systemet mer oversiktlig og enklere å navigere i. Hver modul fokuserer på spesifikke funksjoner, som hjelper til med å opprettholde en forståelig kodebase.

Systemet består av flere python script som hver dekker en avgrenset del av funksjonaliteten:

- **L1.py, L2.py, L3.py:** Hovedscript for hvert funksjonsnivå som styrer hendelseslogikken
- **Image-recognition.py:** Håndterer YOLO basert bildegjenkjenning og returnerer kategori og konfidens
- **Capture-images.py:** styrer kameraet og tar flere bilder med oppvarmingsfase
- **Config.py:** Håndterer filbaner og fil størrelser for bildegjenkjennings logikken

- **Voice-recognition.py:** Håndterer lydopptak, whisper oversetting og tekstfiltrering

Ved å dele opp koden i separate moduler, får vi en god gjenbrukbarhet. Hver modul kan testes og oppdateres uavhengig av de andre modulene, noe som bidrar til å forenkle vedlikehold og videre utvikling. I tillegg kan enkelte moduler bli tatt i bruk i andre fremtidige prosjekter, sånn som image recognition.py kan bli gjenbrukt i andre prosjekter som også kan trenge bildegjenkjenning.

Hvert script er bygget opp rundt en samling av dedikerte funksjoner som utfører spesifikke oppgaver. Dette betyr at hver funksjon har ett klart definert ansvar. Funksjoner som `measure_weight()` leser råverdier fra HX711 og beregner gjennomsnitt over flere målinger, mens `determine_meal()` bestemmer måltidskategorisering basert på tidspunkt. Denne funksjonsbaserte tilnærmingen gjør koden mer lesbar og gjør det enklere å teste hver funksjon isolert.

Alle funksjonene koordineres gjennom en hovedloop som kjører kontinuerlig, etter et system start oppsett, og lytter etter hendelser fra IR sensoren. Når sensoren detekterer, kalles relevante funksjoner basert på dataflyten som beskrevet i 4.6.

Systemet benytter en sentral konfigurasjonsfil (`config.py`) for å samle innstillinger som kan endres uten å modifisere hovedkoden. Filen inneholder blant annet stier til ONNX-modellen og tilhørende klassenavn, bildestørrelse for inferens (320 piksler), konfidensterskel for gyldige deteksjoner (60 %) og IOU-terskel for Non-Maximum Suppression (0,45), der IOU (Intersection over Union) styrer hvor mye to avgrensningsbokser må overlappe før den med lavest konfidens forkastes. Deteksjoner under konfidensterskelen forkastes slik at hendelsen i stedet registreres som en L1-hendelse uten klassifisering. Disse verdiene kan justeres uten å endre selve koden, noe som gjør det enkelt å tilpasse systemets sensitivitet etter behov eller ved overgang til en oppdatert modell.

Feilhåndtering

Systemet er designet for å håndtere feil på komponentnivå uten å stoppe den overordnede driften. Feilhåndteringen skiller mellom to kategorier: midlertidige og permanente feil.

Dersom HX711 ikke responderer innen forventet tid, returnerer `measure_weight()` verdien 0.0 gram og hendelsen forkastes. Dette forhindrer at ugyldige målinger lagres eller sendes videre.

IR-sensoren kan i enkelte omgivelser gi falske triggere forårsaket av bevegelser i nærheten av beholderen. Dette håndteres ved justering av følsomheten via potensiometeret på modulen. I tillegg krever systemet en minimum vektendring for at en hendelse skal registreres, noe som fungerer som et programvarebasert filter mot støytriggere.

Ved *nettverksfeil* eller *serverfeil* (5xx) fra API-et lagres hendelsen lokalt i en JSON-fil og systemet forsøker å sende den på nytt med gradvis økende venteintervall mellom hvert forsøk. Dette sikrer at data ikke går tapt ved midlertidig nettverksbrudd. *Klientfeil* (4xx) behandles derimot som permanente, da disse typisk indikerer feil i dataformatet som ikke vil løse seg av seg selv, og utløser derfor ikke ny sending.

For *subprocess-kallene* i L3 og L4 benyttes en timeout på 30 sekunder. Dersom bildeopptak eller bildegjenkjenning ikke fullfører innen denne grensen, returneres henholdsvis `error` med konfidens 0, og

hendelsen lagres som en L1-hendelse uten klassifisering. Kommunikasjon mellom hovedprosessen og subprosessene skjer via `stdout` og `stderr`, der subprosessene returnerer JSON-strukturert output ved suksess og feilmeldinger via `stderr` ved feil.

Logging og feilsøking

Under utvikling ble `print()`-setninger benyttet aktivt for å følge systemets tilstand i sanntid – blant annet for å verifisere at IR-sensoren trigget korrekt, at vektmålinger lå innenfor forventet område, og at API-kall returnerte forventet statuskode. Etter hvert som systemet ble mer komplett, ble de viktigste hendelsene samlet i en strukturert loggfil ved hjelp av Pythons innebygde `logging`-bibliotek.

Loggfilen registrerer tidsstempel, hendelsestype og eventuelle feilmeldinger for hver kjøring. Dette gjør det enklere å spore opp problemer i ettertid, særlig for feil som ikke reproduseres enkelt under manuell testing – som nettverksfeil ved API-overføring eller timeout i subprocess-kall.

Feil fra subprosessene i L3 fanges opp via `stderr` og skrives til loggen, slik at hovedprosessen kan fortsette driften selv om bildeopptak eller bildegjenkjenning skulle feile. Dette er beskrevet nærmere i avsnitt [5.2.3](#).

Kapittel 6

Testing og resultater

Dette kapitlet beskriver testingen av prototypen og presenterer resultatene som er oppnådd. Først redegjøres det for testmetodikken som er benyttet, inkludert komponenttesting, integrasjonstesting og funksjonell testing. Deretter beskrives systemsimulatoren som ble utviklet i en tidlig fase, etterfulgt av de fysiske testene som ble gjennomført på hardware-komponentene. Kapitlet avsluttes med en gjennomgang av testresultatene.

6.1 Testmetodikk

Testing av prototypen ble gjennomført i flere faser basert på en systematisk tilnærming som kombinerer enhetstesting, integrasjonstesting og systemtesting. Teststrategien følger prinsippene fra testpyramiden, der grunnleggende komponenttester utgjør fundamentet før mer komplekse integrasjonstester gjennomføres.

Isolert komponenttesting

Hver sensor og funksjonalitet ble først testet separat fra det fullstendige systemet for å verifisere grunnleggende funksjonalitet før integrasjon. Dette inkluderte testing av vektceller, IR-sensor, kamera og mikrofon uavhengig av hverandre. Formålet var å identifisere og løse problemer på et grunnleggende nivå for hver komponent før de ble kombinert til et større system.

For vektcellene ble det utviklet et eget testprogram som leser råverdier fra HX711 og konverterer disse til gram. Koden skriver kontinuerlig ut rå data fra HX711 samtidig som den kalkulerer dataen i gram, dette ga en god forståelse på hvordan de rå dataene ble konvertert til gram. Programmet ble holdt så enkelt som mulig for å unngå feil som kan oppstå ved komplisert logikk eller andre avhengigheter. Kjente referansevekter ble benyttet for å validere nøyaktigheten til cellene og hvor stort avviket ble.

IR-sensor ble testet ved å overvåke triggersignaler til Raspberry Pi og verifisere at GPIO pinnene mottok korrekte HIGH/LOW verdier ved deteksjon av bevegelse. Dette sikrer at sensoren fungerer som forventet før integrasjon.

For å validere YOLO modellens ytelse ble det utviklet et live testprogram som viser sanntidsdeteksjoner med boundingboxes og konfidensscore. Programmet åpner USB kamera, kjører YOLO modellen på hver frame, og visualiserer direkte i kamera visningen de detekterte objektene med boundingboxes og konfidens.

Talegjenkjenning ble testet ved å kjøre `voice_recognition.py` i testmodus og evaluere den oversatte tekstens nøyaktighet og filtreringslogikken. Programmet tar opp 3 sekunders lyd, transkriberer med Whisper, og trekker ut objektnavn ved hjelp av trigger ord og frase fjerning.

Integrasjonstesting

Etter at hver komponent var verifisert individuelt ble de integrert i systemet stegvis. Integrasjonstestingene fokuserte på å sikre at komponentene kommuniserte korrekt med hverandre og at dataflyten mellom hver modul fungerte som forventet.

For L1 løsningen ble vektmåling og IR-sensor testet sammen. Når IR-sensoren ble trigget skulle systemet utføre vektmålingen og lagre resultatene i en JSON fil. Suksesskriteriet var at hver triggerhendelse resulterte i korrekt format i JSON filen, samt at trigger hendelser som ikke møtte kravene for vekt endring ikke ble registrert.

For L2 løsningen (tale) ble mikrofon og talegjenkjenning lagt til. Testingen fokuserte på å verifisere at lydopptak startet ved riktig tidspunkt, at whisper modellen returnerte transkribert tekst og at denne teksten ble korrekt kategorisert før den ble kombinert med vektdata fra L1.

For L3 løsningen (bilde) ble kamera og bildegjenkjenning lagt til. Her var fokuset på å sikre at kameraet tok gode bilder, YOLO modellen kunne identifisere objektene og at bilder ble tatt ved riktig tidspunkt under integrasjon med den fullstendige løsningen.

Funksjonell testing

Funksjonell testing ble utført for å bekrefte at systemet oppfylte de definerte brukstilfellene beskrevet i 4.3. Dette inkluderte ende til ende testing hvor det ble gjennomført en komplett arbeidsflyttest: fra triggering av IR-sensor gjennom vektmåling og eventuell tale eller bildegjenkjenning, til lagring av data lokalt før API overføring.

Testmiljø

All testing ble gjennomført under kontrollerte forhold med prosjektets prototypeoppsett bestående av Raspberry Pi 4, vektceller, IR-sensor, kamera og mikrofon. Testene ble utført innendørs, der lysforhold, plassering av komponenter og avstand til kamera og sensorer kunne justeres etter behov.

Dette gjorde det enklere å gjennomføre gjentatte tester og feilsøke problemer under utviklingen. Talegjenkjenningen ble testet med begrenset bakgrunnsstøy, mens bildeanalysen ble gjennomført under stabile lysforhold for å sikre konsistent bildekvalitet.

Selv om dette ga gode forutsetninger for å evaluere systemets funksjonalitet, representerer testene ikke et fullt realistisk kjøkkenmiljø. Faktorer som høyt støynivå, varierende lysforhold, vibrasjoner og hektisk aktivitet kan påvirke systemets ytelse i praksis.

6.2 System-simulator

I sammenheng med prototype-utvikling ble det utviklet en simulator for å kunne teste og verifisere systemets funksjonalitet i en tidlig fase. Simulatoren gjør det mulig å validere systemarkitektur, datamodell, hendelseslogikk og API-design uten å være avhengig av fysisk hardware. Systemet er lagd i en Python-applikasjon og er modulær, som vil si at den er delt opp i flere deler som representerer en funksjon i det ferdige systemet. Målet er at simulerte objekter enkelt kan erstattes med ekte maskinvare, uten at systemet nødvendigvis må endres. Systemet tar kun utgangspunkt i bruk av stemme/mikrofon.

Simulering av vekt og stemme

Simulatoren består av blant annet en vektdel som representerer en enkel versjon av en smartvekt. Når simulatoren kjøres genereres en tilfeldig vektøkning innenfor en realistisk grense, som representerer at matavfall blir kastet i søpla. Den holder også oversikt over beholderens totalvekt til enhver tid og oppdaterer den for hvert «kast». Dette tilsvarer hvordan en fysisk vekt med vektcelle ville fungert i praksis.

Parallelt simuleres stemmebasert brukerinteraksjon istedenfor mikrofon og talegjenkjenning. Simulatoren genererer forhåndsregistrerte tekstfraser som resultatet av en tale-til-tekst-prosess, for eksempel «jeg kaster et bananskall» eller «gammel melk». Dette gjør det mulig å teste hele systemflyten knyttet til stemmebruk uten behov for ekstra maskinvare eller avansert programvare.

Klassifisering og hendelseslogikk

Basert på tekstinput fra stemme-simuleringen skjer en enkel kategorisering av matavfallet. Simulatoren søker etter bestemte nøkkelord i teksten og knytter disse til forhåndsdefinerte kategorier som frukt, meieriprodukter og middagsrester. Denne kategoriseringen bidrar til å gi en overordnet kategori og kan gi et mer spesifikt produkt dersom det kan identifiseres.

Når stemmeinput og vektendring er tilgjengelig, kombineres de til én samlet hendelse som representerer ett enkelt tilfelle av matavfall og inneholder informasjon om hvor hendelsen skjedde, når, hvor mye og hvilke avfall som ble kastet. All denne informasjonen samles i et JSON-objekt som følger samme format uavhengig av om dataene er simulert eller hentet fra fysisk hardware.

Kommunikasjon med backend og bruk i utvikling

Hver hendelse sendes fra simulatoren til et backend-API via HTTP. Når API-et mottar hendelsen er det mulig å lagre, vise og analysere dataene videre. Siden simulatoren og backend-løsningen følger samme dataformat, kunne de testes mot hverandre uavhengig av fysisk hardware. Simulatoren bidro til å avdekke feil og mangler tidlig og ga mulighet til å justere datamodellen før fysisk implementasjon. Dette bidro til å redusere risikoen for tekniske problemer senere, samtidig som systemdesignet forble fleksibelt og lett å utvide. Figur 6.1 viser utkast av en simulert hendelse.

```
[
  {
    "event_id": "f07a8f29-2767-4f5a-89c9-bdf8647b26fd",
    "device_id": "smartwaste-sim-01",
    "timestamp": "2026-02-25T09:16:48.282086Z",
    "weight": {
      "delta_grams": 271,
      "total_grams": 271
    },
    "voice": {
      "raw_text": "rester av spaghetti",
      "language": "no",
      "confidence": 0.9
    },
    "classification": {
      "category": "middagsrester",
      "item": "spaghetti",
      "method": "rule_based",
      "confidence": 0.8
    },
    "metadata": {
      "simulated": true
    }
  }
]
```

Figur 6.1: En enkelt simulert hendelse hentet fra test-API

Begrensninger

Systemet har noen begrensninger da det kun representerer en forenklet modell av det endelige systemet. Vektdataene er basert på tilfeldige verdier som ikke tar hensyn til faktorer som sensorstøy, små vektendringer eller feilregistrering som kan oppstå i en fysisk hardware. Siden stemmeinput er simulert som en ferdig tekst vil den ikke reagere på utfordringer som talegjenkjenning, bakgrunnsstøy, dialekter eller typiske feiltolkninger. Kategoriseringen av matavfallet er regelbasert og begrenset til en håndfull forhåndsdefinerte ord. Dette gjør den mindre fleksibel enn andre AI-baserte løsninger.

Overgangen til fysisk implementasjon

Simulatoren er utviklet med tanke på overgang til fysisk hardware, og de simulerte komponentene kan erstattes med vektceller og mikrofon uten store endringer i resten av systemet. Dette gjorde simulatoren til et nyttig verktøy for tidlig evaluering av systemets design før prototypen ble bygget.

6.3 Fysiske tester

Dette delkapittelet beskriver de fysiske testene som ble gjennomført på prototypens hardware komponenter, samt utfordringer som oppsto underveis og løsninger som ble implementert.

Vektceller og vektmåling

De første testene av vektcellene avdekket utfordringer med stabile målinger. Når en kjent vekt referanse på 650 gram (en tre fjøl) ble plassert på vekten, varierte avlesningene med opptil 200 gram mellom målingene. Dette var langt utenfor akseptabel nøyaktighet.

Testene viste at ustabiliteten hovedsakelig skyldes to faktorer: ujevn montering og feil orientering av vektcellene kombinert med dårlige koblinger mellom cellene og HX711 modulen. Den opprinnelige måten å montere cellene på holdt ikke cellene stabilt nok, og de små forskjellene førte til skjevbelastning. For å løse dette ble det designet og 3D printet nye monteringsfester som beskrevet i [5.1](#). De nye festene sikrer alle fire cellene, som gjør at de er montert med samme vinkler slik at lasten blir fordelt jevnt. I tillegg ble de montert på en stivere underplate for å redusere fleksibiliteten til platen.

Etter forbedringene ble vektmålingene testet på nytt med samme vekt som referanse. Avlesningene viste mindre avvik på noen få gram, noe som ble sett på som akseptabel for prototypen. Ytterligere tester med ulike vekter bekreftet at systemet ga konsistente resultater innenfor akseptabelt område, noe som beskrives nærmere i delkapittel [6.4](#).

IR-sensor

IR-sensoren ble testet isolert før den ble integrert i det fullstendige systemet. Testingen besto i å overvåke GPIO-pinnen sensoren er koblet til, og verifisere at Raspberry Pi mottok korrekte HIGH- og LOW-verdier ved deteksjon av bevegelse. En hånd ble beveget foran sensoren ved ulike hastigheter og avstander for å kartlegge deteksjonsområdet og responsens pålitelighet.

De første testene avdekket at sensoren reagerte på bevegelser som ikke var tiltenkt som hendelser, for eksempel folk som beveget seg i nærheten av prototypen. Dette resulterte i falske triggere som ville ha ført til uønskede registreringer i systemet. Problemet ble løst ved å justere sensorens rekkevidde og følsomhet via det innebygde potensiometeret på modulen, til deteksjonen ble begrenset til området rett over åpningen på avfallsbeholderen.

Etter justeringen ble sensoren verifisert på nytt. Systemet responderte konsekvent på håndbevegelser innenfor det tiltenkte deteksjonsområdet, mens bevegelser utenfor dette området ikke lenger utløste triggere. GPIO-signalet ble bekreftet å gå fra HIGH til LOW ved deteksjon og tilbake til HIGH umiddelbart etter, i tråd med forventet oppførsel beskrevet i avsnitt [2.1](#) og implementasjonen i avsnitt [5.1](#).

Sensoren viste seg å være robust mot omgivelseslys under testforholdene. Den modulerte IR-deteksjonen filtrerte effektivt ut støy fra kunstig belysning i laboratoriet, og ingen falske triggere ble registrert som følge av lysforholdene alene. Under integrasjonstesting med det fullstendige systemet fungerte sensoren som forventet som hendelsestrigger for vektmåling og bilde- eller lydopptak avhengig av funksjonsnivå.

Kamera og bildegjenkjenning

Bildegjenkjenningen ble testet med et utvalg av vanlige kjøkkenobjekter, herunder grønnsaker som gulrøtter, paprika og brokkoli, samt frukt og enkelte meieriprodukter. Objektene ble holdt foran kameraet på ulike avstander opp til maksimalt 35 cm, som beskrevet i avsnitt 5.1.

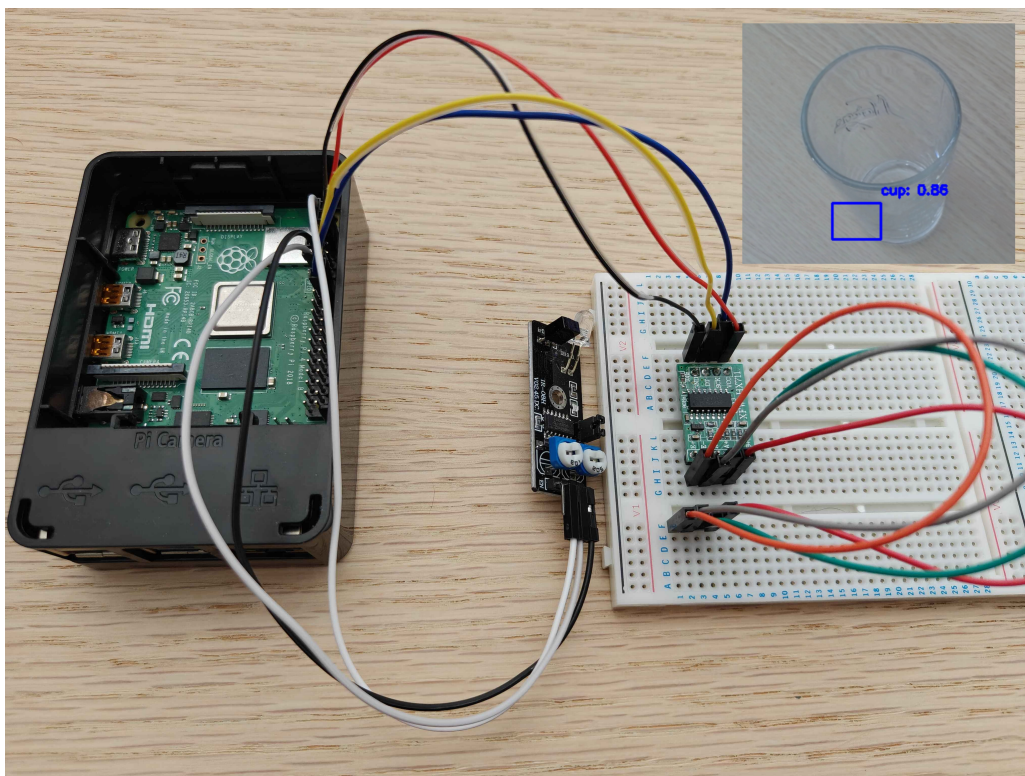
Et problem som dukket opp tidlig i testingen var at kameraet leverte helt svarte bilder ved oppstart. Dette skyldtes at eksponeringstiden var for kort til at kameraet rakk å tilpasse seg lysforholdene i rommet. Løsningen var å innføre en oppvarmingsfase der kameraet leser inn 30 frames over tre sekunder uten å lagre bildene, slik at automatisk justering av eksponering og hvitbalanse kan fullføres før selve opptaket starter. Dette løste problemet og ga konsistent bildekvalitet under testforholdene.

Testingen avdekket også utfordringer knyttet til klassifiseringsmodellen. Den opprinnelige modellen var en klassifikasjonsmodell som tilordner én kategori til hele bildet. I praksis viste det seg at hånden som holder objektet ble tolket som en del av gjenstanden, noe som førte til feilklassifiseringer. Dette var en direkte konsekvens av at modellen ikke skiller mellom objektet og omgivelsene. Basert på denne innsikten ble det besluttet å bytte til en deteksjonsmodell med annoterte avgrensningsbokser, som beskrevet i avsnitt 4.6. Deteksjonsmodellen viste tydelig forbedring når objekter ble holdt i hånden, da modellen nå identifiserer objektet uavhengig av hva som ellers er i bildet.

Lysforhold hadde merkbar innvirkning på resultatene. Under gode lysforhold med kunstig belysning i laboratoriet ga modellen konsistente deteksjoner med høy konfidens. I situasjoner med dårligere lys falt konfidensen tydelig, og enkelte objekter ble ikke gjenkjent. Dette bekrefter behovet for en dedikert lyskilde som beskrevet i avsnitt 8.1, for å sikre tilstrekkelig bildekvalitet uavhengig av omgivelsesbelysningen.

For å redusere påvirkningen av bevegelsesuskarphet og varierende bildekvalitet ble det implementert opptak av tre bilder per hendelse med 0,5 sekunders mellomrom, der deteksjonen med høyest konfidens benyttes som grunnlag for klassifiseringen. Dette ga mer stabile resultater enn enkeltbilder alene, særlig for objekter i bevegelse.

Figur 6.2 viser det fysiske testoppsettet med Raspberry Pi 4, IR-sensor og HX711, samt et eksempel på YOLO-deteksjon under testing.



Figur 6.2: Testoppsett med Raspberry Pi 4, IR-sensor og HX711 koblet til et koblingsbrett, med et eksempel på YOLO-deteksjon av en kopp vist øverst til høyre.

Talegjenkjenning

Talegjenkjenningen ble testet ved å si ulike kommandoer til systemet under varierte forhold. Kommandoene dekket naturlige formuleringer en kjøkkenansatt ville brukt, som «jeg kaster poteter», «kaster gammel melk», «kastet litt pasta» og kortere varianter som bare «gulrøtter» eller «brød». Formålet var å kartlegge både transkripsjonsnøyaktigheten til Whisper og robustheten til filtreringslogikken i `extract_object_name`.

Under stille forhold fungerte talegjenkjenningen godt. De fleste kommandoer ble korrekt transkribert, og filtreringsfunksjonen klarte å trekke ut objektnavnet fra setningene. Trigger words som «kaster» og «kastet» ble gjenkjent som forventet, og REMOVE_PHRASES-listen fjernet omgivende ord slik at kun objektnavnet ble sittende igjen.

Ved testing med bakgrunnsstøy, som musikk eller samtaler i rommet, ble nøyaktigheten merkbart redusert. Whisper transkriberte av og til feil ord eller la til ord som ikke ble sagt. Et gjennomgående eksempel var at ordet «kaster» ble transkribert som «koster», noe som ikke treffer trigger word-listen og dermed fører til at hendelsen logges som en L1-registrering uten kategori. For å håndtere dette ble «koster» og «kostet» lagt til i trigger word-listen som dialekt- og feilvarianter, noe som forbedret gjenkjenningen.

En annen utfordring var kommandoer uten trigger word, for eksempel når brukeren bare sa objektnavnet direkte uten noen innledende setning. I slike tilfeller returnerte `extract_object_name` verdien `None`, og hendelsen ble logget uten kategori. Dette er en kjent begrensning som kan adresseres ved å justere filtreringslogikken til også å akseptere enkeltord som gyldige kommandoer.

Latenstiden var også en merkbar faktor under testing. Whisper base-modellen brukte typisk mellom fem og ti sekunder på å transkribere et opptak på noen sekunder, noe som oppleves som en tydelig pause for brukeren. Dette er en direkte konsekvens av begrenset prosessorkraft på Raspberry Pi 4, og er en kjent avveining ved valg av lokal modell fremfor skybaserte alternativer, som beskrevet i avsnitt 4.6. En større Whisper-modell ville gitt høyere nøyaktighet, men på bekostning av enda lengre ventetid på samme hardware.

Retry-logikken beskrevet i 4.6 (UC-03) ble verifisert ved å si uklare eller stille kommandoer med vilje. Systemet ba brukeren gjenta seg inntil tre ganger før hendelsen ble logget som L1. Dette fungerte som forventet og sikrer at systemet ikke henger seg opp ved mislykkede gjenkjenninger.

Mekanisk stabilitet

En tidlig utfordring under testing var at vektmålingene var ustabile og vanskelige å reproducere. Årsaken var primært ujevn montering av vektcellene, der små skjevheter i oppsettet førte til skjevbelastning og måleavvik på opptil 200 gram. I tillegg viste seg at feil orientering av cellene forsterket problemet, ettersom vektcellene er retningssensitive og må monteres slik at deformasjonen skjer langs riktig akse for å gi korrekte avlesninger.

For å løse dette ble det designet og 3D-printet egne monteringsfester i Autodesk Fusion 360, som beskrevet i avsnitt 5.1. Festene sikrer at alle fire vektceller er montert med samme vinkel og orientering i hvert hjørne av monteringsplaten, noe som gir jevn vektfordeling over hele flaten. Vektplaten er i tillegg montert under søppelbeholderen for å redusere påvirkning fra vibrasjoner og bevegelse i omgivelsene.

En annen utfordring som ble observert var at avfall som kastes hardt ned i beholderen midlertidig påvirker vektavlesningen i det gjenstanden treffer. For å håndtere dette er systemet programmert til å vente noen sekunder etter at IR-sensoren trigges, slik at gjenstanden rekker å roe seg i beholderen før vektmålingen gjennomføres. Dette ga merkbart mer stabile og konsistente resultater.

Etter forbedringene i monteringen og justeringen av målingstidspunktet stabiliserte avlesningene seg til avvik på noen få gram for de fleste testgjenstander, noe som vurderes som akseptabelt for en prototype i denne fasen. Resultatene fra vektmålingen er presentert i detalj i avsnitt 6.4.

6.4 Resultater

I dette delkapittelet presenteres resultatene fra de fysiske testene som ble utført på prototypen. Resultatene er presentert etter komponent og funksjonsnivå, og dekker vektmåling, IR-sensor, kamera og bildegjenkjenning, samt talegjenkjenning.

Vektmåling

Resultatene fra vektmålingen er presentert i tabell 6.1. Målingene viser at systemet ofte lå noe over forventet vekt for de forskjellige gjenstandene som ble testet. Avviket varierte fra 5-10 gram for lettere gjenstander som poteten, og 20-30 gram for tyngre gjenstander som smarttelefonen og deodoranten.

For den letteste gjenstanden, den tomme brusboksen på 15 gram, var avviket størst relativt sett med opptil 10 gram over forventet vekt, noe som tilsvarer et avvik på 33-66%. Dette tyder på at systemet er mindre nøyaktig ved lavere belastninger, noe som er forventet gitt at vektcellene er dimensjonert for opptil 50 kg, som beskrevet i avsnitt 4.5. For tyngre gjenstander som fjølen på 650 gram var avviket på 20–30 gram, som tilsvarer omtrent 3–5% av faktisk vekt.

Resultatene er noe utenfor suksesskriteriet på ± 5 gram definert i avsnitt 3.1, og indikerer at støy, vibrasjoner og montering av vektcellene påvirker målingene, som beskrevet i avsnitt 5.1. Til tross for dette gir systemet stabile og gjentakbare målinger, noe som bekrefter at L1-løsningen fungerer som tenkt for automatisk vektregistrering.

Tabell 6.1: Testresultater for reel vektmåling

Gjenstander	Forventet	Resultat
Fjøl	650g	670 - 680g
Potet	70g	70 - 75g
Løk	185g	195 - 200g
Deodorant	120g	140 - 150g
Smarttelefon	210g	230 - 240g
Tom brusboks	15	20 - 25g

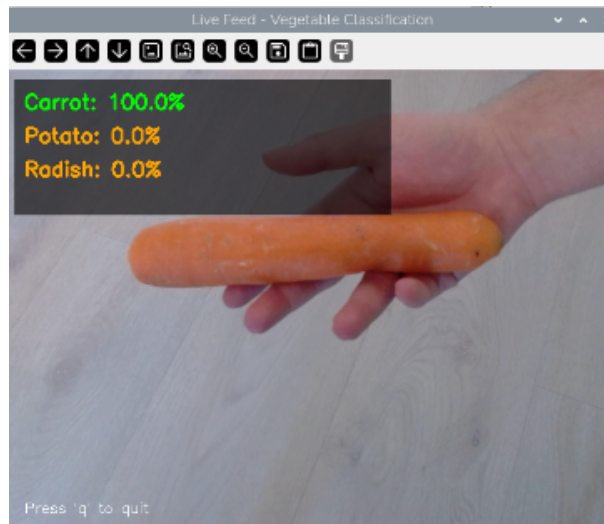
IR-sensoren fungerte stabilt gjennom testperioden. Etter kalibrering av følsomheten via potensiometeret ble det ikke registrert falske triggere under testforholdene, og sensoren responderte konsekvent på håndbevegelser innenfor det tiltenkte deteksjonsområdet.

Bildegjenkjenning

Resultatene fra bildegjenkjenningen viser tydelige forskjeller mellom de to modelltilnærmingene som ble testet, classification og detection. Testene ble gjennomført under kontrollerte forhold med kameraet montert cirka 35 cm over objektene, som beskrevet i 5.1.

Den første tilnærmingen vi testet var en classification basert modell som klassifiserer hele bildet som en gjenstand. Figur 6.3 viser en gulrot holdt i en hånd for testing av denne modellen. Modellen returnerer tre klassifiseringer av de tre gjenstandene med høyest konfidens. Disse klassifiseringene er: Carrot med 100%, Potato med 0% og Radish med 0% konfidens.

Selv om modellen korrekt identifiserte objektet i dette tilfellet, avdekket videre testing ved mindre ideelle forhold en betydelig svakhet: når objektet ble holdt i en hånd, ble hånden tolket som en del av objektet. Dette resulterte i inkonsistente klassifiseringer av objektet, noe som gjorde classification tilnærming lite egnet for bruksscenariet beskrevet i 4.3



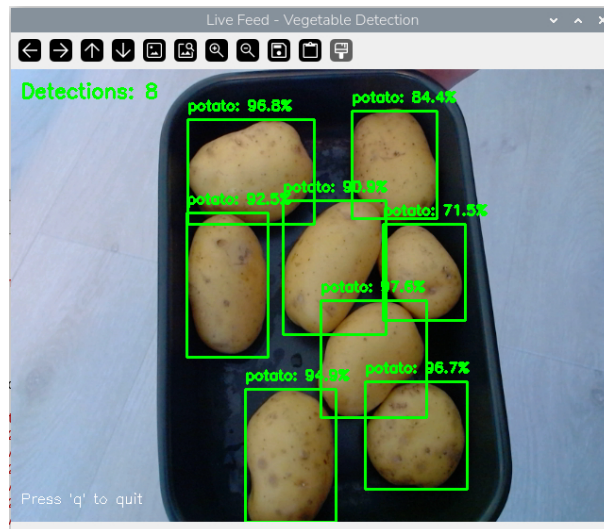
Figur 6.3: En test av bildegjenkjenning med en classification basert modell

Basert på utfordringene med classification modellen ble det besluttet å bytte til en detection basert versjon av modellen. Detection modellen identifiserer ikke bare objekttypen men også hvor hvert objekt er plassert i et bildet.

Figur 6.4 viser et testresultat der åtte poteter ble plassert i en beholder. Modellen detekterte alle åtte objekter med følgende konfidens:

- Potet 1: 97 % konfidens
- Potet 2: 94 % konfidens
- Potet 3: 93 % konfidens
- Potet 4: 92 % konfidens
- Potet 5: 87 % konfidens
- Potet 6: 84 % konfidens
- Potet 7: 79 % konfidens
- Potet 8: 72 % konfidens

Gjennomsnittlig konfidens for testen var 87 %, noe som ligger godt over suksesskriteriet definert i 3.1. Detection modellen viste seg å være langt mer robust enn classification modellen, særlig fordi den kunne skille mellom objektet og hånden eller beholderen som inneholder gjenstanden



Figur 6.4: En test av bildegjenkjenning med en detection basert modell

Tabell 6.2 oppsummerer de viktigste forskjellene mellom de to typene modeller vi har testet.

Tabell 6.2: Sammenligning av classification- og detection-modell

Egenskap	Classification	Detection
Klassifiserer hele bildet	Ja	Nei
Lokaliserer objekter	Nei	Ja
Håndterer hånd i bildet	Nei	Ja
Teller antall objekter	Nei	Ja
Gjennomsnittlig konfidens	100% (men upålitelig)	87%
Egnet for L3-løsningen	Nei	Ja

Resultatene bekrefter at detection modellen er bedre for L3 løsningen. Detection tilnærmingen gir ikke bare informasjon om hva som er i bildet men også hvor. Dette åpner for fremtidig funksjonalitet.

Talegjenkjenning

Talegjenkjenningen ble evaluert gjennom praktiske tester med ulike kommandoer under varierte forhold. Under stille omgivelser fungerte systemet godt. De fleste kommandoer ble korrekt transkribert av Whisper, og filtreringslogikken klarte å trekke ut objektnavnet fra naturlige setninger som «jeg kaster poteter» og «kastet litt pasta».

Ved testing med bakgrunnsstøy ble nøyaktigheten merkbart redusert. Et gjennomgående eksempel var at ordet «kaster» ble transkribert som «koster», noe som ikke traff trigger word-listen og førte til at hendelsen ble logget som en L1-registrering uten kategori. Dette ble håndtert ved å utvide listen med dialekt- og feiloversettingsvarianter, noe som forbedret gjenkjenningen.

Latenstiden var en tydelig faktor under testing. Whisper base-modellen brukte typisk mellom fem og ti sekunder på å transkribere et opptak, noe som oppleves som en merkbar pause for brukeren. Dette er en direkte konsekvens av begrenset prosessorkraft på Raspberry Pi 4 og er den største praktiske svakheten ved L2-løsningen slik den er implementert.

Gjenforsøkslogikken ble verifisert ved å gi uklare kommandoer med vilje. Systemet ba brukeren gjenta seg inntil tre ganger før hendelsen ble logget som en L1-registrering, i tråd med UC-03.

API-integrasjon

API-integrasjonen mot TotalCtrl sitt stagingmiljø ble verifisert gjennom gjentatte testoverføringer under stabile nettverksforhold. Alle gjennomførte kall returnerte HTTP 201 Created, noe som bekrefter at dataformatet er korrekt og at kommunikasjonen fungerer som forventet. Feilhåndteringsmekanismen ble verifisert ved å simulere nettverksbrudd under kjøring. Hendelser ble korrekt mellomagret lokalt, med opprinnelige tidsstempler bevart.

Et eksempel på et vellykket API-svar er vist i figur [4.4](#) i avsnitt [4.6](#).

Kapittel 7

Diskusjon

Dette kapittelet diskuterer prosjektets resultater og erfaringer opp mot de målene og forskningsspørsmålene som ble definert innledningsvis i kapittel 1. Først vurderes det i hvilken grad prosjektets mål er oppnådd, deretter reflekteres det over arbeidsmetodikk og prosjektgjennomføring. Videre gjennomgås tekniske utfordringer som oppsto underveis, før løsningen diskuteres fra et brukerperspektiv og sammenlignes med eksisterende kommersielle løsninger. Kapittelet avsluttes med en vurdering av prosjektets begrensninger og en samlet vurdering.

7.1 Oppnåelse av prosjektets mål

Formålet med prosjektet var å utvikle en fungerende IoT-prototype som automatiserer registrering av matsvinn uten behov for manuell inntasting, som beskrevet i avsnitt 1.5. I det følgende vurderes oppnåelsen av prosjektets mål opp mot de fire forskningsspørsmålene.

FS1 – Klassifiseringsteknologier. Litteraturgjennomgangen i kapittel 2 og de praktiske erfaringene fra implementasjonen viser at Whisper og YOLOv8n er godt egnede teknologier for henholdsvis tale- og bildegjenkjenning på en ressursbegrenset enhet som Raspberry Pi 4. Begge modellene kan kjøres lokalt uten nettverkstilkobling, noe som er avgjørende for systemets robusthet i et kjøkkenmiljø. Avveiningen er redusert nøyaktighet sammenlignet med skybaserte alternativer, særlig under krevende forhold som bakgrunnsstøy og dårlig belysning. Vektmåling via HX711 og fire vektceller fungerer som et pålitelig grunnlag på tvers av alle kompleksitetsnivåer.

FS2 – Nøyaktighet og pålitelighet. Vektmålingene viser stabile og gjentakbare resultater, men med avvik som ligger noe utenfor suksesskriteriet på ± 5 gram for lettere gjenstander, som presentert i tabell 6.1. API-integrasjonen mot TotalCtrl sitt stagingmiljø fungerer som forventet, og systemet håndterer nettverksbrudd ved å mellomlagre hendelser lokalt og synkronisere når forbindelsen gjenopprettes. I tillegg registreres tømmehendelser automatisk ved deteksjon av stor negativ vektendring, noe som sikrer at akkumulert avfallsdata bevares også ved tømming av beholderen. Samlet sett demonstrerer systemet tilstrekkelig pålitelighet for en prototype, men krever videre optimalisering før det er egnet for produksjonsdrift.

FS3 – Brukervennlighet og interaksjonsdesign. Brukerintervjuene ved de to sykehjemmene, beskrevet i avsnitt 3.5, avdekket at løsningen må tilpasses ulike miljøer. L1-løsningen, som ikke krever noen form for brukerinteraksjon utover å kaste avfallet, ble vurdert som særlig godt egnet i hektiske og støyfylte omgivelser. Talegjenkjenning ble ansett som gjennomførbart på hovedkjøkken, men lite egnet på avdelingskjøkken nær beboerarealer. Den modulære arkitekturen gjør det mulig å velge interaksjonsform etter lokale behov, noe som direkte adresserer de varierte kravene som ble identifisert.

FS4 – Skalerbar arkitektur. Den modulære L1–L4-arkitekturen gjør det mulig å rulle ut løsningen gradvis, fra enkel vektmåling til hybrid klassifisering med både bilde og tale. Hvert nivå kan tas i bruk uavhengig av de andre, og systemet kan tilpasses ulike brukerbehov og budsjetter uten å redesigne kjernen. Kodestrukturen med separate Python-moduler og subprocess-basert isolasjon legger til rette for fremtidig utvidelse, som beskrevet i avsnitt 8.1. L4 er ikke implementert i dette prosjektet, men arkitekturen er utformet slik at dette kan realiseres uten store endringer.

Vurdering av suksesskriterier

De tre suksesskriteriene definert i avsnitt 3.1 kan oppsummeres som følger. Vektmålingsnøyaktigheten på ± 5 gram ble ikke fullt ut oppnådd. Avvikene lå på 5–30 gram avhengig av gjenstandens vekt, med størst relativ feil for lette gjenstander. Dette skyldes primært at vektcellene er dimensjonert for opptil 50 kg, noe som gir lavere presisjon ved lave belastninger. Kriteriet var trolig for ambisiøst gitt valgt hardware, men målingene er stabile og gjentakbare, noe som er tilstrekkelig for prototypens formål.

API-påliteligheten på minst 95 % vellykkede overføringer ble oppnådd under stabile nettverksforhold, der alle gjennomførte testoverføringer returnerte HTTP 201 Created. Feilhåndteringsmekanismen fungerte som forventet ved simulert nettverksbrudd.

Kategoriseringsnøyaktigheten på minst 80 % ble oppnådd for bildegjenkjenning under gode lysforhold, der detection-modellen oppnådde en gjennomsnittlig konfidens på 87 % i testene. For talegjenkjenning var nøyaktigheten tilfredsstillende under stille forhold, men falt merkbart ved bakgrunnsstøy, noe som gjør det vanskeligere å bekrefte kriteriet entydig for L2-løsningen under reelle kjøkkenforhold.

7.2 Arbeidsmetodikk og prosjektgjennomføring

Dette avsnittet reflekterer over hvordan prosjektet ble gjennomført metodisk, fra overordnet planlegging til daglig arbeidsflyt. Målet er å vurdere i hvilken grad valgte metoder og verktøy støttet opp under prosjektets mål, og hva som eventuelt kunne vært gjort annerledes.

Milepælsplanen som styringsverktøy

Milepælsplanen fungerte som en god overordnet struktur i starten av prosjektet og ga et felles utgangspunkt for forventninger mellom gruppen og oppdragsgiver. De første fasene med kartlegging og hardware-oppsett gikk som planlagt, og integrasjon mot API og grunnleggende funksjonalitet ble også levert innenfor de planlagte rammene.

Videre måtte planen justeres basert på ny innsikt. Brukerintervjuene ved sykehjemmene hadde spesielt stor innvirkning på retningen prosjektet tok videre. Da gruppen opprinnelig hadde sett for seg én fast løsning, viste det seg at behovene var ganske forskjellige mellom ulike kjøkkenmiljøer. Fokuset ble derfor flyttet over til en mer modulær tilnærming, noe som påvirket både prioriteringer og hva som ble levert i de senere fasene.

Planen hadde likevel noen svakheter. Den ga god oversikt på et generelt nivå, men var mindre nyttig i den daglige prioriteringen av oppgaver. I perioder med mye teknisk arbeid, særlig rundt maskinvareintegrasjon og kalibrering, viste det seg at tidsestimatene hadde vært litt optimistiske. Dette er noe som er vanskelig å planlegge for på forhånd, og gjenspeiler en kjent utfordring i prosjekter der mye av kompleksiteten først avdekkes under praktisk implementasjon.

Alt i alt fungerte milepælsplanen best som et felles styringsverktøy mellom gruppen og oppdragsgiver, ikke som en detaljert arbeidsplan.

Hybrid arbeidsmetodikk i praksis

Den hybride arbeidsmetodikken som kombinerer elementer fra Scrum og Kanban, beskrevet i avsnitt 3.2, viste seg å passe godt for prosjektets karakter. Ukentlige møter med oppdragsgiver skapte en naturlig sprintrytme, og GitHub Issues og Projects ga god oversikt over oppgaver og fremdrift uten å kreve en rigid sprintstruktur.

En styrke ved denne tilnærmingen var fleksibiliteten til å endre prioriteringer underveis. Da brukerintervjuene avdekket at behovene varierte mellom ulike kjøkkenmiljøer, kunne gruppen justere retningen uten at fremdriften stoppet opp. En mer rigid planleggingsmodell ville gjort det vanskeligere å ta de valgene som brukerinnsikten pekte på. Dette er i tråd med prinsippene bak agile arbeidsmetoder, der kontinuerlig tilbakemelding og tilpasning prioriteres fremfor å følge en fast plan [36].

En svakhet var at den løse sprintstrukturen tidvis gjorde det utfordrende å estimere fremdrift presist, særlig i faser med mye praktisk og uforutsigbart arbeid. Kortere og mer definerte iterasjoner med tydeligere akseptansekriterier kunne ha gitt bedre oversikt i disse periodene.

Design science som forskningsramme

Prosjektet følger et design science-orientert forskningsdesign, der kunnskap bygges opp gjennom å utvikle og teste en konkret løsning i praksis [19]. Dette innebar at forskningsspørsmålene ikke ble besvart gjennom tradisjonell hypotesetesting, men gjennom en iterativ prosess der prototypen ble utviklet, testet og forbedret i tett dialog med oppdragsgiver og brukere.

Denne tilnærmingen ga noen klare fordeler. Brukerintervjuene tidlig i prosjektet ga innsikt som direkte påvirket arkitekturvalgene, og den løpende dialogen med TotalCtrl sikret at løsningen forble relevant for et reelt behov. Samtidig stiller design science-orientert forskning krav til at løsningen evalueres systematisk opp mot definerte kriterier [19], noe som ble ivarettatt gjennom suksesskriteriene definert i avsnitt 3.1 og testingen beskrevet i kapittel 6.

En utfordring ved denne tilnærmingen er at evalueringen ble gjennomført under kontrollerte forhold og ikke i et reelt kjøkkenmiljø. Dette begrenser i hvilken grad resultatene kan generaliseres til faktisk drift, og er en naturlig begrensning ved prototypeutvikling innenfor en begrenset tidsramme.

7.3 Tekniske utfordringer

Gjennom utviklings- og testfasen møtte gruppen på flere tekniske utfordringer. Mange av disse er typiske for IoT-systemer med ressursbegrenset hardware, og erfaringene ga innsikt i hva som skiller en fungerende prototype fra en produksjonsklar løsning.

Vektmåling og hardware-presisjon

Vektmålingene var påvirket av støy, vibrasjoner og montering gjennom hele testfasen. Selv etter forbedringer i monteringen ligger avvikene noe utenfor suksesskriteriet på ± 5 gram, særlig for lette gjenstander. Dette illustrerer en grunnleggende utfordring ved bruk av rimelig og tilgjengelig hardware: vektcellene er dimensjonert for opptil 50 kg, noe som gir lavere relativ presisjon ved lave belastninger. Industriell hardware ville gitt bedre resultater, men er verken tilgjengelig eller i tråd med prosjektets formål som MVP (Minimum Viable Product).

Talegjenkjenning og ressursbegrensninger

Talegjenkjenning i et kjøkkenmiljø er krevende av to grunner. For det første påvirker bakgrunnsstøy og dialektvariasjon transkripsjonsnøyaktigheten merkbart. For det andre er Whisper base relativt beregningskrevende for en Raspberry Pi 4, noe som gir en latenstid på typisk fem til ti sekunder per opptak. Kombinasjonen av støy og begrenset prosessorkraft gjør at L2-løsningen er mer sårbar for omgivelsene enn L1, og setter tydelige grenser for hvor godt talegjenkjenning kan fungere på ressursbegrenset hardware uten GPU-akselerasjon.

Bildegjenkjenning og lysforhold

Bildegjenkjenningen var sensitiv for lysforholdene under testing. Under gode lysforhold ga YOLO-modellen konsistente deteksjoner med høy konfidens, mens konfidensen falt tydelig ved dårligere belysning. Dette avdekker at L3-løsningen i sin nåværende form er avhengig av stabile lysforhold for å fungere pålitelig, og er noe som ikke alltid kan garanteres i et reelt kjøkkenmiljø. I tillegg medførte begrenset prosessorkraft på Raspberry Pi 4 merkbar behandlingstid for bildeanalysen, noe som påvirker den totale responstiden for L3-løsningen.

Avhengighetskonflikter i programvaremiljøet

En mer uventet utfordring var at bildegjenkjenningsbibliotekene krevde et separat virtuelt miljø som ikke lot seg kombinere med resten av systemet. Dette tvang frem en subprocess-basert arkitektur som løser problemet, men som introduserer ytelseskostnad og gjør kommunikasjonen mellom prosessene mer kompleks. Det er en påminnelse om at programvareavhengigheter i Python-prosjekter raskt kan bli en kilde til friksjon, særlig på en ressursbegrenset enhet der man ikke alltid kan installere de nyeste bibliotekversjonene.

Utvikling av mellomlagringsstrategien

Under implementasjonen ble den opprinnelige designbeslutningen for lokal mellomlagring revidert. Det første designet la til grunn at hendelser kun skulle lagres lokalt dersom API-kallet feilet – en tilnærming som er vanlig i litteraturen beskrevet som en fallback-mekanisme, der lokal lagring aktiveres som et unntak ved nettverksfeil [18]. I den endelige implementasjonen ble dette endret slik at alle hendelser alltid lagres lokalt på enheten først, uavhengig av nettverksstatus, og at API-kallet deretter skjer parallelt.

Denne endringen ble gjort av to praktiske grunner. For det første forenkler det feilsøking betydelig under utvikling, siden lokale JSON-filer gir et pålitelig og lett tilgjengelig spor av alle registrerte hendelser uavhengig av nettverksforhold. For det andre gjør det systemet mer robust i et kjøkkenmiljø: dersom nettverksforbindelsen er utilgjengelig, fortsetter systemet å registrere hendelser lokalt uten at dette krever noen aktiv feilhåndtering i selve registreringslogikken.

Den implementerte løsningen avviker likevel fra idealet beskrevet i avsnitt 4.6, der en aktiv synkroniseringskø med eksponentiell backoff ville gitt sterkere garantier for datalevering. En slik mekanisme ville automatisk forsøkt å sende lokalt lagrede hendelser på nytt i riktig rekkefølge og med opprinnelige tidsstempler så snart forbindelsen gjenopprettes [2]. I prototypen er mellomlagringen implementert som et sirkulært buffer med kapasitet for fem hendelser, noe som er tilstrekkelig for kortvarige nettverksbrudd, men som ikke gir de samme garantiene som en fullstendig retry-kø. Dette er en bevisst avveining i en prototype-kontekst, og den valgte løsningen gir bedre grunnlag for debugging og offline-drift enn den opprinnelige fallback-tilnærmingen.

Nettverksavhengighet

Som IoT-system er prototypen avhengig av nettverksforbindelse for å overføre data til TotalCtrl sitt API. Den lokale mellomlagringsmekanismen reduserer risikoen for tap av data, men løser ikke alle scenarier. Det sirkulære bufferet har plass til fem hendelser, og ved lengre nettverksbrudd med høy aktivitet risikerer man at eldre hendelser overskrives. Dette er en kjent avveining i prototype-kontekst, men vil kreve en mer robust løsning før systemet kan driftes i et reelt kjøkkenmiljø over tid.

7.4 Brukerperspektiv

Brukerundersøkelsene ved de to sykehjemmene, beskrevet i avsnitt 3.5, ga viktig og verdifull innsikt som har hatt direkte innvirkning på hvordan prototypen ble utviklet. Et gjennomgående funn var at manuell registrering oppleves som tidkrevende og upraktisk, og at ansatte i travle perioder prioriterer kjerneoppgavene sine fremfor å veie og registrere matsvinn. Dette går ikke nødvendigvis på ren vilje, men heller at systemet ikke passer inn i en travel arbeidsdag. En automatisert løsning kan redusere denne belastningen betraktelig.

L1-løsningen, som kun bruker vektmåling uten behov for tale eller bilde, passer særlig godt i slike miljøer. Brukeren trenger ikke gjøre noe annet enn å kaste avfallet, og systemet tar seg av resten. Dette ble også godt mottatt av de ansatte, som viste tydelig at jo mindre de trengte å gjøre, jo mer sannsynlig var det at registreringen faktisk ble gjennomført.

Taleløsningen ble vurdert ulikt avhengig av hvor i institusjonen den skulle brukes. På hovedkjøkkenet, der miljøet er mer kontrollert, ble tale sett på som praktisk og gjennomførbart. På avdelingskjøkkenene var bildet et annet. Her er kjøkkenene ofte plassert i tilknytning til fellesarealer der beboerne oppholder seg, og bakgrunnsstøy fra TV og samtaler gjør talegjenkjenning mindre pålitelig. Informantene fra et av sykehjemmene påpekte i tillegg at uventede lyder kan skape uro hos beboere på psykiatriske og skjermede avdelinger, noe som gjør tale lite egnet i disse miljøene.

Dette støtter valget om å tilby flere måter å registrere matsvinn på, slik det er lagt opp til i den modulære arkitekturen. Ikke alle miljøer er like, og en løsning som fungerer godt ett sted kan være upraktisk et annet sted. Fleksibilitet er derfor viktig for at systemet faktisk skal tas i bruk i praksis, og den modulære arkitekturen gjør det mulig å tilpasse løsningen etter behovene på hvert enkelt sted.

7.5 Posisjonering i markedet

Winnow og Leanpath, beskrevet i avsnitt 2.5, er de mest kjente kommersielle løsningene for automatisk matsvinnregistrering i profesjonelle kjøkken. Begge systemene kombinerer vektorer, kameraer og skybasert teknologi for å registrere og kategorisere matavfall, og er teknisk mer modne enn prototypen som er utviklet i dette prosjektet. De er primært rettet mot større aktører som hoteller, restauranter og cateringbransjen, og krever betydelige investeringer i både maskinvare og infrastruktur.

Vår løsning skiller seg fra disse på flere punkter. Der Winnow og Leanpath er ferdige produkter med avanserte AI-modeller og dedikert hardware, er prototypen bygget med rimelig og tilgjengelig utstyr med fokus på fleksibilitet og lav inngangsbarriere. Dette gjør at løsningen naturlig nok har lavere nøyaktighet og modenhet, men til gjengjeld er den lettere å tilpasse og bygge videre på.

Den viktigste forskjellen ligger i den modulære arkitekturen beskrevet i avsnitt 4.4.1. Der eksisterende løsninger for det meste tilbyr ett fast funksjonsnivå, kan vår løsning konfigureres fra enkel automatisk vektmåling på L1-nivå til en hybrid løsning med både bilde og talegjenkjenning på L4-nivå. Dette gjør den mer tilgjengelig for mindre organisasjoner som sykehjem og kantiner, der behovet eksisterer men ressursene er begrenset, noe som også ble bekreftet gjennom brukerintervjuene i avsnitt 3.5.

Prototypen er derfor ikke ment som en direkte konkurrent til Winnow eller Leanpath, men heller som et alternativ for de som ikke dekkes av de store kommersielle løsningene, som sykehjem og mindre kantiner. Samtidig viser sammenlikningen at det fortsatt er et stykke igjen før løsningen kan måle seg med de kommersielle alternativene på nøyaktighet og driftssikkerhet.

Det er verdt å merke seg at lavere kompleksitet sammenlignet med Winnow og Leanpath ikke nødvendigvis er en svakhet. For målgruppen løsningen er rettet mot er enkelhet i oppsett og vedlikehold like viktig som funksjonalitet. En løsning som er enkel å forstå, sette opp og bygge videre på, har i mange tilfeller større sjanse for faktisk å bli tatt i bruk enn en mer avansert løsning med høyere inngangsbarriere.

7.6 Begrensninger

Prosjektet har hatt en begrenset tidsramme, noe som har påvirket hvor langt det var mulig å komme innenfor prosjektperioden. L4-løsningen ble ikke implementert, og enkelte av de andre nivåene har rom for videre forbedring. Denne rapporten dokumenterer en fungerende prototype som demonstrerer konseptet, men ikke en produksjonsklar løsning.

Maskinvaren som er benyttet er rimelig og tilgjengelig, ikke industriell. Dette setter naturlige grenser for nøyaktighet og driftssikkerhet. Vektcellene er dimensjonert for opptil 50 kg, noe som gir lavere relativ presisjon ved lave belastninger. Som vist i avsnitt 6.3 viste de første testene store avvik, og selv etter forbedringer i montering og kalibrering er målingene ikke på nivå med det man kan forvente av en produksjonsklar løsning. Det samme gjelder bildegjenkjenning, der manglende GPU-akselerasjon på Raspberry Pi 4 gir merkbar behandlingstid og begrenser hvor raskt systemet kan respondere.

AI-modellene for tale- og bildegjenkjenning er trent på åpne datasett som ikke nødvendigvis representerer norske kjøkkenmiljøer. Matavfall kan variere betydelig i utseende avhengig av tilberedning og tilstand, og denne variasjonen gjenspeiles ikke fullt ut i treningsdataene. Talegjenkjenningen er i tillegg sensitiv for bakgrunnsstøy og dialektvariasjon, noe som kan påvirke påliteligheten i et reelt kjøkkenmiljø. Testingen har foregått under kontrollerte forhold uten det støynivået, den varierende belysningen og det høye tempoet som preger en travel kjøkkendag, som beskrevet i avsnitt 6.1.

Et område som ikke ble ferdigstilt innen prosjektets tidsramme er visning av registrerte hendelser i et brukergrensesnitt. Gjennom prosjektperioden arbeidet TotalCtrl med å koble dataene mot sin eksisterende plattform, men denne integrasjonen lot seg ikke fullføre før innleveringsfristen. Dette påvirker ikke prototypens evne til å registrere og overføre hendelsesdata korrekt, da API-integrasjonen er verifisert og fungerer som forventet. Gruppen har planlagt et oppfølgingsmøte med TotalCtrl etter prosjektleveransen for å ferdigstille denne delen før den endelige presentasjonen av prototypen.

Samlet sett gir resultatene gode indikasjoner på at konseptet fungerer, men er ikke tilstrekkelige til å si noe sikkert om hvordan systemet vil oppføre seg over tid i reell drift. Flere faktorer må forbedres og testes videre før løsningen er klar for praktisk bruk, noe som er nærmere beskrevet i avsnitt 8.1.

7.7 Samlet vurdering

Prototypen viser at automatisert matsvinnregistrering er teknisk gjennomførbart, og at den modulære arkitekturen er en velegnet tilnærming for å møte varierte brukerbehov. Løsningen fungerer godt på grunnleggende nivå, mens de mer avanserte nivåene avdekker utfordringer som må adresseres før praktisk drift. Funnene gir et godt grunnlag for konklusjonen i kapittel 8.

Kapittel 8

Konklusjon

Dette kapittelet oppsummerer prosjektets resultater og svarer på problemstillingen som ble definert innledningsvis. Først beskrives anbefalte retninger for videre arbeid og utvikling av løsningen. Avslutningsvis gis en samlet oppsummering av hva som er oppnådd og hvilke funn som er gjort gjennom prosjektperioden.

8.1 Videre arbeid

Prototypen dekker L1, L2 og L3, og demonstrerer at automatisk matsvinnregistrering via vektmåling, talegjenkjenning og bildegjenkjenning er teknisk gjennomførbart. Det er likevel flere naturlige retninger for videreutvikling, både på hardware-, software- og sikkerhetssiden.

Av de områdene som er beskrevet under, vurderes følgende som de tre mest naturlige neste stegene for TotalCtrl: implementasjon av L4 for høyere klassifiseringspålidelighet, trening av modeller på egne datasett fra reelle kjøkkenmiljøer, og tilkobling av en relestyrt lyskilde for å gjøre bildegjenkjenningen robust under varierende lysforhold. Disse tre tiltakene vil alene gi betydelig forbedret ytelse uten å kreve store endringer i kjernesystemet.

L4 – Hybrid klassifisering

L4 er ikke implementert i dette prosjektet, men den modulære kodestrukturen legger til rette for at dette kan realiseres uten store arkitecturendringer. L4 kombinerer bilde- og talegjenkjenning, der systemet først forsøker å klassifisere via kamera, og kun aktiverer mikrofonen dersom bildemodellen returnerer lav konfidens. Dette er nærmere beskrevet i avsnitt [4.4.1](#).

Modelltrening på egne datasett

De nåværende modellene for tale- og bildegjenkjenning er trent på åpne datasett som ikke nødvendigvis representerer norske kjøkkenmiljøer. Ved å trene modellene på egne datasett – med bilder og lydopptak fra reelle brukssituasjoner – vil gjenkjenningsnøyaktigheten kunne forbedres betydelig. Dette gjelder

særlig for bildegjenkjenning, der matavfall kan variere mye i utseende avhengig av tilberedning og tilstand, som beskrevet i avsnitt 1.6.

Lyskilde og bildekvalitet

For å sikre konsistent bildekvalitet uavhengig av omgivelsesbelysning kan en dedikert lyskilde tilkobles via relèmodul på GPIO. Lyskilden aktiveres automatisk når kameraet trigges, og deaktiveres etter opptak. Dette er særlig nyttig i mørkere omgivelser eller på tidspunkter med lav naturlig belysning. Slik systemet fungerer i dag er bildekvaliteten tilfredsstillende under gode lysforhold, men en relestyrt lyskilde vil gjøre løsningen mer robust i reelle kjøkkenmiljøer.

Høytalerfunksjon

En høytaler kan legges til for å gi brukeren tydelig lydfeedback ved registrerte hendelser, for eksempel ved å lese opp bekreftelsen «Gulrøtter, 340 gram registrert» etter vellykket klassifisering. Dette reduserer behovet for at brukeren må se på en skjerm, og gjør systemet mer egnet for hektiske kjøkkenmiljøer der oppmerksomheten er rettet andre steder. Høytalerfunksjonen kan også benyttes til andre varsler, som feil ved nettverkstilkobling eller behov for recalibrering.

Fysisk design

Det fysiske kabinettet som er utviklet i dette prosjektet er ment som en konseptdemonstrasjon, ikke en ferdig løsning. Den endelige formfaktoren, materialvalg og estetisk utforming er TotalCtrl sitt ansvar å videreutvikle i tråd med sine kunders behov og ønsker. Den modulære arkitekturen sikrer at kjernesystemet kan integreres i ulike fysiske innpakninger uten endringer i programvaren.

API-sikkerhet

I den nåværende implementasjonen lagres API-nøkkelen i en lokal konfigurasjonsfil på enheten. For en produksjonsklar løsning bør dette forbedres på flere områder. API-nøkkelen bør krypteres eller håndteres via et dedikert secrets management system fremfor klartekst i en fil. All kommunikasjon skjer allerede over HTTPS, men sertifikatvalidering bør verifiseres eksplisitt i koden. Ved utrulling av flere enheter bør hver enhet ha en unik identifikator og nøkkel, slik at kompromittering av én enhet ikke påvirker resten av installasjonen. Det kan også vurderes å innføre token-basert autentisering med begrenset levetid fremfor statiske API-nøkler, noe som reduserer risikoen ved eventuell lekkasje.

Overgang til dedikert hardware

Raspberry Pi 4 ble valgt for dette prosjektet på grunn av tilgjengelighet, fleksibilitet og egnethet for rask prototyping, som beskrevet i avsnitt 4.5. For en produksjonsklar løsning vil TotalCtrl trolig ha behov for mer kompakt og robust hardware tilpasset et industrielt kjøkkenmiljø.

Ettersom systemet kjører lokale maskinlæringsmodeller for tale- og bildegjenkjenning, stilles det krav til tilstrekkelig prosessorkraft også på den endelige hardware-plattformen. Aktuelle alternativer inkluderer NVIDIA Jetson Nano eller Jetson Orin NX, som er kompakte edge-enheter med dedikert GPU-akselerasjon spesielt utviklet for inferens av maskinlæringsmodeller [26]. Dette vil gi betydelig raskere

responstid enn Raspberry Pi 4, særlig for bildegjenkjenning. Et annet alternativ er Google Coral Dev Board, som benytter en dedikert Edge TPU for akselerert inferens med lavt strømforbruk [16].

For L1-løsningen alene, der kun vektmåling og API-integrasjon er nødvendig, kan langt enklere og rimeligere hardware benyttes – for eksempel en ESP32-basert mikrokontroller med nettverksstøtte [11]. Dette kan være aktuelt for installasjoner der tale- og bildegjenkjenning ikke er ønsket.

Systemets modulære arkitektur og bruk av Python gjør at kjernelogikken i stor grad kan porteres til andre plattformer uten å måtte redesigne hele systemet, forutsatt at målplattformen støtter nødvendige biblioteker eller tilsvarende alternativer.

Andre muligheter for videreutvikling

Utover de allerede beskrevne områdene finnes det flere retninger systemet kan utvikles videre i. Listen under representerer ideer og muligheter som ikke er implementert i dette prosjektet, men som kan gi verdi i en fremtidig produksjonsklar løsning:

- **Overflyllingsvarsling:** Sensor som detekterer når søppelbøtten er full og varsler personalet, noe som også ble etterspurt i brukerintervjuene i avsnitt 3.5.
- **Støtte for flere enheter:** Koordinering av flere bøtter eller enheter på samme lokasjon, med sentral oversikt over all aktivitet.
- **Innkjøpsintegrasjon:** Kobling mot innkjøpssystemer slik at matsvinndata automatisk kan påvirke fremtidige bestillinger og bidra til bedre ressursstyring.
- **ESG-rapportering:** Eksport av matsvinndata til bærekrafts- og ESG-rapportering, i tråd med TotalCtrl sin eksisterende plattform for bærekraftsrapportering.
- **Lokal touchskjerm:** En liten skjerm montert på kabinettet for lokal visning av dagens matsvinn, systemstatus og eventuelle varsler, uten behov for ekstern enhet.
- **Administratorgrensesnitt:** Dedikert grensesnitt for enkel konfigurasjon av funksjonsnivå (L1–L4), kalibrering og enhetsinnstillinger uten behov for direkte kodeendringer.
- **Flerspråklig støtte:** Utvidelse av talegjenkjenningen til å støtte flere språk, noe som kan være relevant i kjøkkenmiljøer med ansatte fra ulike land.
- **Anonymisert brukergjenkjenning:** Mulighet for å knytte registreringer til spesifikke skift eller ansatte, for eksempel via RFID-kort eller PIN-kode, slik at matsvinndata kan analyseres per bruker eller skift.
- **Prediktiv analyse:** Bruk av historiske matsvinndata til å forutsi fremtidig svinn basert på mønstre, for eksempel knyttet til ukedag, måltid eller sesong.
- **Energioptimalisering:** Innføring av hvilemodus når systemet er inaktivt over lengre perioder, for å redusere strømforbruk i installasjoner der enheten er på kontinuerlig.

- **Automatisk oppdatering:** Støtte for over-the-air (OTA) programvareoppdateringer, slik at nye versjoner kan rulles ut til alle enheter uten fysisk tilgang.

8.2 Oppsummering

Prosjektets overordnede problemstilling var hvordan IoT-teknologi kan brukes til å automatisere registrering av matsvinn, slik at data fanges uten behov for manuell inntasting. Gjennom utvikling og testing av en fungerende prototype har gruppen vist at dette er teknisk gjennomførbart med rimelig og tilgjengelig hardware, og at en modulær arkitektur gir den fleksibiliteten som kreves for å møte ulike brukerbehov.

Alle tre kompleksitetsnivåene er implementert og fungerer etter hensikten. L1-løsningen med automatisk vektmåling og API-integrasjon fungerer stabilt og danner et solid grunnlag for hele systemet. L2 og L3 utvider dette med henholdsvis tale- og bildegjenkjenning, og demonstrerer at klassifisering av matavfall kan gjøres lokalt på en Raspberry Pi 4 uten avhengighet av skybaserte tjenester. Resultatet er en fungerende MVP som dekker kravene beskrevet i prosjektbeskrivelsen og som allerede i sin nåværende form kan benyttes til å samle inn verdifulle matsvinndata i reelle kjøkkenmiljøer.

Brukerinnsikten fra sykehjembesøkene var avgjørende for retningen prosjektet tok. Den bekreftet at behovene varierer mellom ulike kjøkkenmiljøer, og at en løsning som ikke tilpasses disse variasjonene vil ha begrenset praktisk verdi. Den modulære arkitekturen er derfor ikke bare en teknisk beslutning, men en direkte respons på reelle brukerbehov.

Prosjektet har levert det det satte seg fore: et dokumentert, testbart og utvidbart system som automatiserer en arbeidsoppgave som i dag løses manuelt og upålitelig. Det gjenstår arbeid før løsningen er produksjonsklar, men kjernen er på plass og gir TotalCtrl et konkret grunnlag å bygge videre på.

Referanseliste

- [1] Alpha Cephei. *Vosk Official Documentation*. Hentet januar 2026. 2023. URL: <https://alphacephei.com/vosk/>.
- [2] Amazon Web Services. *Exponential Backoff and Jitter*. Hentet mars 2026. 2015. URL: <https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/>.
- [3] Xavier Anguera mfl. «Speaker diarization: A review of recent research». I: *IEEE Transactions on Audio, Speech, and Language Processing* 20.2 (2012), s. 356–370.
- [4] Luigi Atzori, Antonio Iera og Giacomo Morabito. «The Internet of Things: A survey». I: *Computer Networks* 54.15 (2010), s. 2787–2805.
- [5] AVIA Semiconductor. *HX711: 24-bit analog-to-digital converter for weigh scales*. Hentet februar 2026. 2020. URL: https://cdn.sparkfun.com/datasheets/Sensors/ForceFlex/hx711_english.pdf.
- [6] Alexei Baevski mfl. «wav2vec 2.0: A framework for self-supervised learning of speech representations». I: *Advances in Neural Information Processing Systems (NeurIPS)*. Bd. 33. 2020, s. 12449–12460. URL: <https://arxiv.org/abs/2006.11477>.
- [7] Gary Bradski. «The OpenCV library». I: *Dr. Dobb's Journal of Software Tools* 25.11 (2000), s. 120–125.
- [8] Deepgram. *Nova-3 Model Documentation*. Hentet januar 2026. 2023. URL: <https://developers.deepgram.com/docs>.
- [9] Deepgram. *Speech Recognition Model Overview*. Hentet januar 2026. 2023. URL: <https://deepgram.com/product/speech-to-text>.
- [10] Alexey Dosovitskiy mfl. «An image is worth 16x16 words: Transformers for image recognition at scale». I: *International Conference on Learning Representations (ICLR)*. 2021.
- [11] Espressif Systems. *ESP32 Series Datasheet*. Hentet februar 2026. 2024. URL: <https://www.espressif.com/en/products/socs/esp32>.
- [12] Roy Thomas Fielding. «Architectural styles and the design of network-based software architectures». Ph.d.-avh. University of California, Irvine, 2000.
- [13] Jacob Fraden. *Handbook of Modern Sensors: Physics, Designs, and Applications*. 5. utg. Springer, 2016.
- [14] Ross Girshick mfl. «Rich feature hierarchies for accurate object detection and semantic segmentation». I: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2014, s. 580–587.

- [15] Google Cloud. *Speech-to-Text documentation*. Hentet januar 2026. 2023. URL: <https://cloud.google.com/speech-to-text>.
- [16] Google LLC. *Coral Dev Board*. Hentet februar 2026. 2024. URL: <https://coral.ai/products/dev-board>.
- [17] Google Research. *Chirp: Universal Speech Model*. Hentet januar 2026. 2023. URL: <https://cloud.google.com/blog/products/ai-machine-learning/introducing-chirp>.
- [18] Jayavardhana Gubbi mfl. «Internet of Things (IoT): A vision, architectural elements, and future directions». I: *Future Generation Computer Systems* 29.7 (2013), s. 1645–1660.
- [19] Alan R. Hevner mfl. «Design Science in Information Systems Research». I: *MIS Quarterly* 28.1 (2004), s. 75–105.
- [20] Andrew G. Howard mfl. «MobileNets: Efficient convolutional neural networks for mobile vision applications». I: *arXiv preprint arXiv:1704.04861* (2017). URL: <https://arxiv.org/abs/1704.04861>.
- [21] Glenn Jocher, Ayush Chaurasia og Jing Qiu. *Ultralytics YOLOv8*. Hentet februar 2026. 2023. URL: <https://github.com/ultralytics/ultralytics>.
- [22] Alex Krizhevsky, Ilya Sutskever og Geoffrey E. Hinton. «ImageNet classification with deep convolutional neural networks». I: *Advances in Neural Information Processing Systems (NeurIPS)*. Bd. 25. 2012, s. 1097–1105.
- [23] Leanpath. *Food Waste Tracking*. Hentet mars 2026. 2023. URL: <https://www.leanpath.com/products/food-waste-tracking>.
- [24] Yann LeCun, Yoshua Bengio og Geoffrey Hinton. «Deep learning». I: *Nature* 521.7553 (2015), s. 436–444.
- [25] Wei Liu mfl. «SSD: Single shot multibox detector». I: *Proceedings of the European Conference on Computer Vision (ECCV)*. 2016, s. 21–37.
- [26] NVIDIA Corporation. *Jetson Modules and Developer Kits*. Hentet februar 2026. 2024. URL: <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/>.
- [27] Douglas O'Shaughnessy. *Speech Communications: Human and Machine*. 2. utg. IEEE Press, 2008.
- [28] OASIS. *MQTT version 5.0*. Tekn. rapp. Hentet februar 2026. OASIS Standard, 2019. URL: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>.
- [29] OpenAI. *Whisper GitHub Repository*. Hentet januar 2026. 2022. URL: <https://github.com/openai/whisper>.
- [30] Alec Radford mfl. «Learning transferable visual models from natural language supervision». I: *Proceedings of the International Conference on Machine Learning (ICML)*. 2021, s. 8748–8763.
- [31] Alec Radford mfl. «Robust speech recognition via large-scale weak supervision». I: *arXiv preprint arXiv:2212.04356* (2022). URL: <https://arxiv.org/abs/2212.04356>.
- [32] Raspberry Pi Foundation. *Raspberry Pi 4 Model B specifications*. Hentet januar 2026. 2023. URL: <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/specifications/>.
- [33] Joseph Redmon mfl. «You only look once: Unified, real-time object detection». I: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, s. 779–788.
- [34] Shayan Rokhva, Babak Teimourpour og Amir Hossein Soltani. *Computer Vision in the Food Industry: Accurate, Real-time, and Automatic Food Recognition with Pretrained MobileNetV2*. Hen-

- tet mars 2026. 2024. arXiv: 2405.11621 [cs.CV]. URL: <https://arxiv.org/abs/2405.11621>.
- [35] Mark Sandler mfl. «MobileNetV2: Inverted residuals and linear bottlenecks». I: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, s. 4510–4520.
 - [36] Ken Schwaber og Jeff Sutherland. *The Scrum Guide*. Hentet januar 2026. 2020. URL: <https://scrumguides.org/scrum-guide.html>.
 - [37] Weisong Shi mfl. «Edge computing: Vision and challenges». I: *IEEE Internet of Things Journal* 3.5 (2016), s. 637–646.
 - [38] SparkFun Electronics. *Getting Started with Load Cells*. Hentet februar 2026. Lisensiert under CC BY-SA 4.0. 2023. URL: <https://learn.sparkfun.com/tutorials/getting-started-with-load-cells>.
 - [39] Mingxing Tan og Quoc V. Le. «EfficientNet: Rethinking model scaling for convolutional neural networks». I: *Proceedings of the International Conference on Machine Learning (ICML)*. 2019, s. 6105–6114.
 - [40] TensorFlow. *TensorFlow Lite documentation*. Hentet februar 2026. 2023. URL: <https://www.tensorflow.org/lite>.
 - [41] Thingiverse. *50kg Load Cell Mount*. <https://www.thingiverse.com/thing:2624188>. Hentet mars 2026. 2017.
 - [42] Aksel Tjora. *Kvalitative forskningsmetoder i praksis*. 4. utg. Gyldendal, 2021.
 - [43] Ashish Vaswani mfl. «Attention is all you need». I: *Advances in Neural Information Processing Systems (NeurIPS)*. Bd. 30. 2017, s. 5998–6008.
 - [44] Winnow Solutions. *Food Waste Management Software*. Hentet mars 2026. 2023. URL: <https://www.winnowsolutions.com/product/food-waste-management-software>.

Vedlegg A

Teknisk dokumentasjon

Fra side A1 finnes teknisk dokumentasjon for prototypen. Dette inkluderer kretsskjema for hardware-oppsettet, oversikt over elektriske tilkoblinger og komponentspesifikasjoner.

Dokumentasjonen er ment som et supplement til TotalCtrl og de som ønsker å sette opp eller videreutvikle prototypen, og gir et mer detaljert bilde av den fysiske implementasjonen enn det som er beskrevet i rapporten.

Vedlegg B

Brukermanual

Fra side B1 finnes brukermanualen for prototypen. Manualen beskriver hvordan systemet settes opp, kalibreres og startes, samt hvordan de ulike funksjonsnivåene (L1–L3) aktiveres.

Manualen er primært rettet mot TotalCtrl og andre som skal ta prototypen i bruk eller bygge videre på den.

Vedlegg C

Kildekode

I vedlegget under mappen B26ITK26_Vedlegg_C finnes kildekoden til systemet som ble utviklet i prosjektet. Koden er organisert i separate Python-moduler for hvert funksjonsnivå og tilhørende komponenter.

Kodeutsnitt fremvist i rapporten viser kun utvalgte deler av implementasjonen. Det er ikke forventet at kildekoden skal utforskes i sin helhet for å forstå prosjektet, men den ligger tilgjengelig for de som ønsker å se konkret implementasjon utover det som er vist i rapporten.

Vedlegg A - Teknisk dokumentasjon (engelsk versjon)

Introduction

This document describes the technical implementation of an IoT-based food waste registration system developed as a bachelor's project at Østfold University College in spring 2026, in collaboration with TotalCtrl. The documentation is designed to provide TotalCtrl and other developers with a complete foundation for understanding, reproducing, and further developing the solution.

The system is built around a modular architecture with four complexity levels (L1–L4), where each level can be implemented independently of the others. This makes it possible to adapt the solution to different user needs and budgets, from simple automatic weight measurement to advanced hybrid classification with both image and voice recognition.

System Architecture

The prototype is built around a Raspberry Pi 4 that functions as a central edge device. The system consists of three main layers: the *sensor layer* that collects data from the environment, the *processing layer* that handles all local data processing on the Raspberry Pi, and the *integration layer* that communicates with TotalCtrl's API via REST over HTTPS.

All sensor measurements, classification, and event logic are performed locally on the device. This reduces network load, provides lower latency, and makes the system robust in cases of unstable network connection. Only structured event data is transferred to TotalCtrl's API, not raw sensor data or images.

Modular Functionality Levels

The system is organized into four functionality levels that build upon each other:

L1 – Weight Measurement

Basic automatic weight measurement without user interaction. The IR sensor detects when waste is discarded, the weight is measured automatically, and data is sent to the API with timestamp and weight. This level requires no input from the user and operates fully automatically.

L2 – Weight and Voice

Extends L1 with voice-based categorization. After weight measurement, the microphone is activated, the user states what is being discarded, and a voice model interprets the command before the category is added to the event data. Microphone and light signal are automatically deactivated when registration is complete.

L3 – Weight and Image

Replaces voice with image recognition. When the IR sensor is triggered, the camera takes three images in rapid succession, an image model analyzes the captures and attempts to identify the waste type automatically. This level requires no active action from the user beyond holding the waste in front of the camera before discarding it.

L4 – Hybrid

The most advanced level combines both image and voice. The system first attempts to categorize via image recognition, and only asks the user to confirm or correct via voice if the model returns low confidence. L4 is not implemented in the current prototype, but the architecture is designed so that this can be realized without major changes to the core system.

Hardware Components

Raspberry Pi 4 Model B

The Raspberry Pi 4 functions as the system's brain and handles all processing, sensor reading, and network communication. The model used has 4 GB RAM and an ARM Cortex-A72 processor with 1.8 GHz clock speed. The device runs Raspberry Pi OS (Debian-based Linux) and provides access to:

- GPIO interface for direct communication with sensors
- Four USB ports for connecting camera and microphone
- Ethernet and WiFi for network connectivity
- HDMI output for external display during development and troubleshooting

Load Cells and HX711

The system uses four 50 kg half-bridge load cells with 1000 ohm strain gauges, mounted in a square formation where each cell forms a corner. The load cells are connected to an HX711 analog-to-digital converter that amplifies and digitizes the signal from the load cells into a format the Raspberry Pi can read via the GPIO interface. The HX711 offers 24-bit resolution and communicates with the Raspberry Pi through a two-wire serial interface consisting of clock and data.

The load cells are attached to the mounting plate with 3D-printed fixtures that hold the cells in place without restricting the mechanical spring effect necessary for correct weight measurement. A fixed base plate sits above the cells to ensure even weight distribution, which is critical for reliable measurements.

The HX711 is connected to the Raspberry Pi via GPIO 16 (data) and GPIO 18 (clock). Power supply to the HX711 comes from the Raspberry Pi's 5V and GND pins.

IR Sensor

An IR-08H V02.45.DC infrared proximity sensor functions as the trigger sensor for the system. The sensor detects when a user moves their hand over the waste bin and thereby initiates weight measurement and possibly image or audio recording. The sensor is mounted at the opening of the waste container so that the user always passes the sensor when something is discarded.

The IR sensor returns a digital TTL signal to the Raspberry Pi via GPIO 11: low signal when an object is detected, high signal otherwise. This makes it simple to integrate with the GPIO interface without the need for analog-to-digital conversion. The sensor's range and sensitivity can be adjusted via the built-in potentiometer on the module to adapt the detection area to the specific setup.

Camera and Microphone

For image and audio recording, a Logitech HD Pro webcam with built-in microphone is used. The camera is connected via USB and controlled via the OpenCV library in Python. The USB connection makes it easy to replace the camera if needed without having to modify the hardware setup or software.

The camera is mounted a maximum of 35 cm above the disposal point to ensure sufficient image quality without hindering normal use. A small hood is mounted over the opening to house the camera module, but does not cover the entire width of the cabinet so that the user can easily hold waste in front of the camera.

Electrical Connections

All connections to the Raspberry Pi are summarized in Table C.1.

Component	Pin (GPIO)	Function
HX711 (DT)	GPIO 16	Data from load cells
HX711 (SCK)	GPIO 18	Clock to HX711
IR sensor (OUT)	GPIO 11	Trigger signal
Camera	USB Port	Imaging
Microphone	USB Port	Audio recording

Tabell C.1: Connections to Raspberry Pi 4

The USB camera with built-in microphone functions as a single unit for both image and audio recording.

Software Architecture

Programming Language and Environment

All software is implemented in Python 3, chosen based on good library support for machine learning, GPIO communication, and HTTP integration. Development was conducted in virtual Python environments (venv) to keep dependencies isolated and reproducible across machines.

For the L3 solution, a separate virtual environment was created for image recognition, as the libraries required here could not be combined with the rest of the system without conflicts. This is solved through a subprocess-based architecture where image capture and image recognition are run as separate processes.

Core Libraries

The system uses the following Python libraries:

- **hx711**: Communication with the load cell amplifier
- **RPi.GPIO**: GPIO control for IR sensor and any relay modules
- **requests**: HTTP communication with TotalCtrl's API
- **OpenCV (cv2)**: Image capture and image handling
- **onnxruntime**: Inference of the YOLO model
- **PyAudio**: Audio recording
- **OpenAI Whisper**: Voice recognition

Code Structure

The code is divided into several separate files, each covering a distinct part of the functionality:

- `L1.py`: Main script for basic weight measurement
- `L2.py`: Main script for weight measurement with voice recognition
- `L3.py`: Main script for weight measurement with image recognition
- `image_recognition.py`: Handles YOLO-based image recognition and returns category and confidence
- `capture_images.py`: Controls the camera and takes multiple images with warm-up phase
- `config.py`: Handles file paths and settings for image recognition logic
- `voice_recognition.py`: Handles audio recording, Whisper transcription, and text filtering

This modular approach provides good reusability, where each module can be developed, tested, and maintained independently of the others. The system can also be easily adapted to different user needs by selecting the level that fits best.

Installation and Setup

Operating System

The system runs on Raspberry Pi OS (Debian-based Linux). Install the latest version of Raspberry Pi OS via Raspberry Pi Imager. Ensure that SSH is enabled if you want remote access to the device.

Virtual Environments

Create two separate virtual environments to handle dependency conflicts:

Main Environment (for L1 and L2)

```
1 python3 -m venv ~/venv_main
2 source ~/venv_main/bin/activate
3 pip install RPi.GPIO hx711 requests pyaudio openai-whisper
```

Image Recognition Environment (for L3)

```
1 python3 -m venv ~/venv_camera
2 source ~/venv_camera/bin/activate
3 pip install opencv-python onnxruntime numpy
```

Load Cell Calibration

Before first use, the load cells must be calibrated to ensure accurate measurements. The calibration process consists of two steps:

1. **Zero point calibration (offset):** Remove all objects from the scale and register the zero point as a baseline value. This is done automatically at system startup.
2. **Scaling coefficient (scale):** Place a known reference weight (for example 1 kg) on the scale and calculate the ratio between the raw value from the HX711 and the actual known weight. This value is stored as a variable in the code and used for all subsequent measurements.

The scaling coefficient may be negative depending on the orientation of the load cells and the polarity of the signals from the HX711. This is normal and is handled automatically in the calculation.

IR Sensor Calibration

The IR sensor is calibrated by adjusting the sensitivity via the potentiometer on the module. Move a hand in front of the sensor at various distances and adjust until the desired detection range is achieved. The goal is to limit detection to the area directly above the waste container opening to avoid false triggers from nearby movements.

Running the System

Starting L1 (Weight Measurement)

Activate the main environment and run the L1 script:

```
1 source ~/venv_main/bin/activate
2 python3 L1.py
```

The system will now:

1. Calibrate the zero point automatically at startup
2. Listen for trigger signal from the IR sensor
3. Measure weight change when the IR sensor detects activity
4. Store the event locally and send it to the API

Starting L2 (Weight Measurement with Voice)

Activate the main environment and run the L2 script:

```
1 source ~/venv_main/bin/activate
2 python3 L2.py
```

The system will now:

1. Calibrate the zero point automatically at startup
2. Listen for trigger signal from the IR sensor
3. Measure weight change and activate the microphone
4. Record audio for 3 seconds
5. Transcribe the audio with Whisper and extract object name
6. Store the event locally and send it to the API with category and weight

Starting L3 (Weight Measurement with Image)

L3 requires two separate processes due to dependency conflicts. The main script runs in the main environment, while image capture and image recognition are run as subprocesses in the camera environment:

```
1 source ~/venv_main/bin/activate
2 python3 L3.py
```

The system will now:

1. Calibrate the zero point automatically at startup
2. Listen for trigger signal from the IR sensor
3. Start image capture via subprocess (runs in camera environment)
4. Take three images with 0.5 second intervals
5. Analyze the images with YOLO via subprocess (runs in camera environment)
6. Measure weight change
7. Store the event locally and send it to the API with category, confidence, and weight

API Integration

Endpoint and Authentication

Communication between the prototype and TotalCtrl's platform occurs via a dedicated REST endpoint in the staging environment. The endpoint receives structured JSON messages over HTTPS and is authenticated with an API key sent in the request header (X-Api-Key).

The API key is stored in a local configuration file on the device and loaded at startup. For production environments, this should be encrypted or managed via a dedicated secrets management system.

Data Structure

Each event sent to the API follows a fixed JSON structure organized into four objects:

measurement

Contains core data for the event:

- `timestamp`: Timestamp in ISO 8601 format with timezone
- `weight_grams`: Weight in grams
- `waste_category`: Main category for the waste
- `waste_subcategory`: Possible subcategory (optional)

classification

Describes how the classification was performed:

- `method`: Classification method (image, voice, or weight)
- `confidence`: Confidence score from the model (0-100%)
- `alternatives`: Possible alternative classifications (optional)

context

Specifies context for the event:

- meal_period: Meal the event belongs to (breakfast, lunch, dinner, or other)
- waste_reason: Reason for the food waste (optional)

device

Identifies the device:

- device_id: Unique device ID
- location: Physical location (optional)
- firmware_version: Software version (optional)

Example API Request

```
1 import requests
2 import json
3 from datetime import datetime
4
5 # API configuration
6 API_URL = "https://staging.totalctrl.no/api/waste"
7 API_KEY = "your-api-key-here"
8
9 # Event data
10 event_data = {
11     "measurement": {
12         "timestamp": datetime.now().isoformat(),
13         "weight_grams": 340,
14         "waste_category": "vegetables",
15         "waste_subcategory": "carrots"
16     },
17     "classification": {
18         "method": "image",
19         "confidence": 87.5,
20         "alternatives": []
21     },
22     "context": {
23         "meal_period": "lunch",
24         "waste_reason": "overproduction"
25     },
26     "device": {
27         "device_id": "RPI4-001",
28         "location": "main_kitchen",
29         "firmware_version": "1.0.0"
30     }
```

```

31 }
32
33 # Send to API
34 headers = {
35     "X-Api-Key": API_KEY,
36     "Content-Type": "application/json"
37 }
38
39 response = requests.post(
40     API_URL,
41     headers=headers,
42     data=json.dumps(event_data)
43 )
44
45 if response.status_code == 201:
46     print("Event registered")
47 else:
48     print(f"Error: {response.status_code}")

```

Error Handling and Retry Logic

The system distinguishes between two types of errors in API communication:

Temporary errors (network errors and 5xx errors) are handled by storing the event locally and the system attempts to resend it with gradually increasing wait intervals between each attempt. This ensures that data is not lost during brief network outages.

Permanent errors (4xx errors) indicate errors in the data format and do not trigger resending. These events are logged locally for manual review.

The timestamp is always set locally on the device when the event is registered, so that the correct time is preserved even with delayed transmissions.

Local Intermediate Storage

Circular Buffer

The system always keeps the last five events stored locally in a circular buffer. When capacity is reached, the oldest event is overwritten. This ensures that data is not lost during brief network outages, while storage space on the Raspberry Pi is not overfilled.

Events are stored as separate JSON files with the following naming convention:

- waste_event1.json
- waste_event2.json

- `waste_event3.json`
- `waste_event4.json`
- `waste_event5.json`

Emptying Events

In addition to regular waste events, the system also handles emptying events for the entire waste container. When the system detects a large negative weight change (below -100 grams), this is interpreted as the waste container being removed or emptied.

The system then sends a separate event to the API with the total accumulated waste weight since the previous emptying. This information is stored in a separate file (`total_waste.json`) and provides a basis for analyzing waste volume per emptying period.

Voice Recognition (L2)

Model and Configuration

The L2 solution uses the OpenAI Whisper base model for voice recognition. The model runs locally on the Raspberry Pi and requires no network connection. Whisper was chosen because it has good support for the Norwegian language and can run offline.

Audio recording is configured with the following parameters:

- Buffer size: 1024 bytes
- Audio format: 16 bits
- Channels: Mono
- Sampling frequency: 16 kHz
- Recording time: 3 seconds (adjustable)

Text Filtering and Object Extraction

After Whisper has transcribed the audio to text, a filtering process is run to extract the object name:

1. The system checks if the text contains trigger words such as `kaster`", `kastet`", `kasta`", `koster`", or `kostet`"
2. If a trigger word is found, predefined phrases such as `"jeg kaster en"`, `kastet et`", and `"vi kaster"` are removed
3. The text is cleaned of extra spaces
4. The remaining word is returned as the object name

If no trigger word is found or no text remains after filtering, the event is registered as an L1 event without a category.

Retry Mechanism

In case of unclear speech or low confidence, the system asks the user to repeat up to three times. After three failed attempts, the event is logged as an L1 event with only weight and timestamp.

Image Recognition (L3)

Model and Configuration

The L3 solution uses a YOLOv8n detection model trained on a dataset from Roboflow. The model is exported to ONNX format and run via the `onnxruntime` library for faster inference on CPU.

The model supports recognition of 12 categories:

- | | |
|----------------|--------------|
| 1. Beetroot | 7. Eggplant |
| 2. Bell pepper | 8. Garlic |
| 3. Cabbage | 9. Onion |
| 4. Carrot | 10. Potato |
| 5. Cucumber | 11. Tomato |
| 6. Egg | 12. Zucchini |

Image Capture and Warm-up Phase

When the IR sensor is triggered, the system starts an image capture sequence:

1. The camera goes through a warm-up phase where 30 frames are read over three seconds without the images being saved. This gives the camera time to adjust exposure, white balance, and focus.
2. Three images are taken with 0.5 second intervals and saved as `waste1.jpg`, `waste2.jpg`, and `waste3.jpg`
3. The images are analyzed by the YOLO model
4. The object with the highest average confidence across all three images is returned as the result

Confidence Threshold and Fallback

The model has a confidence threshold of 60%. Detections below this threshold are discarded, and the event is instead registered as an L1 event without classification. This ensures that the system does not log misclassifications with low confidence.

Non-Maximum Suppression (NMS)

After the model has run, overlapping detections of the same object are filtered using Non-Maximum Suppression with an IOU threshold of 0.45. This ensures that each object is only counted once even if the model returns multiple overlapping bounding boxes.

Configuration and Customization

Configuration File (config.py)

The system uses a central configuration file for settings that can be changed without modifying the main code:

```
1  # File paths
2  MODEL_PATH = "models/yolov8n.onnx"
3  CLASS_NAMES_PATH = "models/classes.txt"
4  IMAGE_FOLDER = "images/"
5
6  # Image settings
7  IMAGE_SIZE = 320 # Pixels
8  CONFIDENCE_THRESHOLD = 0.6 # 60%
9  IOU_THRESHOLD = 0.45
10
11 # Weight settings
12 WEIGHT_THRESHOLD = 5 # Minimum grams for registration
13 EMPTYING_THRESHOLD = -100 # Grams for emptying event
14
15 # API settings
16 API_URL = "https://staging.totalctrl.no/api/waste"
17 API_KEY_FILE = "api_key.txt"
18
19 # Device identification
20 DEVICE_ID = "RPI4-001"
21 LOCATION = "main_kitchen"
22 FIRMWARE_VERSION = "1.0.0"
```

Meal Categorization

The system automatically determines which meal an event belongs to based on time:

- **Breakfast:** 05:00 – 09:00
- **Lunch:** 09:00 – 13:00
- **Dinner:** 13:00 – 18:00
- **Evening meal:** 18:00 – 00:00
- **Night:** 00:00 – 05:00

These time intervals can be adjusted in the code as needed to adapt to specific operating hours in different kitchen environments.

Troubleshooting and Maintenance

Common Problems and Solutions

Unstable Weight Measurements

Weight measurements that vary greatly between readings or give unexpected values can have several causes. The first check should be that all four load cells are correctly mounted and oriented in the same direction. Then the weight plate must lie flat and stable on the cells, and the cables between the load cells and HX711 must be securely attached. If the problem persists, run the calibration process again. Finally, the surroundings should be checked for vibrations that may affect the measurements.

IR Sensor Does Not Trigger

When the system does not respond to waste disposal, start by checking that the IR sensor is connected to GPIO 11 and that the power supply (5V and GND) is correctly connected. Sensitivity can be adjusted via the potentiometer on the module. Also check that the sensor is not blocked by objects or dirt. For more thorough testing, the GPIO pin can be read directly in Python to verify that the sensor is sending a signal.

Camera Produces Black Images

Black or very dark images are usually caused by too short warm-up time for the camera. Increase the warm-up time in `capture_images.py` and check the lighting conditions in the room. Consider adding a dedicated light source. Also verify that the camera is correctly connected via USB and that OpenCV can access it by testing with `cv2.VideoCapture(0)`.

Voice Recognition Performs Poorly

When Whisper transcribes wrong words or returns empty results, background noise is often the cause. Reduce the noise level in the room where the system is used, and increase the recording time to 4-5 seconds to capture more context. The user should speak more clearly and closer to the microphone. The list of trigger words and error variants in `voice_recognition.py` can be expanded for better recognition. If problems persist, an upgrade to a larger Whisper model (small or medium) can be considered if hardware permits.

API Calls Fail

Events that are stored locally but not sent to TotalCtrl usually indicate network problems or incorrect configuration. Check the network connection on the Raspberry Pi and verify that the API key in `api_key.txt` is correct. The API URL in `config.py` must point to the correct endpoint. The log file provides detailed error messages that can reveal the cause. Test the API connection manually with `curl` or Postman to isolate the problem.

Logging and Diagnostics

The system logs all events and error messages to a log file that can be used for troubleshooting. The log file contains timestamps, event types, and any error messages for each run. To read the log file in real time, use the command `tail -f system.log`. Old logs can be deleted with `rm system.log` when they are no longer needed.

Regular Maintenance

Optimal operation requires systematic maintenance at different levels. On a weekly basis, the weight plate should be checked to ensure it is clean and free of residue, the IR sensor must be verified for correct detection, and the camera lens should be checked for cleanliness.

Monthly maintenance includes recalibration of the load cells, checking all electrical connections, and updating software if new versions are available. Log files should be reviewed for unexpected errors or patterns that may indicate future problems.

Quarterly, more thorough checks are performed where the API integration is verified for correct function, all function levels (L1, L2, L3) are thoroughly tested, and it is assessed whether the models for voice and image recognition should be updated based on changed usage patterns or new datasets.

Security and Privacy

API Security

In the current implementation, the API key is stored in a local configuration file on the device. This is sufficient for a prototype environment, but production environments require stricter security measures. The API key should be encrypted or managed via a dedicated secrets management system such as HashiCorp Vault or AWS Secrets Manager. All communication occurs over HTTPS, but certificate validation should be verified explicitly in the code to prevent man-in-the-middle attacks.

When deploying multiple devices, each device must have a unique identifier and key to enable granular access control and audit trails. Token-based authentication with limited lifetime should be considered instead of static API keys, as this significantly reduces the risk of compromised keys.

Data Handling

The system handles four types of data with different privacy implications. Weight data consists of pure numerical values without personally identifiable information and represents minimal privacy risk. Audio recordings are only stored temporarily during processing and deleted immediately after transcription, so that no voice prints or audio data are preserved permanently. Images follow the same principle in that they are stored temporarily during processing and deleted after classification. Only event data in the form of structured JSON objects with timestamps, weight, and category are transferred to TotalCtrl's API.

The most important privacy benefit of this architecture is that neither images nor audio recordings are sent to TotalCtrl's API. All sensitive processing occurs locally on the edge device, and only aggregated,

structured data are transferred. This significantly reduces privacy risk compared to cloud-based solutions that send raw sensor and media data to external servers for processing.

Access Control

Production environments require strict access control to the Raspberry Pi device. SSH access must be limited to authorized administrators through the use of strong passwords or, even better, key-based authentication. Default user accounts such as `pi` should be disabled or renamed to prevent brute-force attacks against known user accounts.

Firewall (UFW) should be activated and configured to limit open ports to only what is strictly necessary for the system's function. The operating system and all libraries must be kept updated with security updates, which can be automated through `unattended-upgrades` on Debian-based systems.

Further Development

L4 – Hybrid Classification

L4 is not implemented in the current prototype, but the architecture is designed so that this can be realized without major changes to the core system. Implementation of L4 starts by running image recognition first, just as in L3. If the confidence is below a defined threshold, for example 70%, the microphone is activated so that the user can confirm or correct the category via voice. The result from both models is then combined and sent to the API with information about both the image recognition's original suggestion and the user's possible correction.

This hybrid approach provides the best of both worlds: automatic recognition when the system is confident, and user-assisted classification when uncertainty is high. Over time, the data from user corrections can also be used to improve the image recognition model through active learning.

Dedicated Light Source

To ensure consistent image quality independent of ambient lighting, a dedicated light source can be connected via relay module on GPIO. The light source is activated automatically when the camera is triggered and deactivated after recording, so that it does not disturb normal kitchen operations.

The proposed implementation uses a relay module connected to GPIO 17 that controls an LED strip or spot mounted above the disposal point. The light source is activated 0.5 seconds before image capture starts to give the camera time to adjust exposure, and is deactivated immediately after the last image is taken. This ensures optimal lighting conditions for image recognition without being energy-intensive or disruptive.

Speaker Function

A speaker can be added to provide the user with clear audio feedback on registered events. This significantly improves the user experience by having the system confirm what has been registered. Implemen-

tation requires a USB speaker or 3.5mm jack connected to the Raspberry Pi, as well as a text-to-speech library such as pyttsx3 or gTTS.

Audio confirmations can be simple messages such as "Carrots, 340 grams registered" after successful registration, or warnings in case of errors such as Please repeat for unclear voice commands or "Calibration necessary for deviations in weight measurements. This makes the system more intuitive and reduces user uncertainty about whether the event was actually registered.

Model Training on Custom Datasets

To improve recognition accuracy, the models can be trained on custom datasets adapted to specific usage environments. For image recognition, this involves collecting images of food waste from real kitchen environments where the system will be used. The images are annotated with bounding boxes and categories, and the YOLOv8n model is trained on the new dataset. After training, the model is exported to ONNX format and replaces the existing model in the system.

For voice recognition, the Whisper model can be fine-tuned on Norwegian kitchen terminology by collecting audio recordings of commands from real users. Testing with different dialects and background noise ensures that the model functions robustly in production environments. This is especially important in Norway with significant dialect variation.

Transition to Dedicated Hardware

For production environments, dedicated hardware can provide significant improvements in performance and reliability. NVIDIA Jetson Nano or Orin NX provides GPU acceleration for faster image recognition, which reduces the delay between disposal and registration. Google Coral Dev Board offers dedicated Edge TPU for highly efficient inference with low power consumption.

For the L1 solution alone, where only weight measurement and API integration are necessary, an ESP32-based solution may be sufficient. This significantly reduces cost and power consumption compared to Raspberry Pi. Industrial load cells with higher precision and robustness should be considered for production environments where accuracy and long lifespan are critical.

Vedlegg B - Brukermanual

Om denne manualen

Denne brukermanualen beskriver hvordan prototypen for automatisk matsvinnregistrering tas i bruk fra start til slutt. Manualen er rettet mot to grupper:

- **Kjøkkenpersonale** som bruker systemet i det daglige arbeidet med å registrere matsvinn.
- **Systemansvarlige** som setter opp og vedlikeholder prototypen.

Forutsetning: Det forutsettes gjennom hele manualen at prototypen allerede er *påskrudd, koblet til strøm* og *tilkoblet nettverket*. Dersom dette ikke er tilfellet, kontakt systemansvarlig før du fortsetter.

Prototype – viktig å vite

Dette er en **fungerende prototype**, ikke et ferdig produkt. Systemet har ingen lyd- eller lysfeedback til brukeren i sin nåværende form. All systemstatus er kun synlig som tekstutskrift i terminalen på Raspberry Pi, og er primært beregnet for systemansvarlig og utvikling. Høytaler og indikatorlys er planlagt som del av videre utvikling.

For daglig bruk betyr dette at du som bruker ikke får noen synlig eller hørbar bekreftelse fra selve prototypen etter at avfall er kastet. Registreringen kan verifiseres ved å sjekke terminalen, eller ved å logge inn i TotalCtrl i etterkant.

Tre kompleksitetsnivåer

Prototypen kan kjøres i tre ulike modi. Hvert nivå beskrives i en egen seksjon i denne manualen. Kun *ett* nivå kjøres om gangen, og valget gjøres av systemansvarlig ved oppstart.

Nivå	Kortnavn	Hva registreres
L1	Vekt	Vektendring og tidspunkt. Helautomatisk, ingen brukerhandling.
L2	Vekt og tale	Vektendring, tidspunkt og varenavn via talekommando.
L3	Vekt og bilde	Vektendring, tidspunkt og varenavn via bildegjenkjenning. Helautomatisk, ingen talekommando.

Oversikt over systemets komponenter

Komponent	Funksjon og plassering
Avfallsbøtten	Beholderen du kaster matavfall i. Bøtten sitter på en vektplate. Vektcellene er montert under platen og registrerer vektendringer automatisk.
Infrarød sensor	Montert ved åpningen på bøtten. Oppdager bevegelse og starter registreringen. Rekkevidde og følsomhet er forhåndsinnstilt av systemansvarlig.
Kamera	Plassert i hetten over åpningen, inntil 35 cm over kastepunktet. Brukes til bildegjenkjenning i L3.
Mikrofon	Innebygget i kameraet (Logitech). Brukes til talegjenkjenning i L2.
Raspberry Pi 4	Hjernen i systemet. Behandler all data lokalt og sender hendelser til TotalCtrl. All systemstatus vises som tekstutskrift i terminalen.
TotalCtrl-plattformen	Skybasert plattform der alle registreringer lagres og kan analyseres av kjøkkensjef og ledelse.

Tilkoblinger på Raspberry Pi 4

Tabellen under viser hvilke komponenter som er koblet til Raspberry Pi og på hvilken måte.

Komponent	Tilkobling	Funksjon
HX711 (datatilkobling)	GPIO 16	Data fra vektceller
HX711 (klokkesignal)	GPIO 18	Klokke til HX711
IR-sensor	GPIO 11	Triggersignal
Kamera med mikrofon	USB-port	Bilde og lyd

Oppstart og kalibrering

Systemet kalibrerer vektcellene automatisk ved hver oppstart. Kalibreringen tar noen sekunder og krever ingen handling fra brukeren, men stiller ett viktig krav:

Kritisk: Bøtten må være tom ved oppstart

Avfallsbøtten må være helt tom når systemet startes. Dersom det ligger avfall i bøtten under oppstart, registreres feil nullpunkt og alle etterfølgende vektmålinger vil være unøyaktige for hele kjøringen.

Tøm bøtten *før* systemet startes. Dersom systemet allerede er i gang med avfall i bøtten, kontakt systemansvarlig for recalibrering via omstart.

Slik starter systemansvarlig opp systemet

Systemet startes fra en terminal på Raspberry Pi. Velg *ett* av de tre nivåene:

Startkommandoer (for systemansvarlig)

Nivå	Kommando i terminal
L1 – kun vektmåling	<code>python3 L1.py</code>
L2 – vekt og talegjenkjenning	<code>python3 L2.py</code>
L3 – vekt og bildegjenkjenning	<code>python3 L3.py</code>

For L2 lastes talegjenkjenningsmodellen (Whisper) inn ved oppstart, noe som tar noen ekstra sekunder. Systemet melder «System ready. Waiting for events...» i terminalen når det er klart til bruk.

Hva skjer automatisk under oppstart

1. Systemet initialiserer vektcellene og HX711-omformereren.
2. Det ventes noen sekunder slik at elektronikken stabiliserer seg.
3. Systemet tar ti avlesninger og beregner gjennomsnittet som nullpunkt (offset). *Bøtten må være tom i dette steget.*
4. Nullpunktet lagres og brukes som referanse for alle videre målinger i denne kjøringen.
5. **(Kun L2)** Whisper-talemodellen lastes inn i minnet slik at gjenkjenning skjer raskt for alle påfølgende hendelser.
6. Systemet melder klart i terminalen og lytter etter hendelser fra den infrarøde sensoren.

Nivå L1 – Automatisk vektmåling

Om nivå L1

L1 er det enkleste og mest helautomatiske nivået i systemet. Du trenger ikke gjøre annet enn å kaste avfallet i bøtten. Systemet registrerer vektendringen og sender dataen til TotalCtrl uten noen form for brukerinteraksjon. Registreringen skjer stille – ingen lyd eller lys gis til brukeren. Utskrift av hendelsesdata er synlig i terminalen.

Fremgangsmåte

Slik bruker du L1

- 1. Kontroller at systemet er startet av systemansvarlig** og at terminalen viser «Waiting for events...».
- 2. Kast matavfallet i bøtten.** Den infrarøde sensoren ved åpningen oppdager bevegelsen automatisk.
- 3. Vent to til tre sekunder.** Systemet venter kort slik at avfallet roer seg ned på vektcellene, og tar deretter en serie avlesninger over tre sekunder for å beregne et stabilt gjennomsnitt.
- 4. Registreringen er fullført.** Hendelsen lagres lokalt og sendes til TotalCtrl. Utskrift i terminalen bekrefter hendelsen. Du kan fortsette arbeidet ditt umiddelbart.

Hva systemet registrerer automatisk

Felt	Forklaring
Vektendring	Differansen mellom ny totalvekt og forrige registrerte totalvekt i bøtten, oppgitt i gram.
Tidsstempel	Nøyaktig dato og klokkeslett da hendelsen inntraff.
Måltidskategori	Automatisk bestemt ut fra tidspunktet på døgnet. Se tabell C.2.
Klassifiseringsmetode	Settes til <i>vekt</i> siden ingen tale- eller bildegjenkjenning er aktiv i L1.

Tabell C.2: Automatisk måltidskategorisering etter klokkeslett

Kategori	Tidsrom	Typisk brukssituasjon
Frokost	05:00–09:00	Frokostproduksjon og -servering
Lunsj	09:00–14:00	Lunsjproduksjon og -servering
Middag	14:00–19:00	Middagsproduksjon og -servering
Kveldsmat	19:00–00:00	Kveldsservering og opprydding
Natt	00:00–05:00	Nattskift

Minimumsvekt for registrering

Systemet registrerer kun hendelser med **positiv** vektendring. Bevegelser foran sensoren uten at noe kastes, eller svært lette gjenstander, filtreres automatisk bort. For pålitelig registrering bør gjenstanden veie minst 10 gram.

Nivå L2 – Vektmåling med talegjenkjenning

Om nivå L2

L2 utvider L1 med muligheten til å si hva du kaster. Mikrofonen aktiveres automatisk i det samme øyeblikket som IR-sensoren trigges, parallelt med at vektmålingen starter. Du har et vindu på tre sekunder til å si varenavnet. Systemet gjenkjenner kommandoen og inkluderer varenavnet i registreringen.

Systemet gir **ingen hørbar eller synlig tilbakemelding** til brukeren. All statusutskrift er synlig i terminalen. Dersom talegjenkjenningen ikke identifiserer et varenavn, lagres hendelsen kun med vekt og tidspunkt, som i L1.

Fremgangsmåte

Slik bruker du L2

1. **Kontroller at systemet er startet av systemansvarlig** og viser «System ready» i terminalen.
2. **Kast matavfallet i bøtten.** Den infrarøde sensoren oppdager bevegelsen og aktiverer mikrofonen og vektmålingen *samtidig*.
3. **Si hva du kaster – umiddelbart etter at du kaster.** Du har tre sekunder på deg. Bruk gjerne en naturlig setning, for eksempel: «Jeg kaster poteter» eller «Kaster litt pasta». Terminalen viser «Si hva du kastet:» som bekreftelse på at mikrofonen lytter.
4. **Vent – systemet behandler talen.** Talegjenkjenning med Whisper tar typisk fem til ti sekunder på Raspberry Pi 4. Systemet kombinerer deretter resultatet med vektmålingen og sender hendelsen til TotalCtrl. Terminalen viser hendelsesdata når registreringen er fullført.

Responstid i L2

Talegjenkjenning lokalt på Raspberry Pi 4 tar typisk **fem til ti sekunder**. Dette er en kjent begrensning ved bruk av lokal modell på begrenset hardware, og akseptert som en avveining i prototypefasen. Brukeren kan gå videre med arbeidet mens systemet behandler i bakgrunnen.

Slik formulerer du kommandoen

Systemet benytter nøkkelord som utlødere. Kommandoen bør inneholde et ord i «kaster»-familien etterfulgt av varenavnet.

Eksempler på gyldige kommandoer

Du sier	Systemet registrerer
«Jeg kaster poteter»	poteter
«Kaster gulrøtter»	gulrøtter
«Kastet litt pasta»	pasta
«Jeg kasta gammel melk»	gammel melk
«Kosta brød» (dialektform)	brød
«Jeg kastet et egg»	egg

Systemet er også testet med kortere varianter som kun varenavnet direkte, men dette kan gi dårligere gjenkjenning. Bruk en full setning med utløserord for best resultat.

Dersom talegjenkjenningen ikke lykkes

Dersom systemet ikke klarer å trekke ut et varenavn fra talekommandoen – enten fordi bakgrunnsstøy overstyrte opptaket, fordi utløserordet manglet, eller fordi Whisper transkriberte feil – vil hendelsen automatisk lagres kun med vekt og tidspunkt, tilsvarende en L1-registrering. Det gis ingen feilmelding til brukeren. Terminalen vil vise at klassifiseringsmetoden er satt til «weight» i stedet for «voice».

Tips for god talegjenkjenning

- Snakk tydelig i normalt tempo.
- Hold deg nær mikrofonen – kameraet med innebygget mikrofon befinner seg i hetten over botten.
- Reduser bakgrunnsstøy dersom mulig. Whisper er merkbart mer nøyaktig i stille omgivelser enn i et støyende kjøkken.
- Unngå å snakke mens du fortsatt beveger hånden i eller over botten, da dette kan maskere lyden.
- Bruk alltid et utløserord som «kaster» eller «kastet» etterfulgt av varenavnet.

Hva systemet registrerer i L2

Felt	Forklaring
Vektendring	Som i L1.
Tidsstempel	Som i L1.
Måltidskategori	Som i L1, automatisk etter tidspunkt.
Varekategori	Objektnavnet tolket fra talekommandoen. Settes til «unknown» dersom ingen kategori ble identifisert.
Klassifiseringsmetode	Settes til <i>voice</i> ved vellykket gjenkjenning. Settes til <i>weight</i> dersom ingen kategori ble identifisert.

Nivå L3 – Vektmåling med bildegjenkjenning

Om nivå L3

L3 erstatter talegjenkjenning med automatisk bildeanalyse. Kameraet tar tre bilder av gjenstanden du holder foran det, og en YOLO-modell identifiserer avfallstypen automatisk. Systemet krever ingen talekommando og gir **ingen hørbar eller synlig tilbakemelding** til brukeren. All statusutskrift er synlig i terminalen.

Du må holde gjenstanden synlig foran kameraet *før* du kaster den i bøtten. Dette er fordi bildeopptak skjer sekvensielt *før* vektmålingen, siden kameraet trenger å se gjenstanden mens den fremdeles holdes i hånden.

Fremgangsmåte

Slik bruker du L3

1. **Kontroller at systemet er startet av systemansvarlig** og viser «System ready» i terminalen.
2. **Hold gjenstanden foran kameraet.** Kameraet befinner seg i hetten over åpningen på avfallsbøtten, inntil 35 cm over kastepunktet. Hold gjenstanden tydelig og stille for kameraet i ett til to sekunder.
3. **Beveg hånden mot åpningen.** Den infrarøde sensoren trigges og kameraet tar umiddelbart tre bilder med 0,5 sekunders mellomrom. Kameraet gjennomgår først en kort oppvarmingsfase (automatisk) slik at eksponering justeres.
4. **Kast gjenstanden ned i bøtten.** Etter bildeopptak venter systemet på at gjenstanden treffer vektcellene og roer seg, deretter måles vektendringen.
5. **Systemet analyserer bildene.** YOLO-modellen analyserer alle tre bildene og velger kategorien med høyest gjennomsnittskonfidens. Hendelsen lagres og sendes til TotalCtrl. Terminalen viser resultat og konfidens.

Viktig rekkefølge i L3

I L3 **må** gjenstanden holdes foran kameraet *før* den kastes. Kaster du avfallet uten å holde det foran kameraet først, vil bildene mangle og hendelsen registreres kun med vekt og tidspunkt, som i L1.

Riktig rekkefølge: **hold foran kamera → beveg mot åpning → kast i bøtten.**

Gjenstander som gjenkjennes av systemet

YOLO-modellen er trent på grønnsaker og råvarer som er vanlige i norske kjøkkenmiljøer. Tabellen under viser alle 12 støttede kategorier:

Tabell C.3: Kategorier som gjenkjennes av bildemodellen i L3

Nr	Kategori	Nr	Kategori	Nr	Kategori
1	Rødbete	5	Agurk	9	Løk
2	Paprika	6	Egg	10	Potet
3	Kål	7	Aubergine	11	Tomat
4	Gulrot	8	Hvitløk	12	Zucchini

Gjenstander utenfor de 12 kategoriene

Dersom gjenstanden ikke tilhører noen av de tolv kategoriene, eller dersom bildemodellen er for usikker (konfidens under 60 prosent), registreres hendelsen kun med vekt og tidspunkt, som i L1. Data går ikke tapt, men varenavnet vil mangle. Terminalen vil vise at klassifiseringsmetoden er satt til «weight» i stedet for «image».

For avfallstyper utenfor listen, vurder å benytte L2 (talegjenkjenning) i stedet dersom dette er tilgjengelig.

Tekniske tidsgrenser i L3

Bildeopptak og bildegjenkjenning kjøres som egne prosesser med en automatisk tidsgrense på 30 sekunder hver. Dersom en av prosessene ikke fullføres innen denne grensen, avbrytes den og hendelsen lagres kun med vekt, tilsvarende L1. Dette forhindrer at systemet låser seg og sikrer at registrering alltid fullføres.

Tips for best mulig bildegjenkjenning

- **Lysforhold:** Sørg for tilstrekkelig belysning. Dårlig lys er den vanligste årsaken til lav konfidens.
- **Avstand:** Hold gjenstanden inntil 35 cm fra kameraet for best bildekvalitet.
- **Ro:** Hold gjenstanden stille i ett til to sekunder – bevegelse gir uskarpe bilder.
- **Enkeltgjenstander:** Systemet fungerer best med én gjenstand om gangen.

Hva systemet registrerer i L3

Felt	Forklaring
Vektendring	Som i L1.
Tidsstempel	Som i L1.
Måltidskategori	Som i L1, automatisk etter tidspunkt.
Varekategori	Kategorien med høyest gjennomsnittskonfidens på tvers av de tre bildene, eller «unknown» ved lav konfidens.
Konfidens	Modellens sikkerhetsnivå (0–100 prosent) for den valgte kategorien.
Klassifiseringsmetode	Settes til <i>image</i> ved vellykket gjenkjenning. Settes til <i>weight</i> ved konfidens under 60 prosent eller ved teknisk feil i bildeprosessen.

Tømming av avfallsbøtten

Systemet oppdager automatisk når bøtten fjernes og håndterer dette uten manuell nullstilling.

Fremgangsmåte ved tømming

1. **Ta ut avfallsbeholderen** fra systemet på vanlig måte.
2. **Systemet registrerer den store vektreduksjonen** automatisk. En reduksjon på over 100 gram tolkes som at bøtten er fjernet eller tømt. Terminalen viser «TRASH CAN EMPTIED» med total akkumulert vekt.
3. **Tømmehendelsen lagres og sendes til TotalCtrl.** Total akkumulert avfallsvekt siden forrige tømming lagres med tidsstempel og måltidskategori.
4. **Sett bøtten tilbake.** Systemet rekalibrerer nullpunktet automatisk basert på den nye, tomme bøtten.
5. **Systemet er klart igjen** og venter på neste hendelse.

Akkumulert avfallsvekt

Mellom hver tømming holder systemet løpende oversikt over total akkumulert avfallsvekt. Denne overføres til TotalCtrl ved tømmehendelsen og gir grunnlag for å analysere svinnevolum per tømmeperiode.

Nettverksbrudd og lokal lagring

Systemet er designet for å tåle kortvarige nettverksavbrudd uten at registreringer går tapt.

Slik fungerer lokal lagring

Alle hendelser lagres *alltid* lokalt på Raspberry Pi *før* de sendes til TotalCtrl. Dette er en fast del av dataflyten, ikke et unntak ved feil. Dersom nettverket er utilgjengelig:

- Hendelsen beholdes i en lokal JSON-fil.
- Systemet forsøker å sende på nytt med gradvis lengre venteintervall mellom hvert forsøk.
- Når nettverket er tilbake, sendes alle ventende hendelser automatisk med de opprinnelige tidsstemplene.
- TotalCtrl vil vise korrekte data og tidspunkt selv om overføringen var forsinket.

Kapasitetsgrense for lokal lagring

Systemet lagrer de **fem siste hendelsene** lokalt i et sirkulært buffer. Ved svært langvarige nettverksavbrudd – mer enn fem nye hendelser – overskrives den eldste hendelsen. Sørg for stabil nettverks-tilgang for å unngå datatap.

Du som bruker trenger ikke gjøre noe ekstra ved nettverksbrudd. Bare fortsett å kaste avfall som normalt. Systemet håndterer synkronisering automatisk.

Se registreringer i TotalCtrl

Alle godkjente hendelser overføres til TotalCtrl sin skybaserte plattform og kan sees og analyseres der av kjøkkensjef og ledelse.

Slik ser du data i TotalCtrl

1. **Åpne TotalCtrl-plattformen** i nettleseren og logg inn med dine tilgangsinformasjoner.
2. **Naviger til matsvinnsoversikten** i menyvalget for matsvinn eller svinregistrering.
3. **Se registreringene** med varekategori, vektendring, tidsstempel og klassifiseringsmetode.
4. **Filtrer** på dato, kategori eller enhet etter behov for å identifisere mønstre i matsvinnet.

Merk om datavisning

Visning av data direkte i TotalCtrl sitt grensesnitt ferdigstilles av TotalCtrl i samarbeid med gruppen etter prosjektleveransen. Dataene overføres til stagingmiljøet og er tilgjengelige via API allerede i dag. Kontakt TotalCtrl for informasjon om tilgang og navigasjon i plattformen.

Feilsøking og vanlige situasjoner

Tabellen under beskriver de vanligste situasjonene du kan møte og hva du bør gjøre. Merk at systemet ikke gir hørbare eller visuelle feilmeldinger – all statusinformasjon er synlig i terminalen.

Symptom	Mulig årsak	Tiltak
Ingenting skjer i terminalen når du kaster avfall	IR-sensoren trigges ikke av bevegelsen	Kontroller at bevegelsen skjer rett over åpningen på bøtten. Prøv med en noe saktere bevegelse. Kontakt systemansvarlig dersom problemet vedvarer.
Terminalen viser åpenbart feil vektmålinger	Bøtten var ikke tom ved oppstart	Stopp systemet, tøm bøtten helt og start det på nytt for korrekt nullpunktskalibrering.
L2: Terminalen viser «weight» som metode i stedet for «voice»	Talegjenkjenningen klarte ikke å trekke ut et varenavn	Hendelsen er likevel registrert med vekt. Bruk en tydelig setning med «kaster» og varenavnet neste gang. Bakgrunnsstøy reduserer nøyaktigheten merkbart.
L3: Terminalen viser lav konfidens eller metoden er «weight»	Dårlig lys, for stor avstand, eller gjenstanden er utenfor de 12 kategoriene	Sørg for god belysning og hold gjenstanden stille foran kameraet. Dersom gjenstanden ikke gjenkjennes, registreres hendelsen med vekt. Vurder å bruke L2 i stedet.
L3: Terminalen viser «timeout»	Bildeopptak eller bildeanalyse brukte over 30 sekunder	Hendelsen er lagret med kun vekt. Dette kan skje ved høy systembelastning. Ingen handling nødvendig – systemet fortsetter normalt.
Data mangler i TotalCtrl	Nettverksbrudd under eller etter registrering	Data er lagret lokalt og overføres automatisk når nettverket er tilgjengelig igjen. Vent noen minutter og sjekk TotalCtrl på nytt.
Systemet reagerer på bevegelser utenfor bøtten	IR-sensorens rekkevidde er for vid	Kontakt systemansvarlig for justering av sensorens følsomhet via potensiometeret på modulen.
L2 tar lang tid etter at avfall er kastet	Whisper-modellen transkriberer lyden lokalt	Dette er normalt. Talegjenkjenning tar typisk fem til ti sekunder på Raspberry Pi 4. Systemet er i gang og registreringen fullføres automatisk.

Hurtigreferanse

Denne siden er laget som en enkel påminnelse og kan henges opp i nærheten av enheten for raskt oppslag.

Hurtigreferanse – Daglig bruk

L1 – Kun vektmåling

1. Kast avfallet i bøtten.
2. Vent to til tre sekunder.
3. Systemet registrerer automatisk og stille. Ferdig.

L2 – Vekt og talegjenkjenning

1. Kast avfallet i bøtten.
2. Si umiddelbart hva du kaster – for eksempel: «*Jeg kaster gulrøtter.*» Du har tre sekunder på deg.
3. Vent noen sekunder mens systemet behandler talen.
4. Ferdig. Sjekk terminalen eller TotalCtrl for bekreftelse.

L3 – Vekt og bildegjenkjenning

1. Hold gjenstanden foran kameraet i ett til to sekunder.
2. Beveg hånden mot åpningen – kamera tar bilder automatisk.
3. Kast gjenstanden ned i bøtten.
4. Systemet analyserer og registrerer automatisk. Ferdig.

Tømming av bøtten (alle nivåer)

1. Ta ut bøtten som normalt.
2. Systemet registrerer tømmingen automatisk.
3. Sett bøtten tilbake. Systemet recalibrerer selv.

Ingen synlig bekreftelse?

Dette er normalt for prototypen. Sjekk terminalen på Raspberry Pi, eller kontroller registreringene i TotalCtrl.

Husk: Bøtten må alltid være tom ved oppstart

Dersom avfallsbøtten ikke er tom når systemet startes, vil nullpunktskalibreringen bli feil og vektmålingene vil være unøyaktige gjennom hele kjøringen. Tøm alltid bøtten *før* systemet startes på nytt.